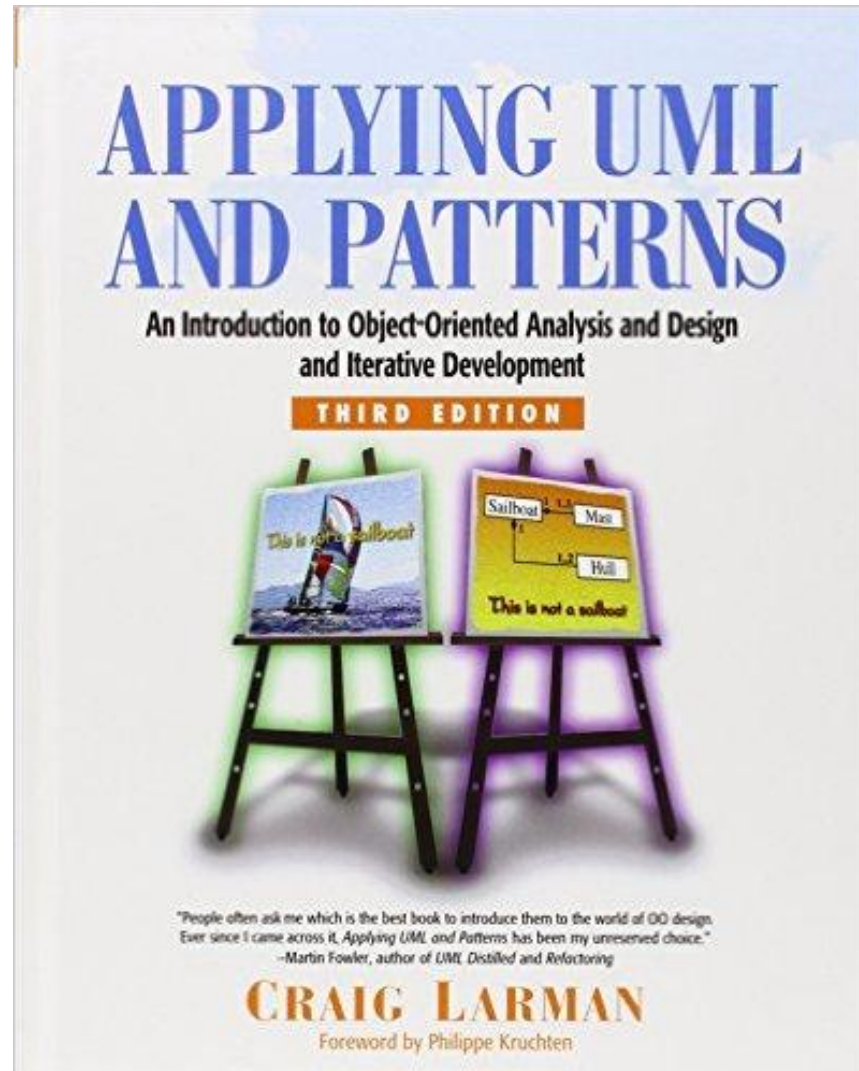


OOAD&UML

건국대학교
유준범

No.	Contents		HR
1	An Introduction to OOAD	OOAD	0.5
		UP	0.5
		OOPT	0.5
2	An Introduction to UML	UML Overview	1.0
		Case Studies with UML	0.5
3	OOA	Use Case	0.6
		System Sequence Diagram	0.4
		Domain Model	0.5
4	OOD	Sequence Diagram	1.0
		Class Diagram	1.5
5	OOI	Visibility	0.5
		Mapping Designs to Code	0.3
6	Summary		0.2



CONTENTS AT A GLANCE

PART I INTRODUCTION

- 1 Object-Oriented Analysis and Design 3
- 2 Iterative, Evolutionary, and Agile 17
- 3 Case Studies 41

PART II INCEPTION

- 4 Inception is Not the Requirements Phase 47
- 5 Evolutionary Requirements 53
- 6 Use Cases 61
- 7 Other Requirements 101

PART III ELABORATION ITERATION 1 — BASICS

- 8 Iteration 1—Basics 123
- 9 Domain Models 131
- 10 System Sequence Diagrams 173
- 11 Operation Contracts 181
- 12 Requirements to Design—Iteratively 195
- 13 Logical Architecture and UML Package Diagrams 197
- 14 On to Object Design 213
- 15 UML Interaction Diagrams 221
- 16 UML Class Diagrams 249
- 17 GRASP: Designing Objects with Responsibilities 271
- 18 Object Design Examples with GRASP 321
- 19 Designing for Visibility 363
- 20 Mapping Designs to Code 369
- 21 Test-Driven Development and Refactoring 385
- 22 UML Tools and UML as Blueprint 395

PART IV ELABORATION ITERATION 2 — MORE PATTERNS

- 23 Iteration 2—More Patterns 401
- 24 Quick Analysis Update 407
- 25 GRASP: More Objects with Responsibilities 413
- 26 Applying GoF Design Patterns 435

PART V ELABORATION ITERATION 3 — INTERMEDIATE TOPICS

- 27 Iteration 3—Intermediate Topics 475
- 28 UML Activity Diagrams and Modeling 477
- 29 UML State Machine Diagrams and Modeling 485
- 30 Relating Use Cases 493
- 31 Domain Model Refinement 501
- 32 More SSDs and Contracts 535
- 33 Architectural Analysis 541
- 34 Logical Architecture Refinement 559
- 35 Package Design 579
- 36 More Object Design with GoF Patterns 587
- 37 Designing a Persistence Framework with Patterns 621
- 38 UML Deployment and Component Diagrams 651
- 39 Documenting Architecture: UML & the N+1 View Model 655

PART VI SPECIAL TOPICS

- 40 More on Iterative Development and Agile Project Management 673

OOAD

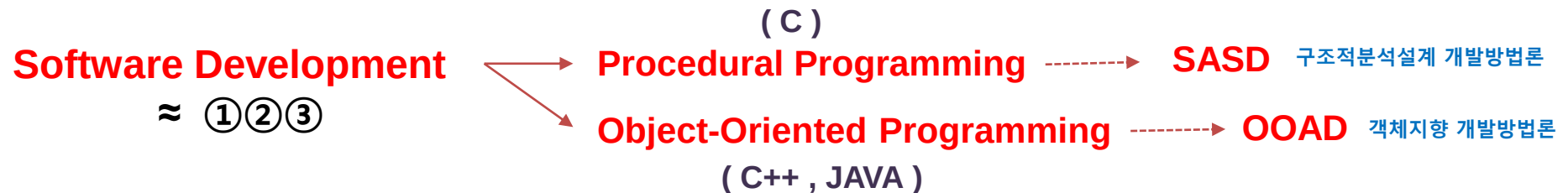
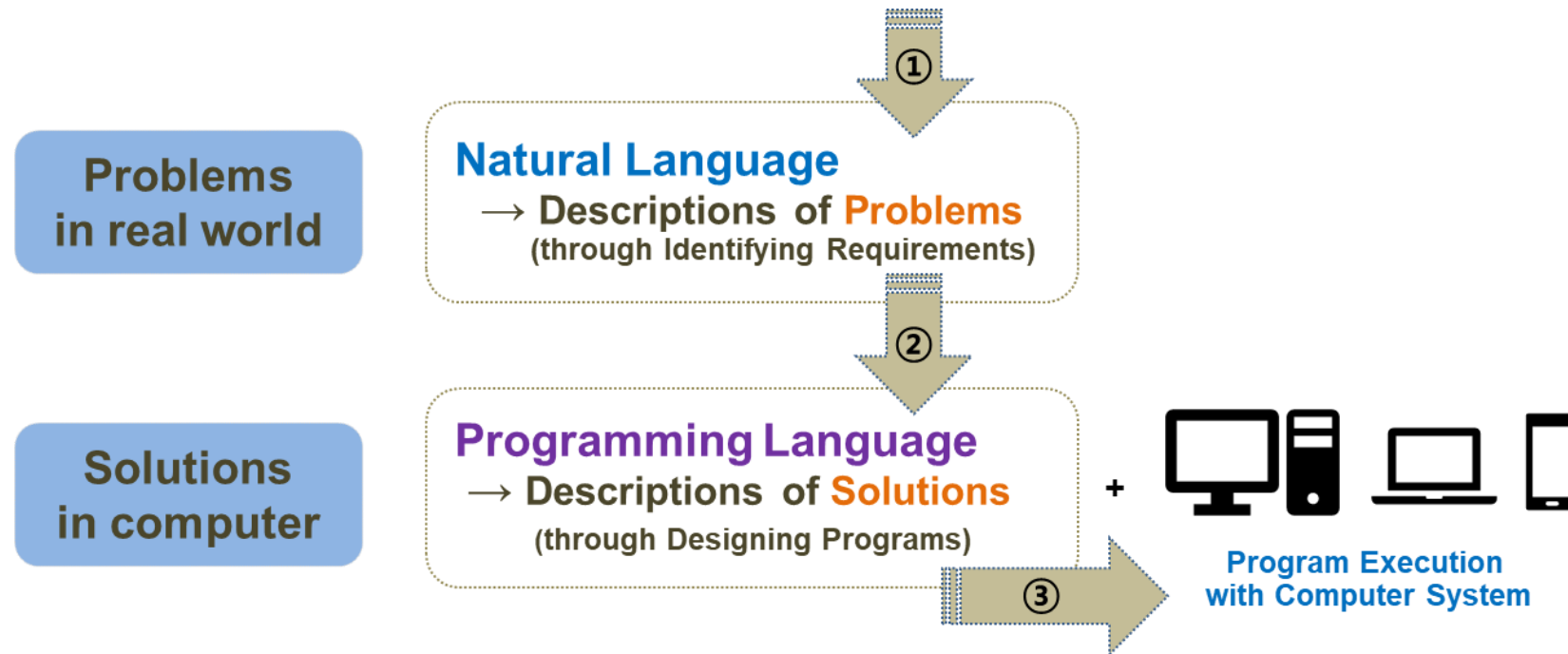
Design Patterns

Architecture
Style

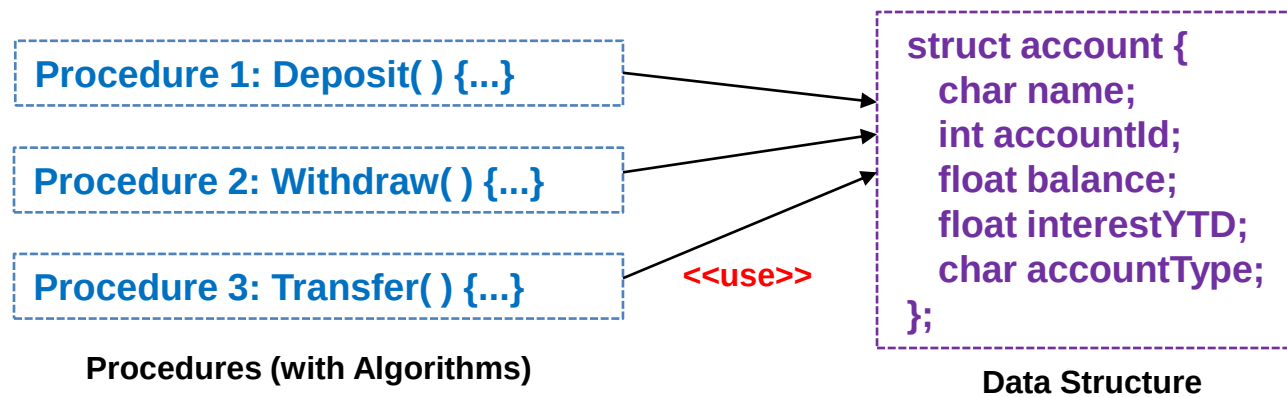
Architecture
Description

An abstract geometric background featuring a solid blue field. In the upper right, there is a large, semi-transparent blue circle. Overlapping its top edge is a smaller circle with diagonal white and blue stripes. Below the striped circle is a small, solid white sphere. In the lower left, a large, semi-transparent blue circle is partially visible. A solid white sphere is positioned on the edge of this circle. In the lower right, there is a 3x3 grid of small white dots. On the far left, there are faint, light blue diagonal lines.

- Software Development $\approx$ Solving Problems with Software working on a Computer



- A program is organized with **procedures**.
 - **Procedure/Function**
 - Building-block of procedural programs
 - **Statements** changing values of **variables**
 - Focusing on data structures, algorithms, and sequencing of steps
 - **Algorithm** : a set of instructions for solving a problem (Control-Centric)
 - **Data structure** : a construct used to organize data in a specific way (Data-Centric)
 - Most computer languages (from FORTRAN to **c**) are procedural ones.



A Procedural Program



```
*polynomial.c - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

#include <stdio.h>
#include <stdlib.h>

int p1Coef, p1Degree, p2Coef, p2Degree;

//다항식 하나를 list로 보고, 다항식에 들어가는 각 항을 해당 list의 node로 본다.

typedef struct Term {
    int coef; //각 항의 계수
    int degree; //각 항의 차수
    struct Term *next; //next term을 알 수 있는, 단순연결리스트
} Term;

typedef struct polynomial {
    Term *head;
    Term *tail;
} Polynomial;

void addTerm(Polynomial *p, int coef, int degree) {
    //임의의 term이 자신의 다음에 오는 next term만 알 수 있으므로 새로 입력되는 항은 앞에 덧붙인다
    Term* termNew;
    Term* temp;

    termNew = (Term *)malloc(sizeof(Term));
    termNew->coef = coef;
    termNew->degree = degree;
    termNew->next = NULL;

    if (p->head == NULL) {
        p->head = termNew;
    }
    else {
        temp = p->head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = termNew;
    }
}

void tSort(Polynomial *p) {
    int temp;
    //다항식을 입력받은 후 차수가 높은 것부터 낮은 것까지 정렬한다
    Term *term = (Term *)malloc(sizeof(Term));
    term = p->head;
    while ((term->next != NULL) {
        if (p->head->degree > p->head->next->degree) {
```

```
*polynomial.c - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

int main() {

    Polynomial *p1 = (Polynomial *)malloc(sizeof(Polynomial));
    Polynomial *p2 = (Polynomial *)malloc(sizeof(Polynomial));
    Polynomial *p3 = (Polynomial *)malloc(sizeof(Polynomial));

    while (1) {
        printf("\n*****\n");
        printf("첫 번째 다항식에 들어갈 각 항의 계수와 지수를 순서대로 입력하세요\n");
        printf("계수에 0을 입력할 경우 더이상 입력 받지 않습니다\n");
        printf("*****\n");

        p1->head = NULL;
        do {
            scanf_s("%d %d", &p1Coef, &p1Degree);
            if (p1Coef != 0) {
                addTerm(p1, p1Coef, p1Degree);
            }
        } while (p1Coef != 0);

        printf("\n*****\n");
        printf("두 번째 다항식에 들어갈 각 항의 계수와 지수를 순서대로 입력하세요\n");
        printf("계수에 0을 입력할 경우 더이상 입력 받지 않습니다\n");
        printf("*****\n");

        p2->head = NULL;
        do {
            scanf_s("%d %d", &p2Coef, &p2Degree);
            if (p2Coef != 0) {
                addTerm(p2, p2Coef, p2Degree);
            }
        } while (p2Coef != 0);

        printf("\n A(x)=");
        printPoly(p1);
        printf("\n B(x)=");
        printPoly(p2);

        p3->head = NULL;
        addP(p1, p2, p3);

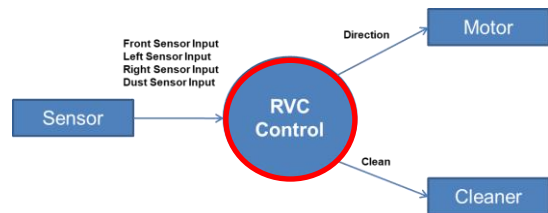
        printf("\n C(x)=");
        printPoly(p3);

    }
}
```

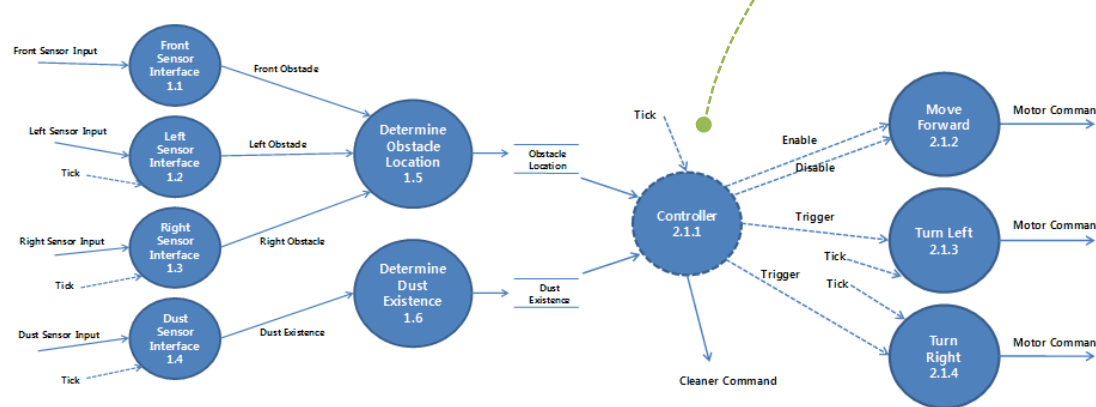
Structured Analysis and Structured Design



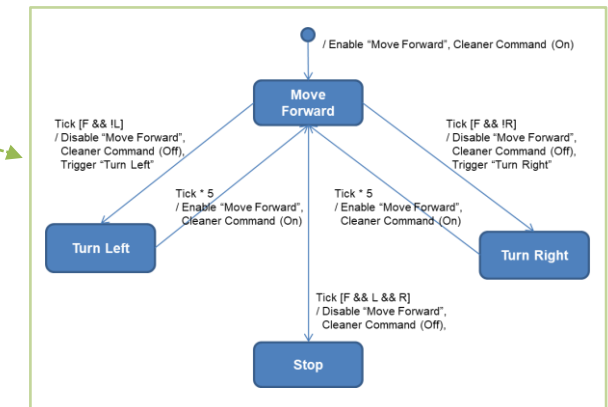
- **SASD** (Structured Analysis and Structured Design)
 - A traditional software development methodology for procedural programs like C
 - **Structural modeling** in **Top-Down Divide & Conquer** manner
 - Divide large, complex problems into smaller, more easily handled ones
 - Functional view of the problem using **DFD** (Data Flow Diagram)
 - Control functionalities using **FSM** (Finite State Machine)



DFD Level 0

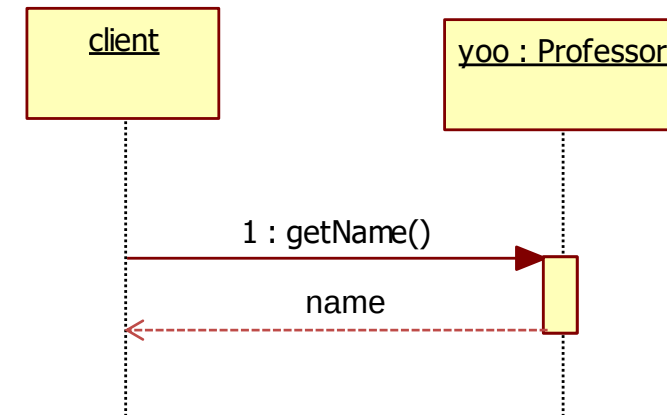
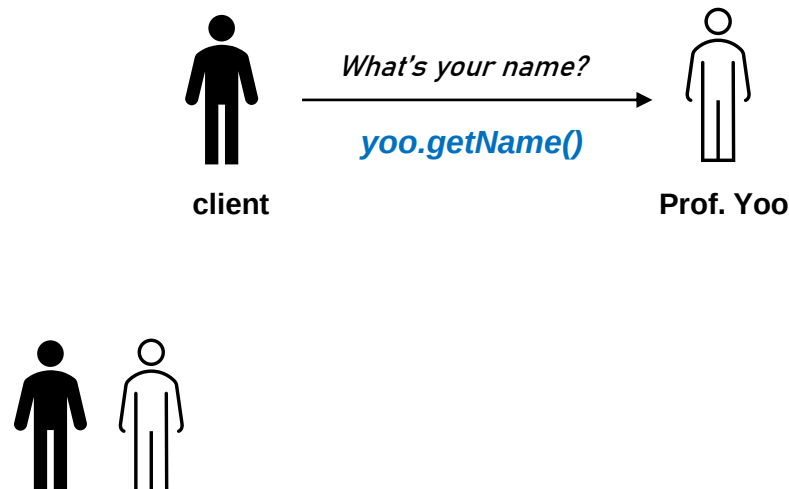
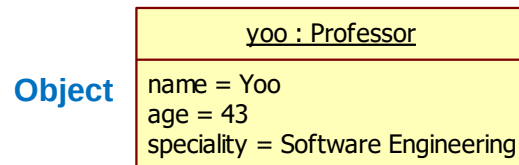
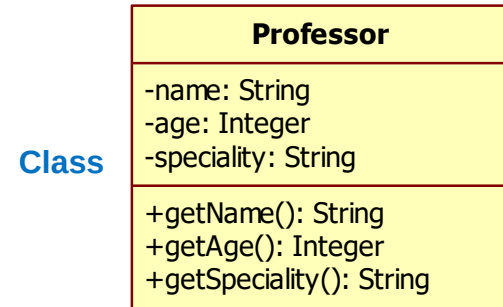


DFD level 3



An FSM for 2.1.1 Controller

- A program is organized with **objects**.
 - **Objects**
 - Consisting of **data** and **operations**
 - *Encapsulation (→ Information Hiding), Inheritance, Polymorphism*
 - Providing system functionalities through **object communications**
 - **Object communication** : an object **calls** public operations of other objects with its data



```
Factorials.java - Windows 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
import java.util.Scanner;

public class Factorials {
    static int fac;

    public static int recursiveFactorial(int n) {
        if (n == 0) return 1;
        else return n*recursiveFactorial(n-1);
    }

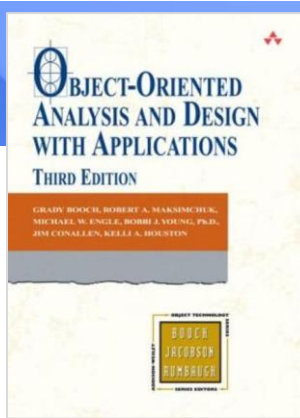
    public static void main(String[] args) {
        while(true) {
            System.out.println("계산하고자 하는 factorial 값을 입력하세요.");
            Scanner keyboard = new Scanner(System.in);
            if(keyboard.hasNextInt()) {
                fac = keyboard.nextInt();
            }else {
                System.out.println("잘못된 입력입니다! 프로그램을 종료합니다.");
                break;
            }

            System.out.print( fac + "! = ");

            if(fac == 0) {
                System.out.println("0! = 1");
            }else {
                int i = fac;
                while(i > 1) {
                    System.out.print(i + "*");
                    i--;
                }

                System.out.println("1 = " + recursiveFactorial(fac) + "\n");
            }
        }
    }
}
```

Object-Oriented Analysis and Design (OOAD)



- **OOAD** (Object-Oriented Analysis and Design)
 - *“Identifying your requirements and creating a domain model, and then add methods to the appropriate classes, and define the messaging between the objects to fulfill the requirements”*
 - **Object-Oriented Analysis (OOA)**
 - Identify requirements → **Use-Case model**
 - Discover the domain concepts/objects → **Domain model**
 - **Object-Oriented Design (OOD)**
 - Define software objects (static model) → **Class diagram**
 - Define how they collaborate to fulfill requirements (dynamic model) → **Sequence diagram**
 - Applicable to various development process models.
 - **Waterfall**
 - **UP (Iterative, Agile)**
- But it is not enough, because OOAD involves **deep principles**.
 - Deciding what methods belong to where and how objects should interact carries consequences should be undertaken seriously.
 - **OOAD** should be a **dynamic model-driven approach** to avoid “**maintenance nightmare**”



*Maintenance
Nightmare*

An OOAD Example - Dice Game



Define use cases

Define domain
model

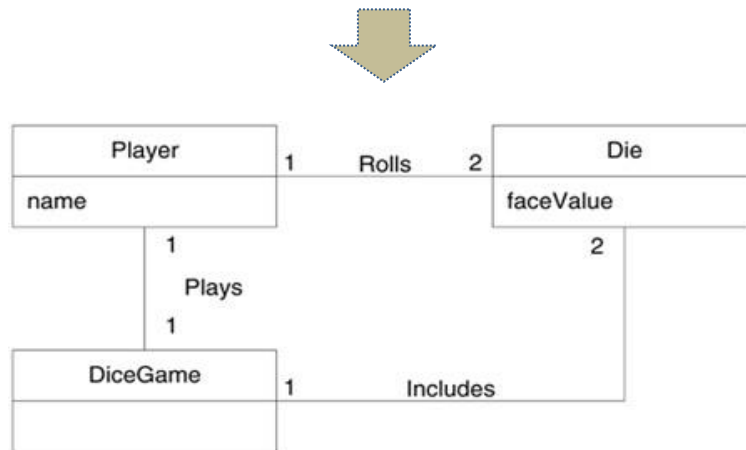
Define interaction
diagrams

Define design class
diagrams

OOA

Use Case : **Play a Dice Game**

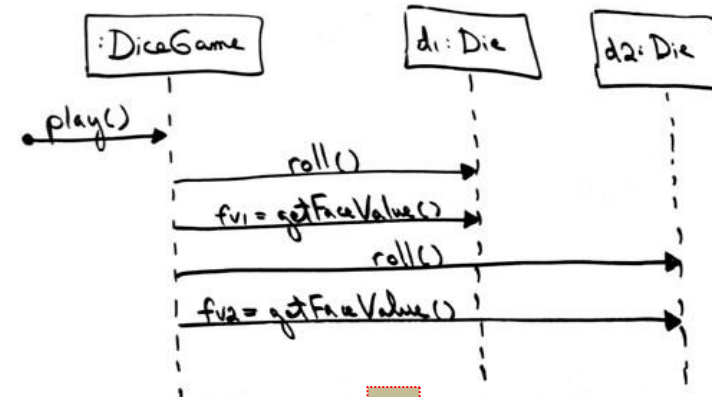
- Player requests to roll the dice.
- System presents results.
- If the dice's face value totals seven, player wins; otherwise, player loses.



Domain Model

OOD

Interaction Diagram (Sequence Diagram)



Dynamic model-driven !!!



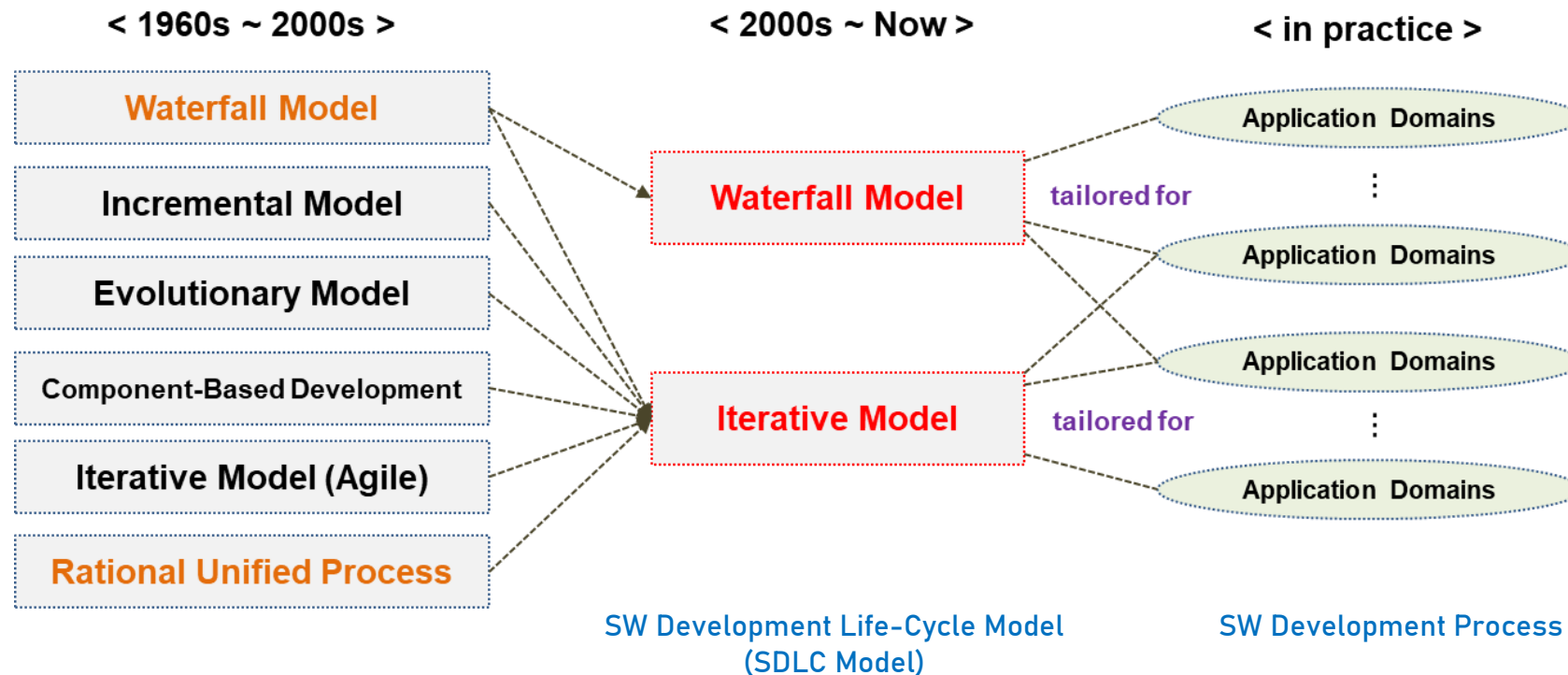
Design Class Diagram

A decorative blue sidebar on the left side of the slide. It features several geometric elements: a large dark blue circle at the bottom left, a smaller white circle with a blue shadow above it, a white circle with a blue shadow further up, a circle with white diagonal stripes in the top right, and a 3x3 grid of small white dots in the bottom right.

An Introduction to OOAD - UP

- **Software (Development) process models**

- Defining **Who** is doing **What**, **When** to do it, **How** to reach a certain goal.
- A process is a finely and thoroughly tailored result of a process model.



- **Agile development** is **an umbrella terminology** - a group of methodologies weighting rapid prototyping and rapid development experiences.
 - Lightweight in terms of documentation and process specification
 - Example: XP(eXtreme Programming), TFD(Test First Development), Pair Programming, Refactoring, CI/CD, Scrum, Sprint
- Agile methods attributes
 - **Iterative** (several cycles)
 - **Incremental** (not delivering the product at once)
 - Actively involve **users** to establish requirements
- **Agile Manifesto**
 - Individuals and interactions **over** processes and tools
 - Working software **over** comprehensive documentation
 - Customer collaboration **over** contract negotiation
 - Responding to change **over** following a plan

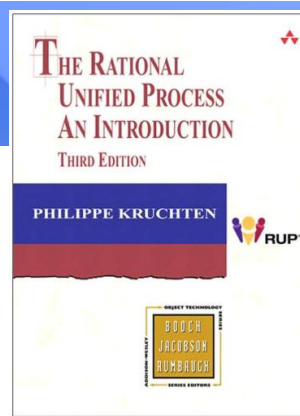
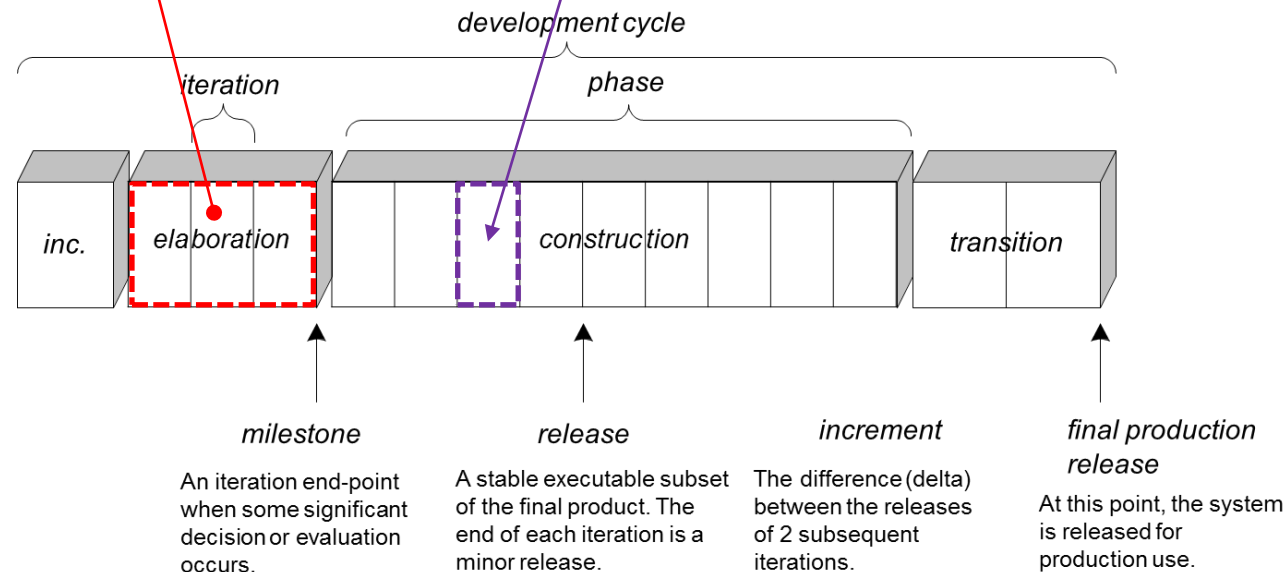
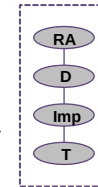


<http://agilemanifesto.org>

UP (Unified Process)

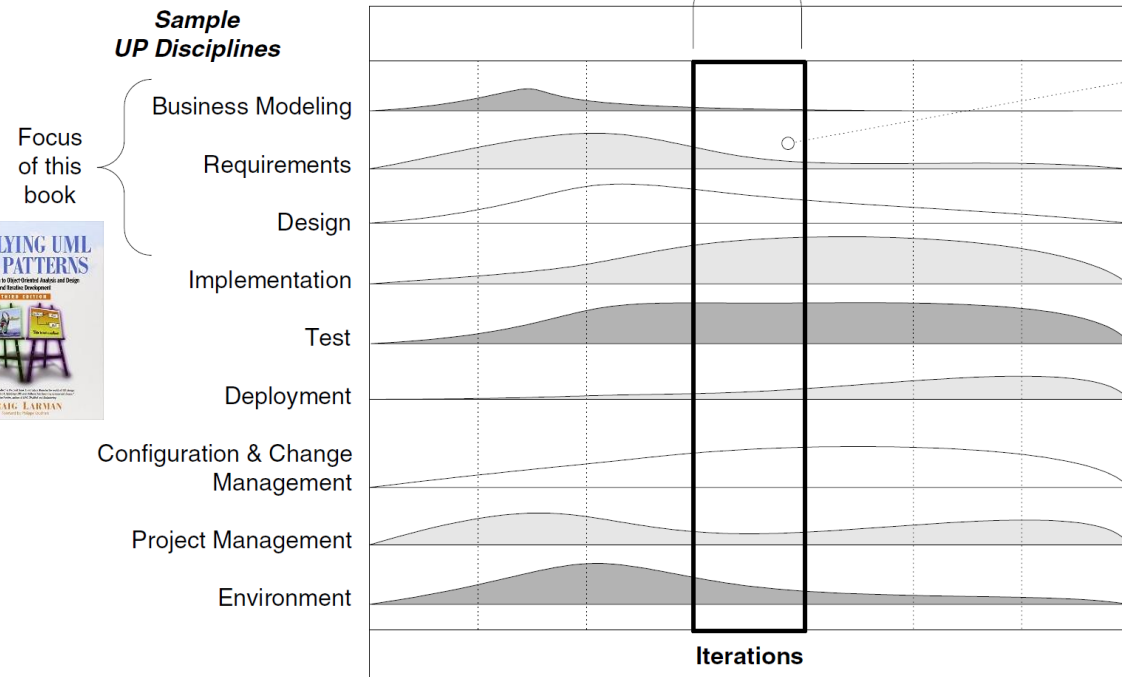
- **Rational Unified Process (RUP) or UP**

- A Software development approach that is
 - **Iterative** (Incremental, Evolutionary)
 - Each iteration includes a mini waterfall cycle of 3 ~ 4 weeks
 - **Risk-driven / Client-driven / Architecture-centric**
 - **Use-case driven**
- A Well-defined and well-structured software engineering process
 - **4 Phases** and **9 Disciplines**
- A de-facto industry standard for developing OO software



- **Development Case**
 - Documenting **the choice** of practices and UP artifacts for a project

A four-week iteration (for example).
A mini-project that includes work in most disciplines, ending in a stable executable.



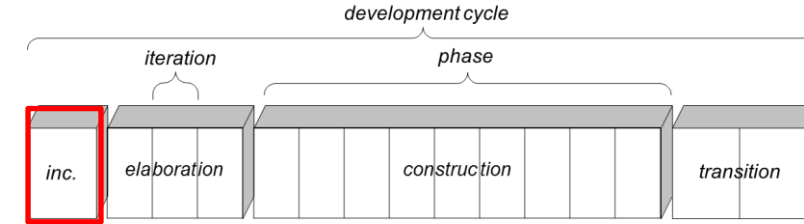
Sample Unified Process Artifacts and Timing (s-start; r-refine)

Discipline	Artifact	Incep. I1	Elab. E1..En	Const. C1..Cn	Trans. T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	
Implementation	Implementation Model (code, html, ...)		s	r	r

UML for Requirements Model
- Use-Case (+System Sequence Diagram)

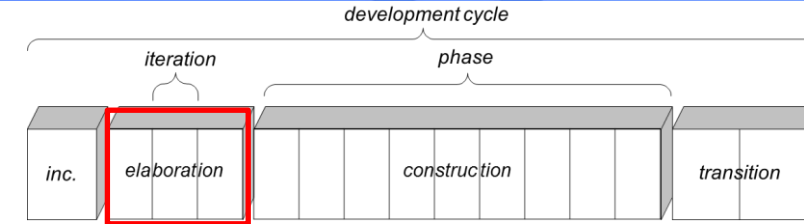
UML for Design Model
- Class Diagram, Sequence Diagram
- Package Diagram, Component Diagram, Deployment Diagram
- Statechart Diagram, Activity Diagram

- **Inception is a short (only one week) step** to elaboration including:
 - A short [requirements workshop](#) and [feasibility study](#)
 - Most actors, goals, and use cases named
 - [Most use cases written in brief format](#) (10~20% are written in fully dressed detail)
 - [Most influential and risky requirements identified](#)
- Important artifacts starts in Inception :
 - **Use-Case model** : **Functional** requirements
 - Supplementary specification : All other (**Non-functional, Quality**) requirements
 - No specific detailed model provides.



Discipline	Artifact Iteration→	Incep. I1	Elab. E1..En	Const. C1..Cn	Trans. T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		

- **Elaboration** is the initial series of iterations during which:
 - The core, risky software **architecture** is programmed and tested.
 - The majority of requirements are discovered and stabilized.
 - The major **risks** are mitigated or retired.
- Important artifacts start in Elaboration :
 - **Domain model** : **Domain objects/concepts**
 - **Design model** : **Design class diagram, Sequence diagram**
 - SW Architecture Document (AD)



Discipline	Artifact	Incep.	Elab.	Const.	Trans.
	Iteration →	I1	E1..En	C1..Cn	T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model (SSDs)	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	

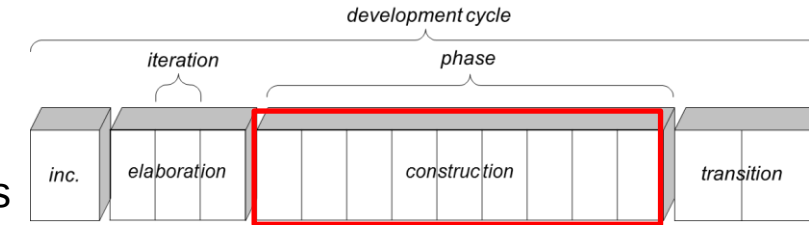
UML Requirements Model

- Use-Case Model
- System Sequence Diagram

UML Design Model

- Design Class Diagram
- Sequence Diagram

- **Construction** is a series of iterations including
 - [Incremental implementation](#) of the remaining lower risk and easier elements
 - Preparation for deployment in the transition phase
 - Applying TDD and Refactoring
 - A typical incremental and iterative development, but not evolutionary

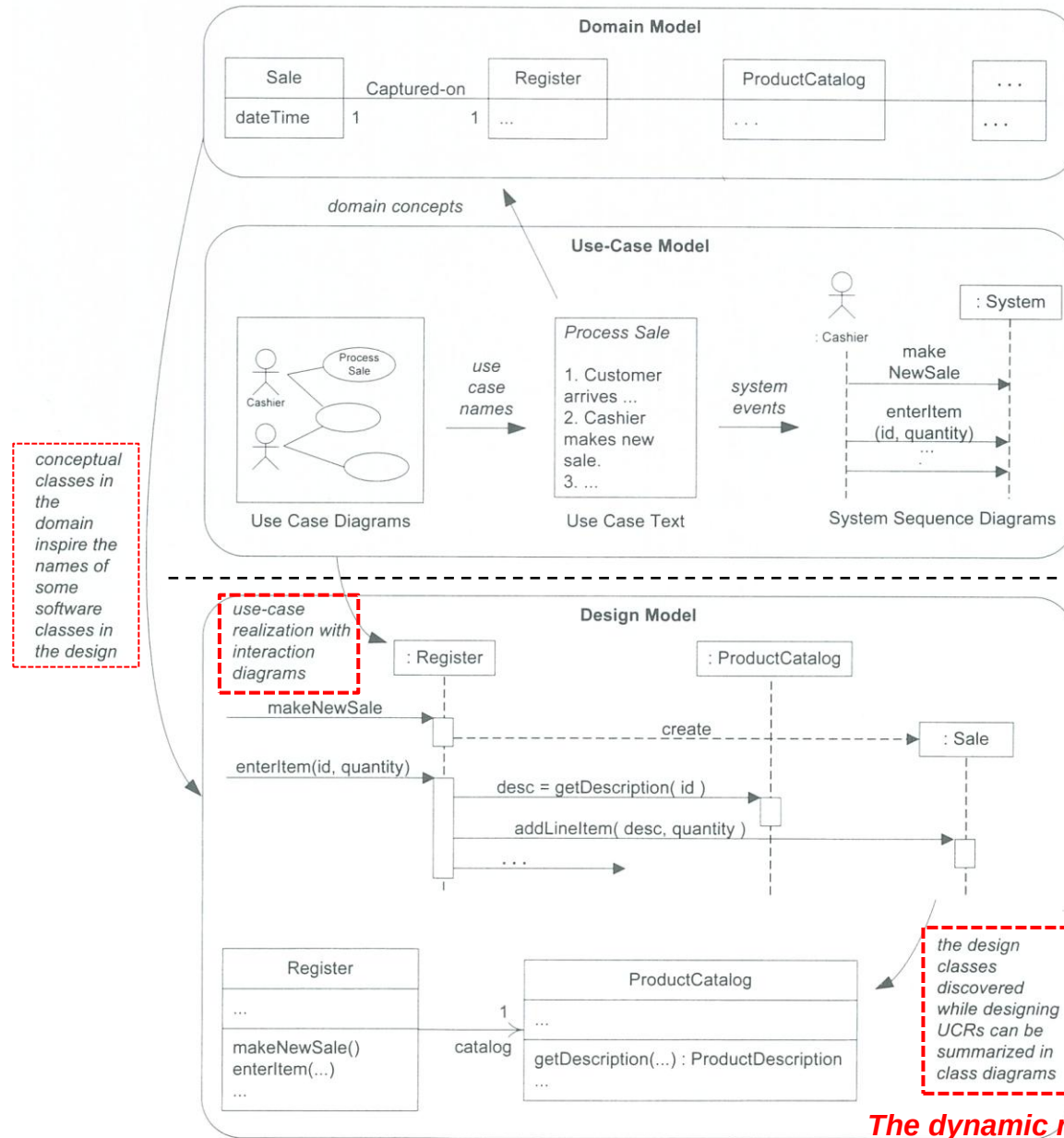


Discipline	Artifact	Incep.	Elab.	Const.	Trans.
	Iteration →	I1	E1..En	C1..Cn	T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model (SSDs)	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	

UML Design Model

- + Design Class Diagram
- + Sequence Diagrams
- + Package Diagram
- + Component Diagram
- + Deployment Diagram

Sample Unified Process Artifact Relationships



Domain Model

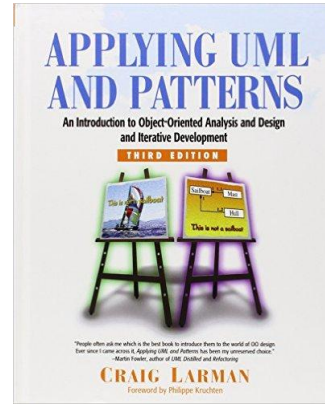
Use-Case Model
(+System Sequence Diagram)

OOA
OOD

Sequence Diagrams

Class Diagram

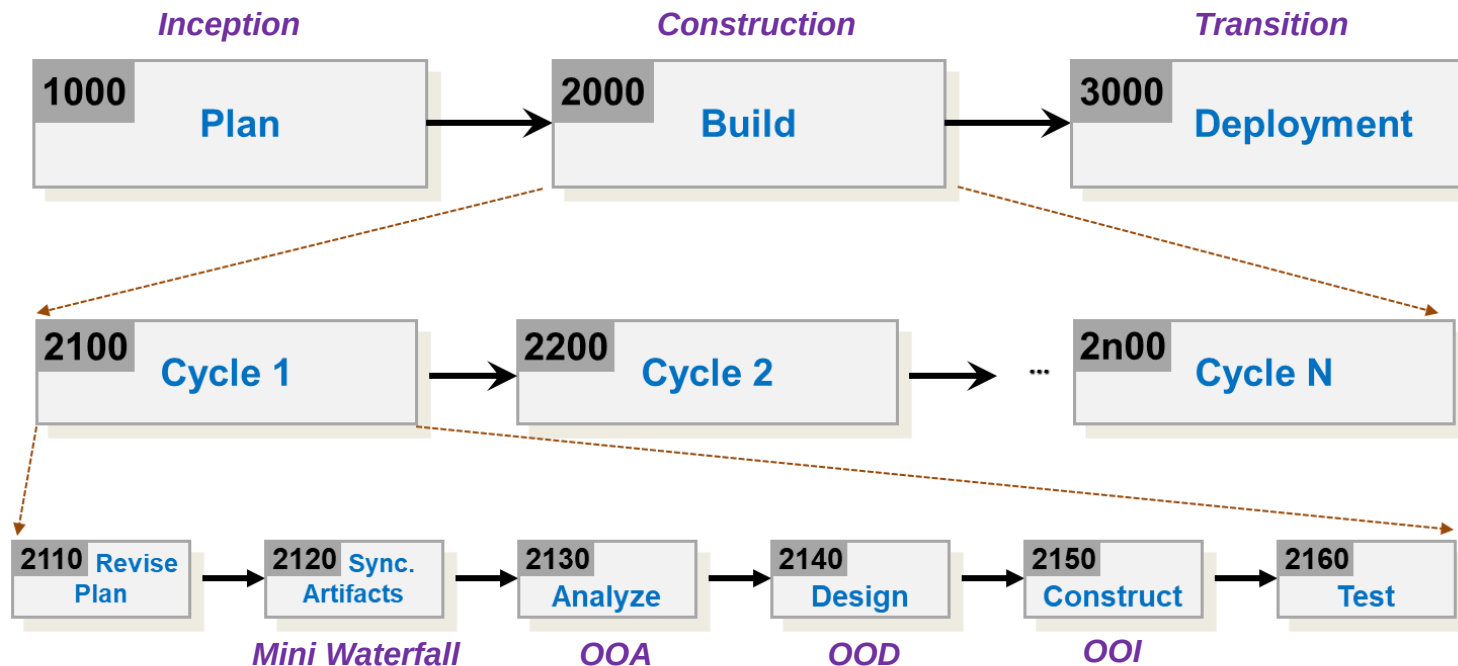
The dynamic model-driven approach!!!



A decorative blue sidebar on the left side of the slide. It features several geometric elements: a large dark blue circle at the bottom left, a smaller white circle with a blue shadow above it, a white circle with a blue shadow further up, a circle with white diagonal stripes in the top right, and a 3x3 grid of small white dots in the bottom right.

An Introduction to OOAD - OOPT

- An object-oriented software development process **based on UP**
 - Tailored to software engineering classes in universities
 - **No risk analysis** for software architecture : **No elaboration phase** of UP
 - All activities are the same as the UP, but organized into a series of artifacts provided with templates.
 - Consisting of iterations of mini waterfalls as the UP Constructions phase
 - Additionally focus on traceability from requirements to test cases



ISSN 2383-630X(Print) / ISSN 2383-6296(Online)
Journal of KIISE, Vol. 44, No. 5, pp. 510-521, 2017. 5
<https://doi.org/10.5626/JOK.2017.44.5.510>

OOPT: 소프트웨어공학 교육을 위한 객체지향 소프트웨어 개발 방법론

(OOPT: An Object-Oriented Development Methodology for
Software Engineering Education)

정 세 진 * 이 동 아 * 김 의 섭 * 장 천 현 ** 유 준 범 ***
(Sejin Jung) (Dong-Ah Lee) (Eui-Sub Kim) (Chun-Hyon Chang) (Junbeom Yoo)

요 약 소프트웨어 개발 프로세스(Software Development Process: SDP)는 소프트웨어공학 교육에서 가장 기초적이며 중심적인 역할을 한다. 모든 소프트웨어는 개발의 시작부터 마지막까지를 모두 포함하는 특정 SDP에 기반해서 개발된다. 따라서, SDP 교육은 소프트웨어공학의 제반 기술에 대한 이해를 도울 수 있다. 본 논문은 대학의 소프트웨어공학 수업에서 활용할 수 있는 소프트웨어 개발 방법론(프로세스)인 OOPT(Object Oriented Process with Traceability)를 소개한다. OOPT는 객체지향 소프트웨어를 개발하기 위한 방법론으로서, 각 단계마다 구체적인 요구사항과 산출물을 정의하고 있으며, 단위/시스템 시험 및 추적성 분석 등의 추가적인 내용들도 포함하고 있다. 본 논문은 OOPT에 대한 적용 사례로서 다년간의 건국대학교 컴퓨터공학과 소프트웨어공학 관련 수업들을 소개하고 있으며, 향후 개선 및 발전 방향을 포함한다.

키워드: OOPT, 소프트웨어공학 교육, 객체지향 개발방법론, 소프트웨어 개발방법론

- The **Inception** phase of UP
 - Project planning

Functional Requirements	Use Cases	Category
R1.1 Make reservation	1. Make Reservation	Evident
R1.2 Remove reservation	2. Remove Reservation	Evident
R1.3 Lend Item	3. Lend Item	Evident
R1.4.1 Return title	4. Return Title	Evident
R1.4.2 Calculate Late-Return-Fee	5. Calculate Late-Return-Fee	Hidden
R1.5 Calculate Replacement Fee	6. Get Replacement Fee	Evident
R1.6 Notify Availability	7. Notify Availability	Hidden
R2.1 Add title	8. Add Title	Evident
R2.2 Remove title	9. Remove Title	Evident
R2.3 Update title	10. Update Title	Evident
R2.4 Add items	11. Add Item	Evident
R2.5 Remove item	12. Remove Item	Evident
R2.6 Update item	13. Update Item	Evident
R3.1 Add borrower	14. Add Borrower	Evident
R3.2 Remove borrower	15. Remove Borrower	Evident
R3.3 Update borrower	16. Update Borrower	Evident
R4.1 Validates system access	17. Log-IN	Evident
R4.2 Validates system access	18. Log-Out	Evident
R5.1 Compute total # of items checked out	19. Count Loans	Evident

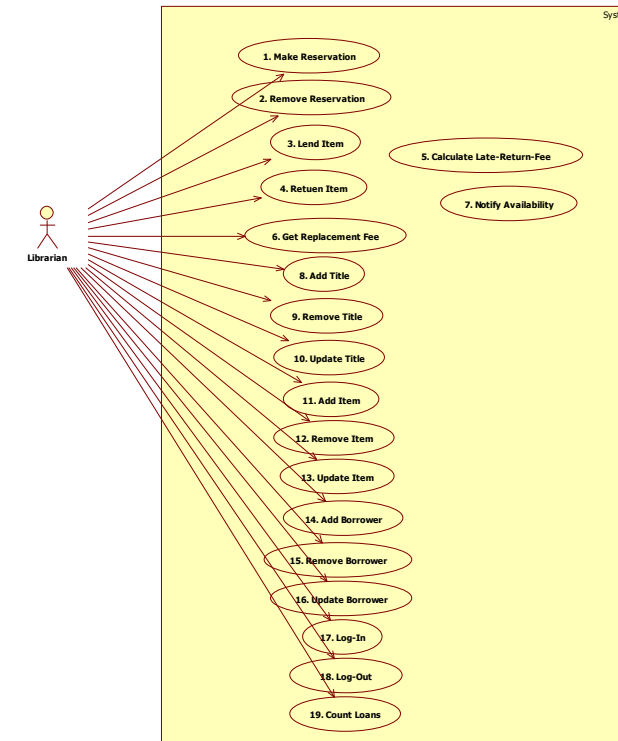
Functional Requirements $\approx$ Use Cases

*Identified, Derived, Analyzed and Organized
by*

*Requirements Elicitation & Analysis
at Requirements Analysis Step
in case of Waterfall Model*

Use Case Description (Brief format)

Use Case	1. Make Reservation
Actors	Librarian
Description	- This use case begins when a borrower arrives at the counter and then requests reservation. - For a registered borrower, it makes a reservation slip (software-wise). - For an unregistered borrower, the librarian registers the person and makes a reservation for the person.



Use Case Diagram

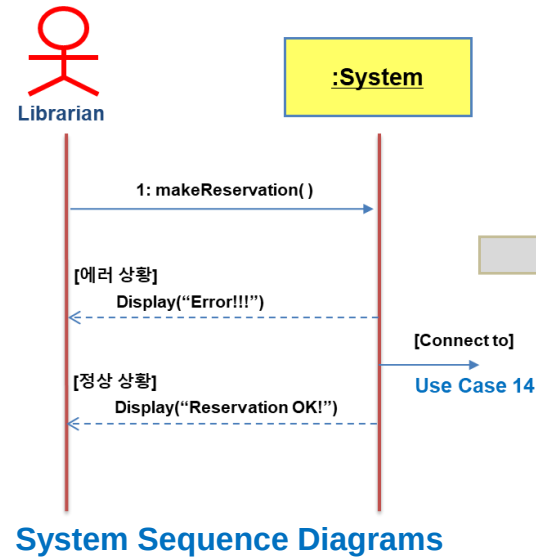
• OOA (Object-Oriented Analysis)

USE CASE: 1. Make Reservation

1. (A) A librarian requests the reservation of title.
2. (S) Check if corresponding title exist.
3. (S) Check if corresponding borrower exist.
4. (S) If the borrower does not exist, invoke "Add Borrower".
(→ connect to other Use Case)
5. (S) Create reservation information.

Use Cases (Casual)

for each use case



System Sequence Diagrams

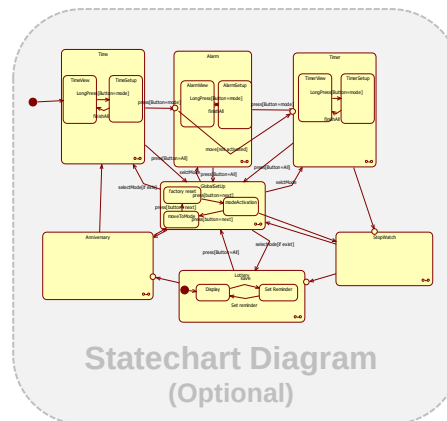
System Operations (System Interface)

```

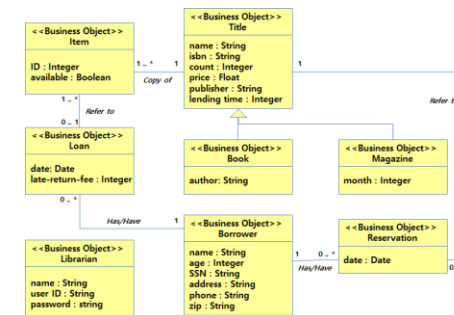
System
+selectTimeViewMode()
+selectTimeSetupMode()
+changeValue()
+goNext()
+selectAlarmViewMode()
+selectAlarmSetupMode()
+addAlarm()
+deleteAlarm()
+clearAlarmNotice()
+setValue()
+startTimer()
+pauseTimer()
+resetTimer()
+clearTimerNotice()
+startStopWatch()
+stopStopWatch()
+restartStopSwatch()
+resetStopSwatch()
+createNewAnniversary()
+inputDateTime()
+selectAnniversary()
+deleteAnniversary()
+alert()
+dismiss()
+requestCreateLotteryNumber()
+saveLotteryNumber()
+setReminder()
+select4Mode()
+requestFactoryReset()
+requestChangeCurrentMode()
    
```

System Function	Source Use Case	Destination in sequence diagram
Make reservation	→ Make Reservation	→ makeReservation()
Remove reservation	→ Remove Reservation	→ removeReservation()
Lend item	→ Lend Item	→ lendItem()
Return item	→ Return Item	→ returnItem()
Calculate Late-Return Fee	→ Calculate Late-Return Fee	→ getReservationFee()
Calculate Reservation Fee	→ Calculate Reservation Fee	→ getReserveFee()
Apply Availability	→ Apply Availability	→ applyAvailability()
Apply Availability	→ Apply Availability	→ applyAvailability()
Update title	→ Update Title	→ updateTitle()
Update item	→ Update Item	→ updateItem()
Remove item	→ Remove Item	→ removeItem()
Update item	→ Update Item	→ updateItem()
Add borrower	→ Add Borrower	→ addBorrower()
Remove borrower	→ Remove Borrower	→ removeBorrower()
Update borrower	→ Update Borrower	→ updateBorrower()
Validate system access	→ Log On	→ logOn()
Complete state of items checked out	→ Log Out	→ logOut()

Traceability Table

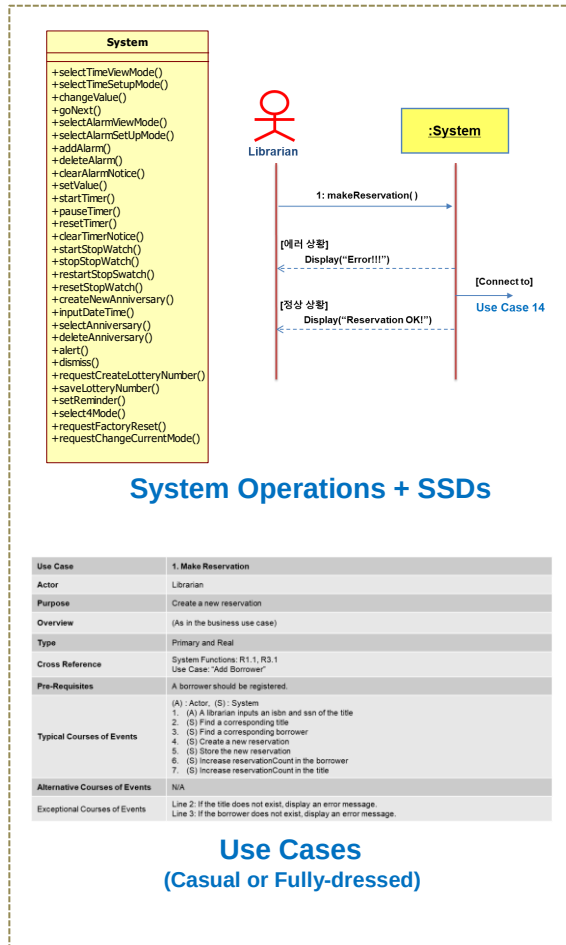


Statechart Diagram
(Optional)

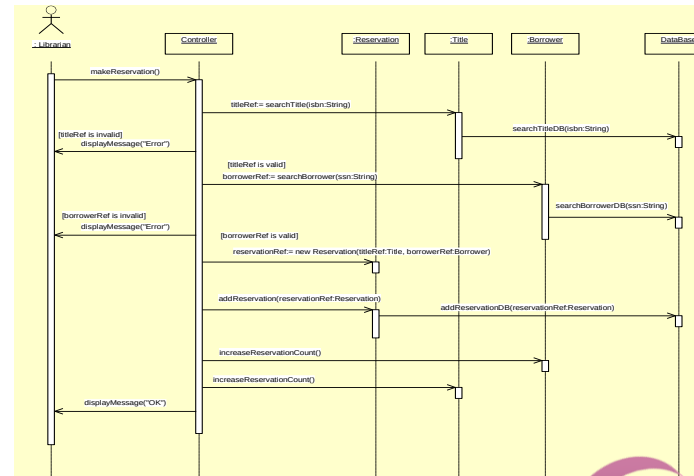


Domain Model

• OOD (Object-Oriented Design)



Sequence Diagrams



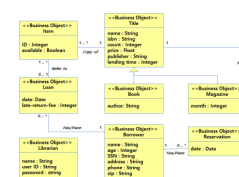
System Architecture

Component Diagram + Package Diagram

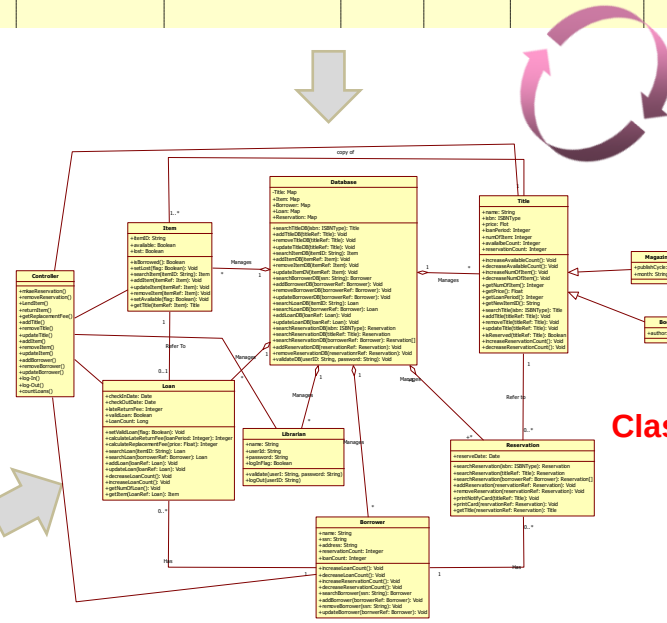
for each use case
(+SSD)



Domain Model



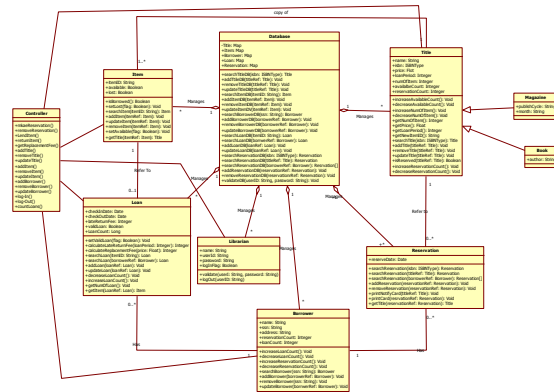
Class Diagram



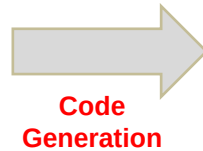
Class	Database	Title	Borrower	Reservation
Database	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer
Title	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer
Borrower	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer
Reservation	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer	isbn: String title: String borrower: String reservationCount: Integer

Traceability Table

- OOI (Object-Oriented Implementation)



Class Diagram
(Static Model)



Skeleton Code

```
class A
{
  variables:

  aaa();
  bbb();
  ...
}
```

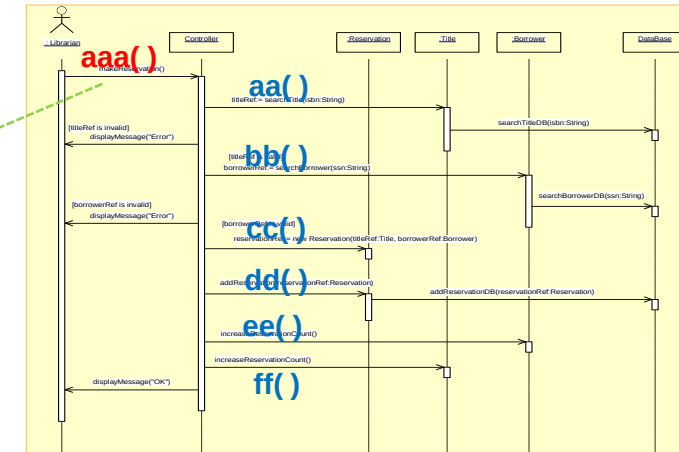
Realization

Concrete Code

```
class A
{
  variables:

  aaa()
  {
    aa();
    bb();
    cc();
    dd();
    ee();
    ff();
  }
  bbb();
  ...
}
```

System Operation



Sequence Diagrams
(Dynamic Model)

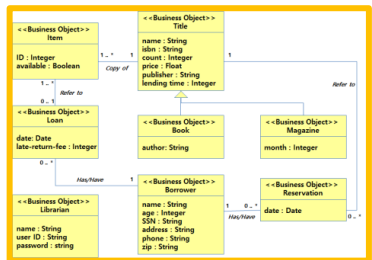
with a hint

+ 변수 선언
+ 조건문
+ 초기화
+ Visibility 확보

OOPT as OOAD



- : Requiring Object-Oriented Thinking***
- : Mechanical Transformation
- : Mapping with a hint



Domain Model

User Case	1. Make Reservation
Actor	Librarian
Purpose	(As in the business use case)
Overview	(As in the business use case)
Type	Primary and Essential
Cross Reference	System Functions R1, R2, 1 Use Case: "Add Borrower"
Pre-Conditions	Borrower should have an ID card
Typical Courses of Events	(A) Actor: ID: System 1. (A) Librarian requests the reservation of title 2. (S) Check if a corresponding title exists 3. (S) Check if a corresponding borrower exists 4. (S) If the borrower does not exist, create "Add Borrower" 5. (S) Create reservation information
Alternative Courses of Events	N/A
Exceptional Courses of Events	Line 1: If invalid reservation information is entered, indicate an error.

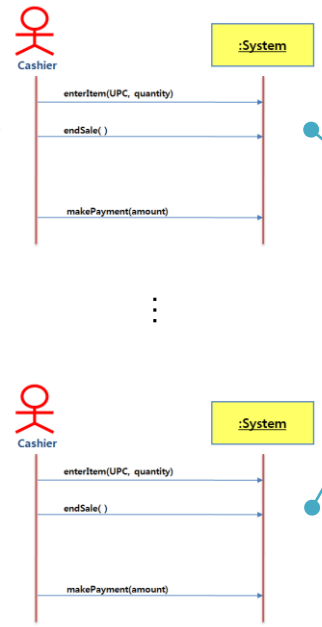
(Refined) Use Cases

Traceability Table

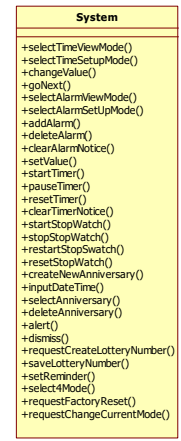
Functional Requirements	Use Cases	Category
R1.1 Make reservation	1. Make Reservation	Evident
R1.2 Remove reservation	2. Remove Reservation	Evident
R1.3 Lend Item	3. Lend Item	Evident
R1.4.1 Return title	4. Return Title	Evident
R1.4.2 Calculate Late-Return-Fee	5. Calculate Late-Return-Fee	Hidden
R1.5 Calculate Replacement Fee	6. Get Replacement Fee	Evident
R1.6 Notify Availability	7. Notify Availability	Hidden
R2.1 Add title	8. Add Title	Evident
R2.2 Remove title	9. Remove Title	Evident
R2.3 Update title	10. Update Title	Evident
R2.4 Add item	11. Add Item	Evident
R2.5 Remove item	12. Remove Item	Evident
R2.6 Update item	13. Update Item	Evident
R3.1 Add borrower	14. Add Borrower	Evident
R3.2 Remove borrower	15. Remove Borrower	Evident
R3.3 Update borrower	16. Update Borrower	Evident
R4.1 Validates system access	17. Log-In	Evident
R4.2 Validates system access	18. Log-Out	Evident
R5.1 Compute total # of items checked out	19. Count Loans	Evident

User (Functional) Requirements

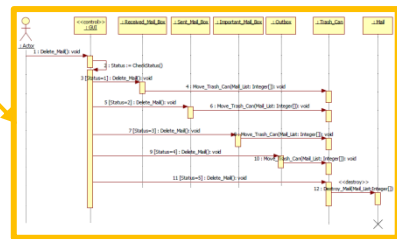
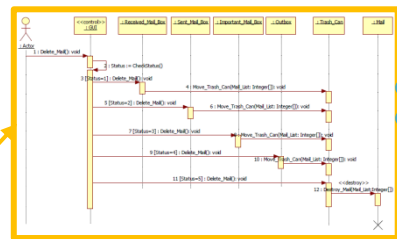
Use Cases



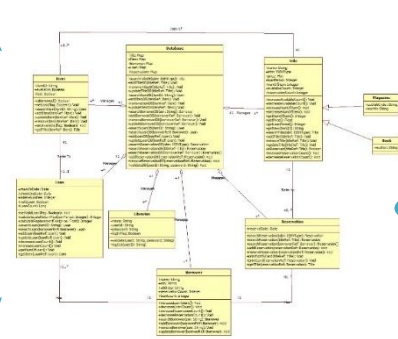
System Sequence Diagrams



System Operations



Sequence Diagrams



Class Diagram

```
class A
{
    variables:
    aaa()
    {
        aa();
        bb();
        cc();
        dd();
        ee();
        ff();
    }
    bbb();
    ...
}
```

Requirements Engineering

Stage 1000. Plan (Inception)

Stage 2030. Analyze (OOA)

Stage 2040. Design (OOD)

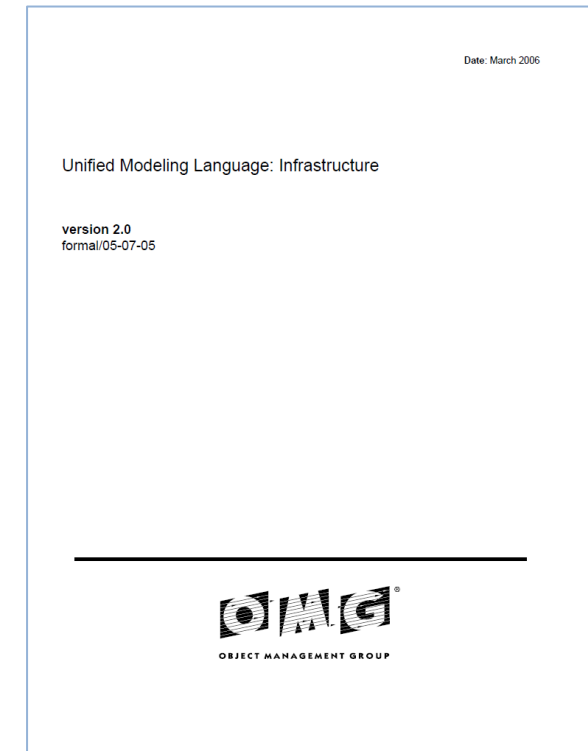
Stage 2050. Construct (OOI)

A decorative blue sidebar on the left side of the slide. It features several geometric elements: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a white circle with a blue outline further up, a dark blue circle with a white outline above that, and a white circle with a blue outline at the top. There are also some diagonal lines and a small grid of white dots near the bottom right of the sidebar.

An Introduction to UML

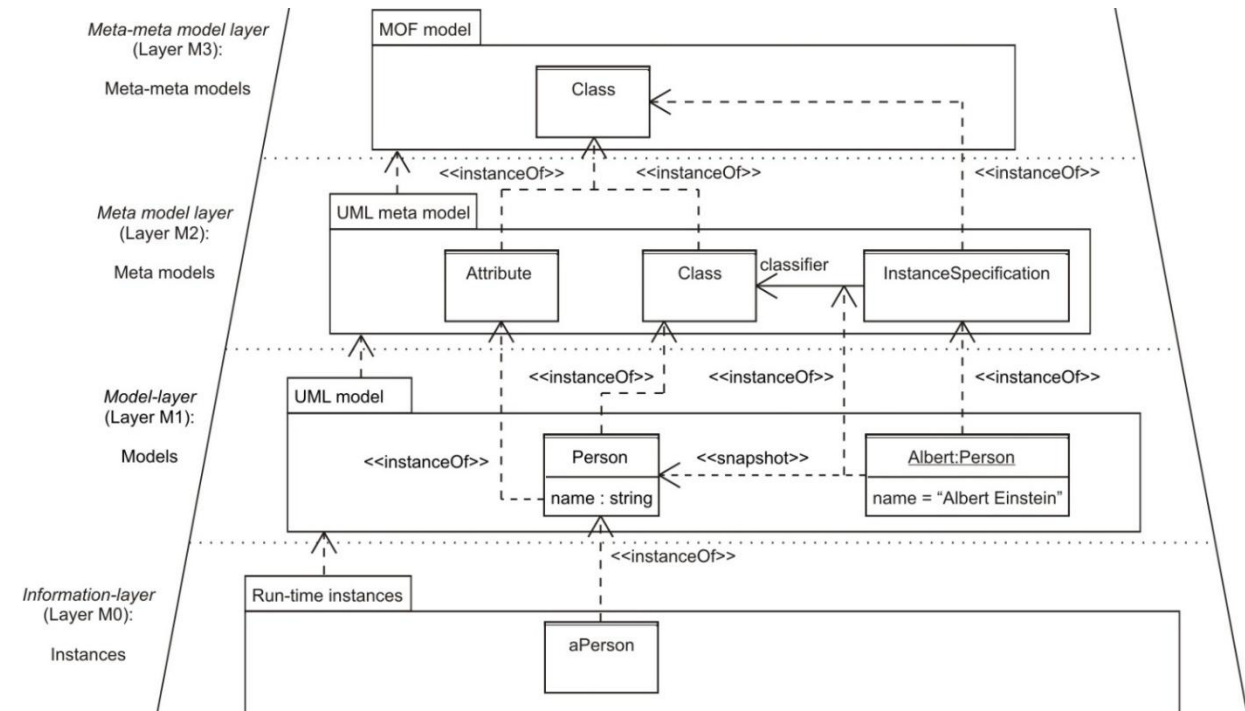
- UML

- **Unified Modeling Language** for
 - Visualizing, Specifying, Constructing, and
 - Documenting the artifacts of software-intensive systems.
- Offer vocabulary and rules for communication
 - <http://www.uml.org/>
- Combine the best of the best from
 - Data Modeling (Entity Relationship Diagrams)
 - Business Modeling (Workflow)
 - Object Modeling
 - Component Modeling (Middleware, COTS, etc.)

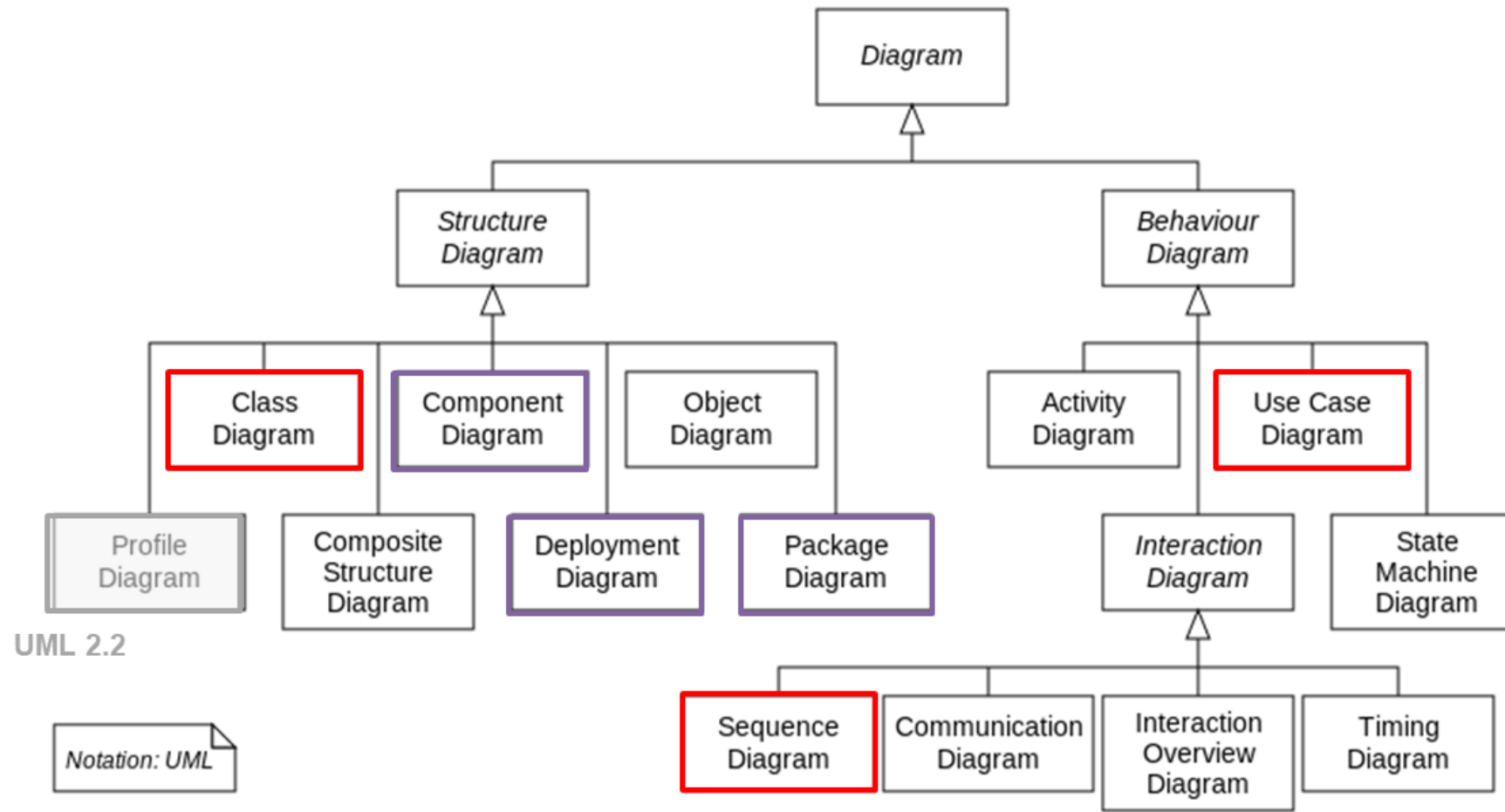


- **3 ways to apply (use) UML**
 - **Sketch**
 - Conceptual perspective
 - Informal and incomplete diagrams are created to discuss with each other.
 - Used at the Inception and OOA phases
 - **Blueprint**
 - Specification perspective
 - Relatively detailed design diagrams can be used for code generation.
 - Used at the OOA and OOD phases
 - **Programming language**
 - Implementation perspective
 - Complete executable code will be automatically generated.
 - Still under development in terms of theory, tool robustness and usability.

- **4-layer metamodel architecture**
 - instance → model → meta model → meta-meta model
- **MOF (Meta Object Facility)** defines a four-layer meta model hierarchy.
 - Layer M3: Meta-meta model layer (**The MOF model**)
 - Layer M2: Meta model layer (**The UML meta model**)
 - Layer M1: Model layer (**The UML model**)
 - Layer M0: Information layer (**the Application**)
- MOF and UML are aligned.
 - The UML infrastructure contains all the concepts needed for the specification of UML and MOF.

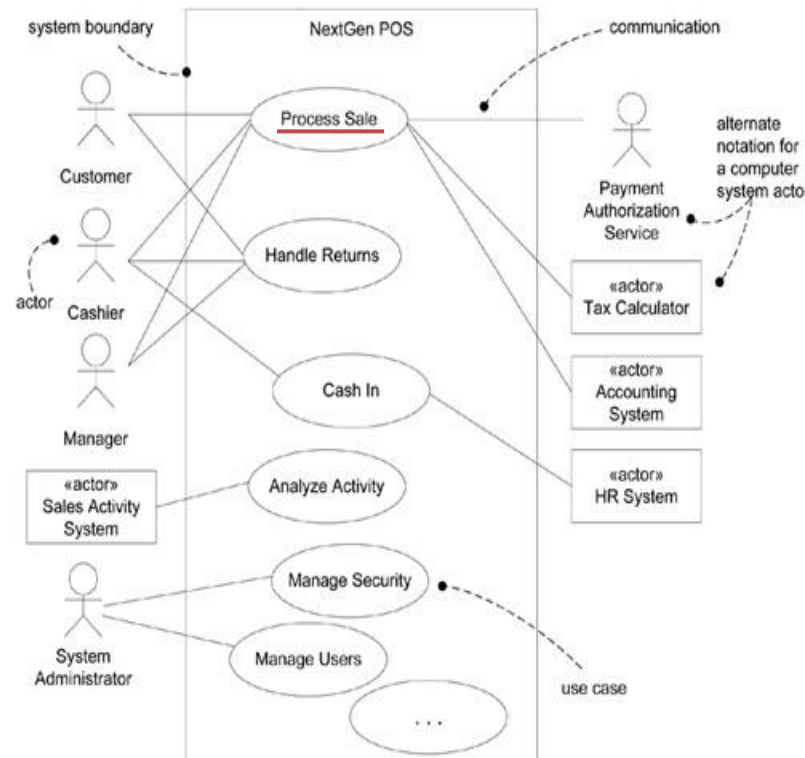


- 13 UML diagrams



1. Use Case Diagram

- **Use case diagrams** illustrate the name of use cases and actors, and the relationships between them.
 - **Use case**
 - A collection of related success and failure [scenarios](#)
 - Describing how an actor uses the system to achieve a goal (= the name of the use case)
 - **Actor** : something with behavior, such as a person, computer, or organization

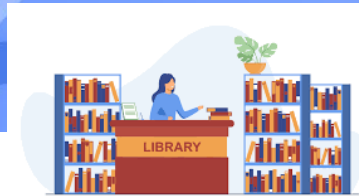


Main Success Scenario (or Basic Flow):

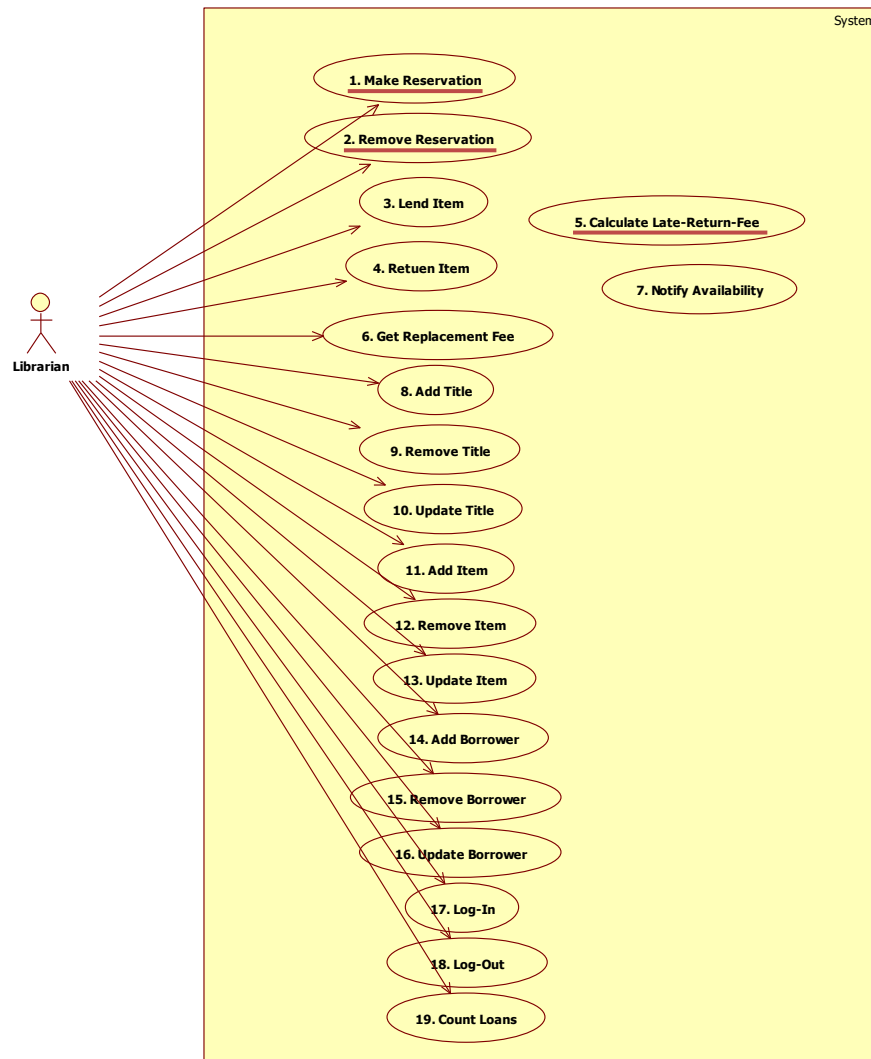
1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total.
Price calculated from a set of price rules.
Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting system (for accounting and commissions) and Inventory system (to update inventory).
9. System presents receipt.
10. Customer leaves with receipt and goods (if any).

Use Case : Process Sale

Example: Uses Cases



Library Management System



Use Case Diagram

Use Case	1. Make Reservation
Actors	Librarian
Description	- This use case begins when a borrower arrives at the counter and then requests reservation. - For a registered borrower, it makes a reservation slip (software-wise). - For an unregistered borrower, the librarian registers the person and makes a reservation for the person.
Use Case	2. Remove Reservation
Actors	Librarian
Description	- A borrower who made a reservation can cancel his/her reservation. • Explicitly cancels the reservation (Evident) - When a borrower checks out an item which he/she previously reserved, this use case is invoked automatically. (Hidden)
Use Case	5. Calculate Late-Return-Fee
Actors	None (Hidden)
Description	- This use case can begin when the late items are returned or every midnight. - This use case computes the penalty amount for items returned late. - It first computes the number of extra days held by the borrower, then multiplies it by a pre-determined daily rate for late returns.

Use Cases (Planning / OOA)

Use Case	1. Make Reservation
Actors	Librarian
Description	- This use case begins when a borrower arrives at the counter and then requests reservation. - For a registered borrower, it makes a reservation slip (software-wise). - For an unregistered borrower, the librarian registers the person and makes a reservation for the person.



Use Case	1. Make Reservation
Actor	Librarian (Evident)
Purpose	(As in the Inception)
Overview	(As in the Inception)
Type	Primary and Casual
Cross Reference	System Functions: R1.1, R3.1 Use Case: "Add Borrower"
Pre-Requisites	Borrower should have an id_card.
Typical Courses of Events	(A) : Actor, (S) : System 1. (A) A librarian requests the reservation of a title 2. (S) Check if a corresponding title exists 3. (S) Check if a corresponding borrower exists 4. (S) Create a reservation information
Alternative Courses of Events	Line 3: (S) If the borrower's information is out of date, request for the update. (A) A librarian updates up-to-date information of the borrower.
Exceptional Courses of Events	Line 1~3: If invalid reservation information is entered, indicate an error.

Use Cases (Planning / OOA)

Use Case	5. Calculate Late-Return-Fee
Actors	None (Hidden)
Description	- This use case can begin when the late items are returned or every midnight. - This use case computes the penalty amount for items returned late. - It first computes the number of extra days held by the borrower, then multiplies it by a pre-determined daily rate for late returns.



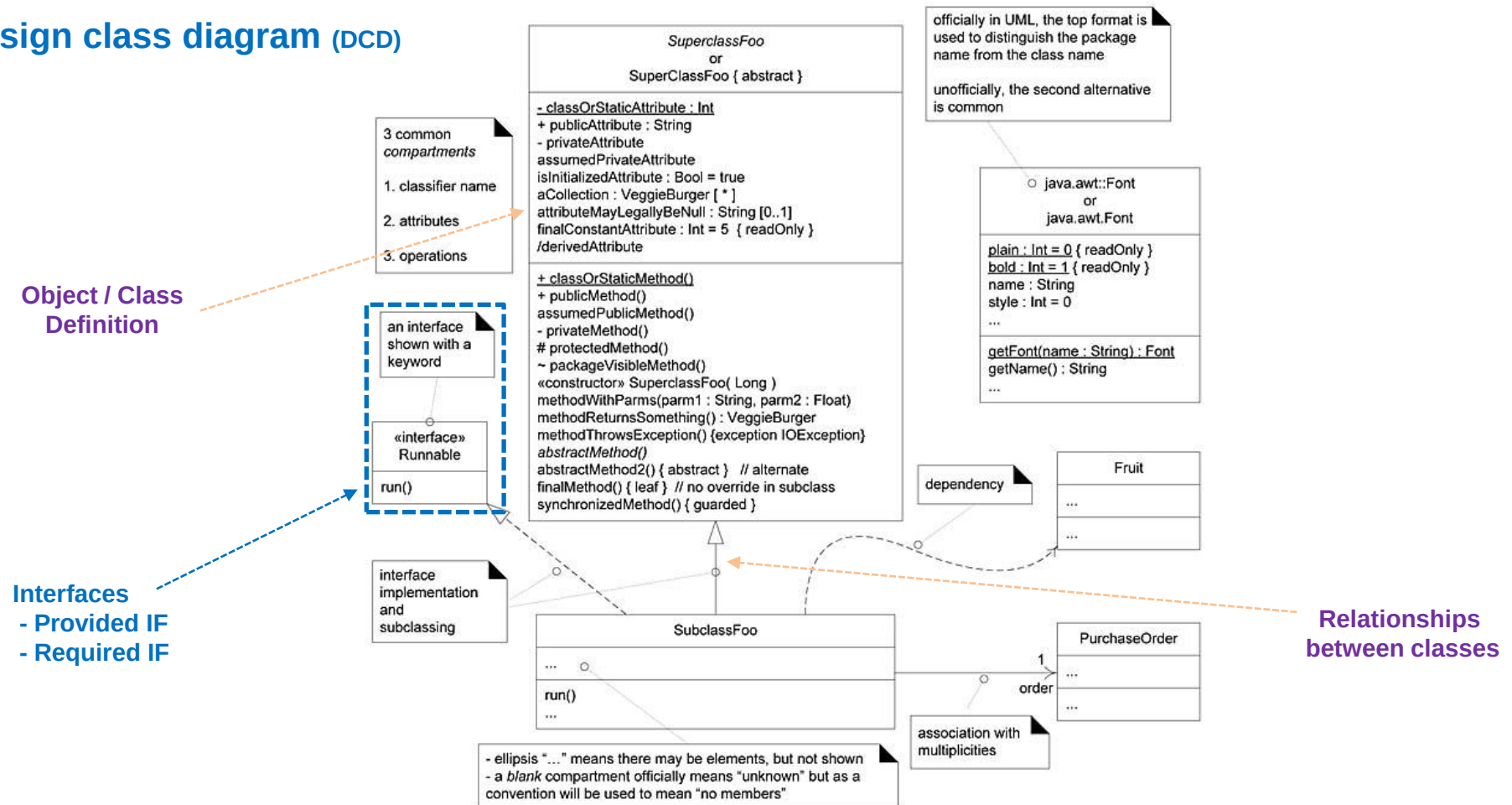
Use Case	5. Calculate Late-Return-Fee
Actor	None (Hidden)
Purpose	(As in the Inception)
Overview	(As in the inception)
Type	Primary and Casual
Cross Reference	System Functions: R1.4.1, R1.4.2 Use Case: "Return Item"
Pre-Requisites	Lending due-date should be over.
Typical Courses of Events	(A) : Actor, (S) : System 1. (S) Compute late-return time 2. (S) Compute late-return fee 3. (S) Print the late-return fee
Alternative Courses of Events	N/A
Exceptional Courses of Events	N/A

Use Cases (OOA / OOD)

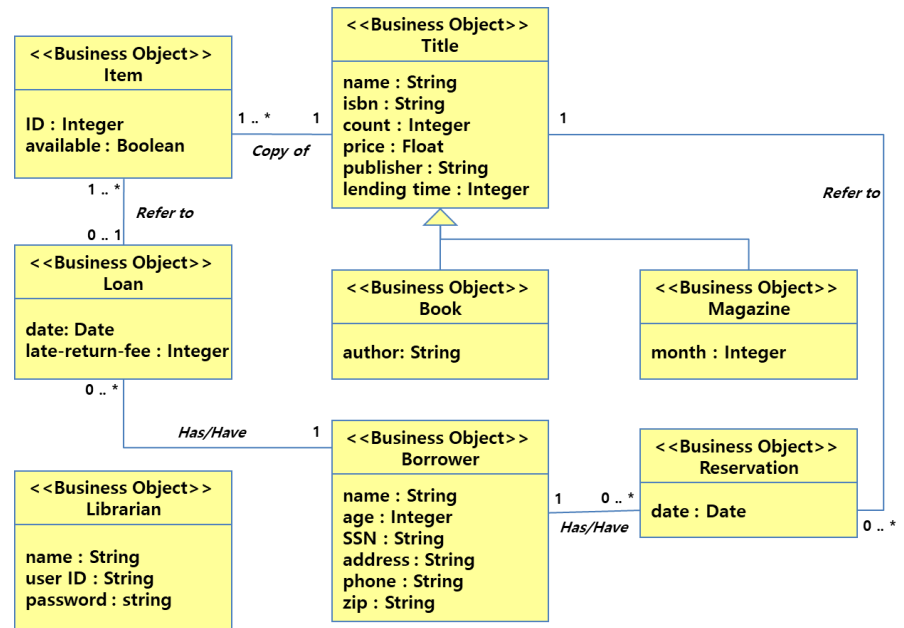
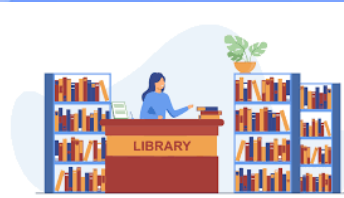
2. Class Diagram



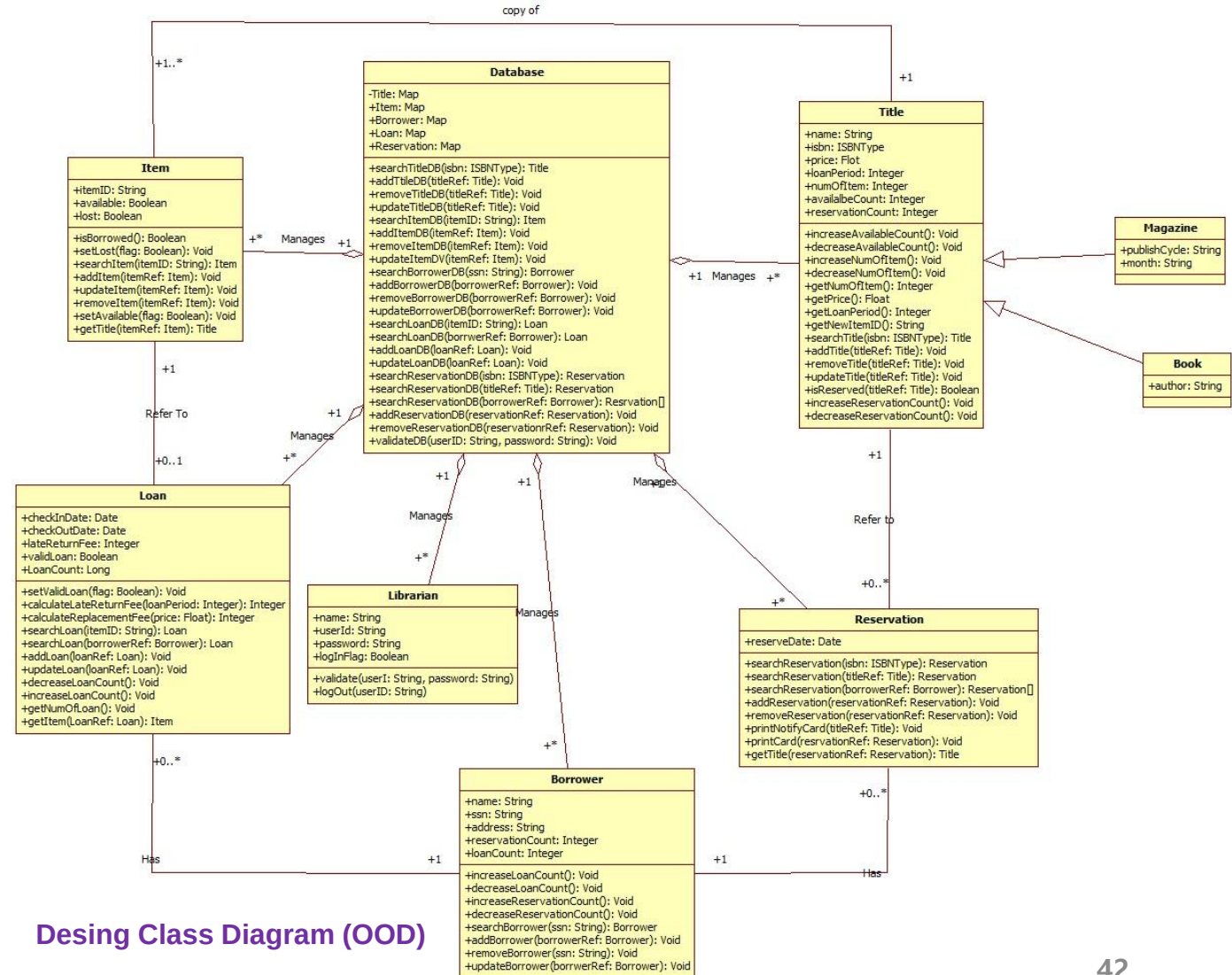
- **Class diagrams** show the **classes** of the system, their inter-relationships, and the operations and attributes of the classes.
 - at OOA : Domain model
 - at OOD : Design class diagram (DCD)



Domain Model → Design Class Diagram

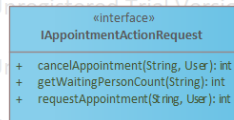


Domain Model (OOA)

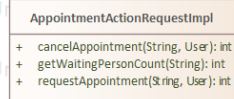


Desing Class Diagram (OOD)

Provided Interface



Provided Interface



* Observer Pattern *

- concrete observable에서 thread를 활용하여 [QA-03] 측면에서 용이

1. 정보 수신

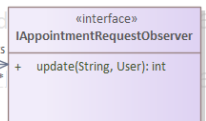


-observable

Observer Pattern

2. observer에게 notify

-observers



3. concrete implementor 설정



-observable

* Concrete Observer에서 notify 수신 예시 *

```

int update(String hospitalId, User user) {
    if (user == null) {
        abstraction = new RefinedAppointmentAbstraction(new AppointmentCheck());
    } else {
        if (user.checkIn == true)
            abstraction = new RefinedAppointmentAbstraction(new AppointmentIn());
        else
            abstraction = new RefinedAppointmentAbstraction(new AppointmentOut());
    }
    return abstraction.handleRequest(hospitalId, user);
}
  
```

* Adapter Pattern *

- 개인정보 DB 또는 병원정보 DB의 Interface 변경 시, concrete adapter에만 영향을 주므로 [QA-04] 측면에서 용이

* Design Pattern별 객체 정리 *

Observer Pattern

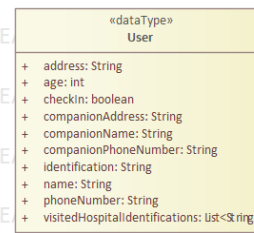
- observable : ISendAppointmentRequestObservable
- concrete observable : SendAppointmentRequestObservableImpl
- observer : IAppointmentRequestObserver
- concrete observer : AppointmentRequestObserverImpl

Bridge Pattern

- abstraction : AppointmentAbstraction
- refined abstraction : RefinedAppointmentAbstraction
- implementor : IAppointmentImplementor
- concrete implementor : AppointmentIn, AppointmentOut, AppointmentCheck

Adapter Pattern

- adapter(target) : IPrivacyDBAdapter, IHospitalDBAdapter
- concrete adapter : PrivacyDBAdapterImpl, HospitalDBAdapterImpl
- adaptee : IPrivacyDBRequest, IHospitalDBRequest

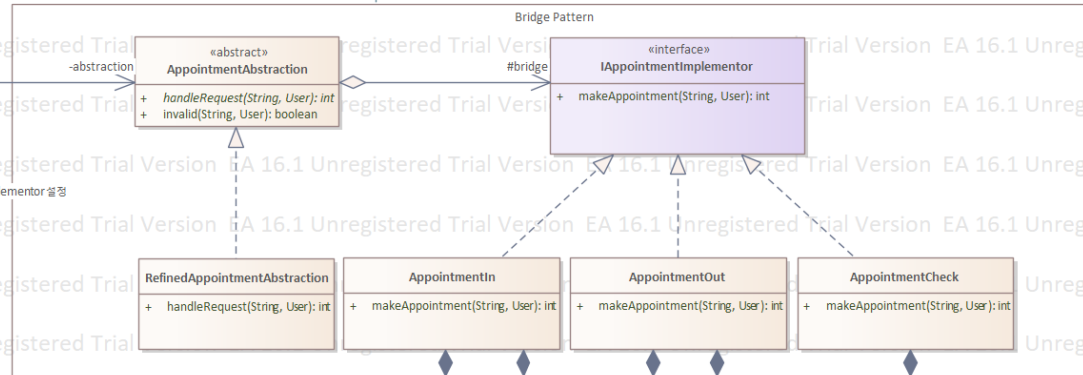


(from Data Type)

User 정보
* 추가식별 필요

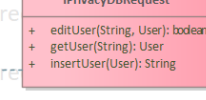
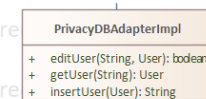
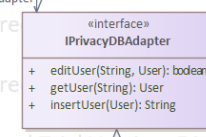
* Bridge Pattern *

- 진료 접수, 진료 취소, 진료 대기자 현황 확인의 처리 행위 변경 시, 해당 concrete implementor만 영향을 받으므로 [QA-04] 측면에서 용이

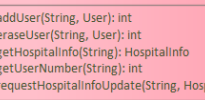
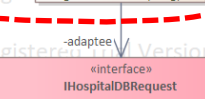
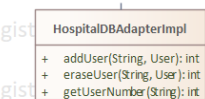
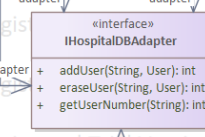


4. Required Interface로 정보 전송

Adapter Pattern



Adapter Pattern



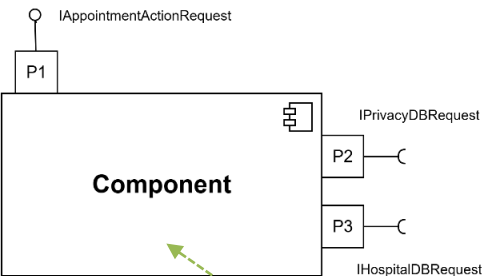
Required Interfaces

* Interface 규칙 *

- Interface의 명칭은 {대문자}로 시작

* Class 규칙 *

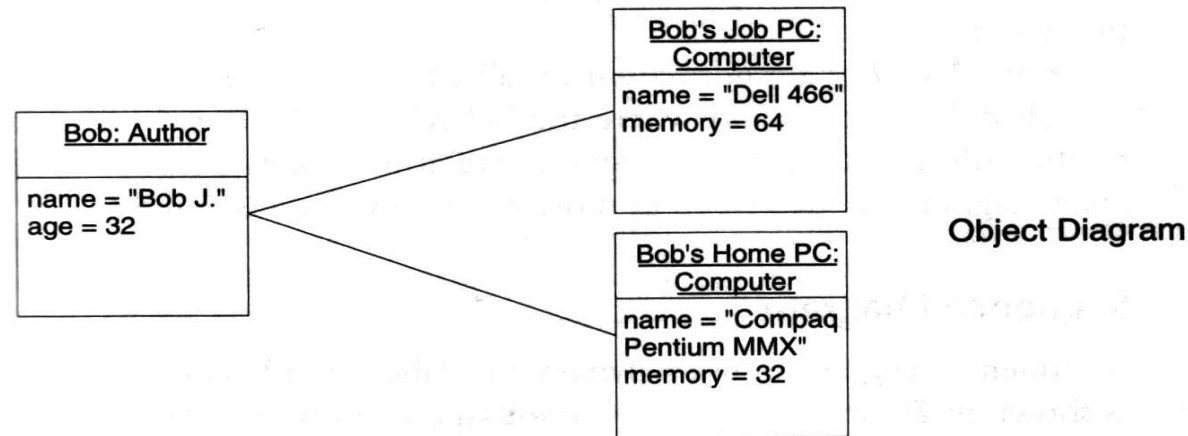
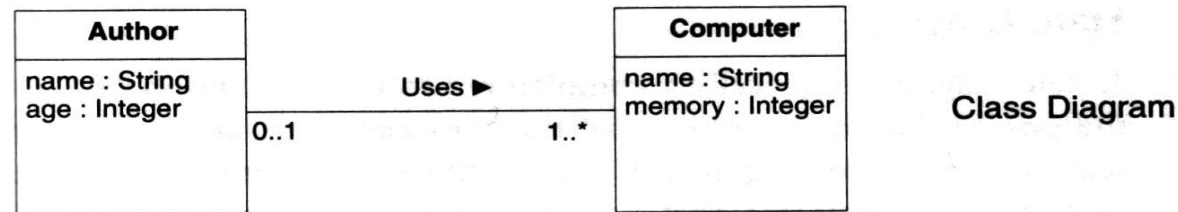
- abstract 또는 interface를 상속받는 구현부는 xxxImpl로 통일
- 어떠한 Pattern을 사용했는지 표현하기 위해 크게 이질감이 없다면 Pattern 용어 적용



3. Object Diagram



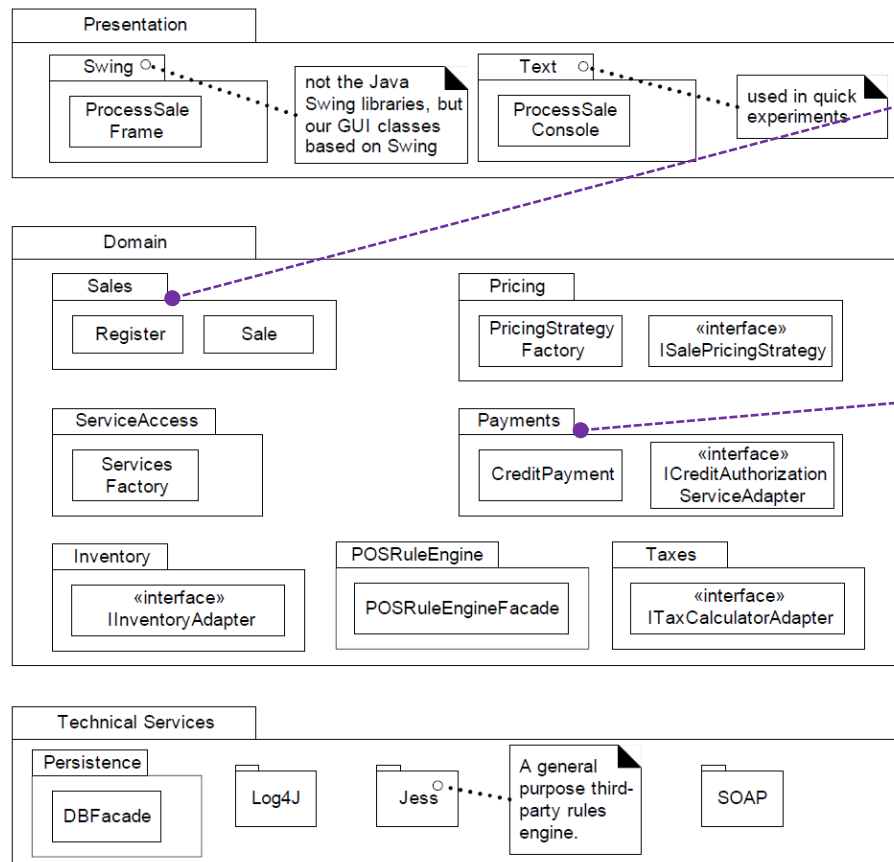
- **Object diagrams** are useful for exploring real world examples of objects and their relationships.
 - Shows instances of classes at a specific point of time. (i.e., snapshot)



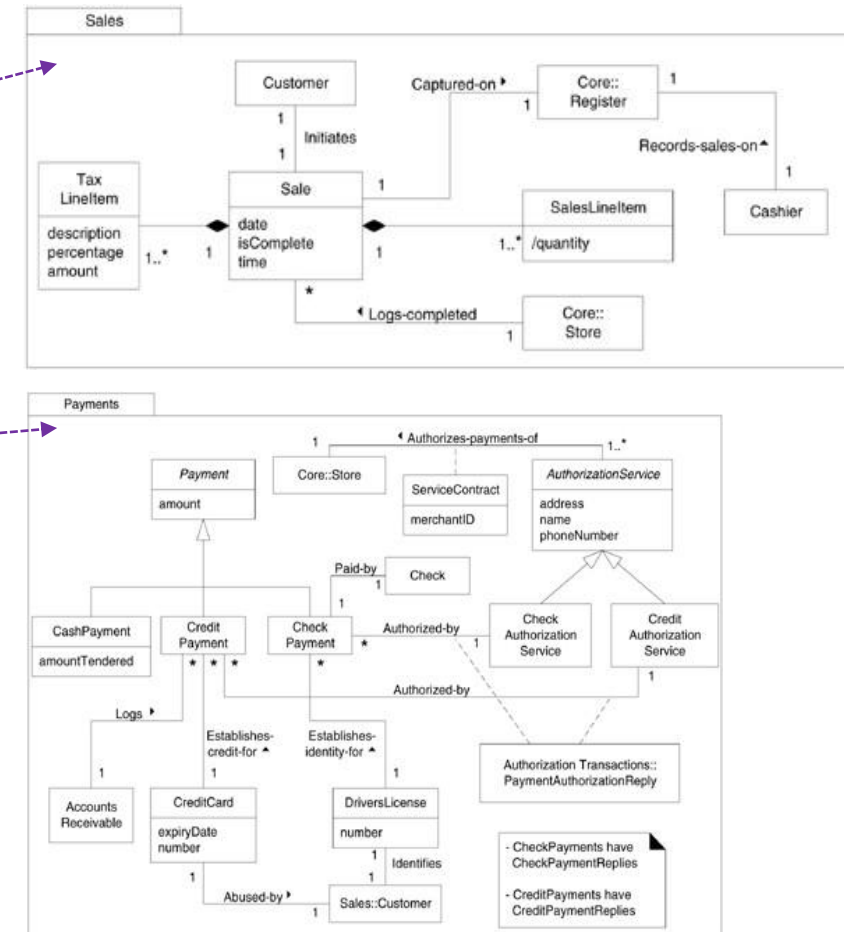
4. Package Diagram



- **Package diagrams** group classes into **packages** and simplify complex class diagrams.
 - A package is a collection of **logically** related UML elements.
 - Easy to describe the **layered architecture style** and their composition.



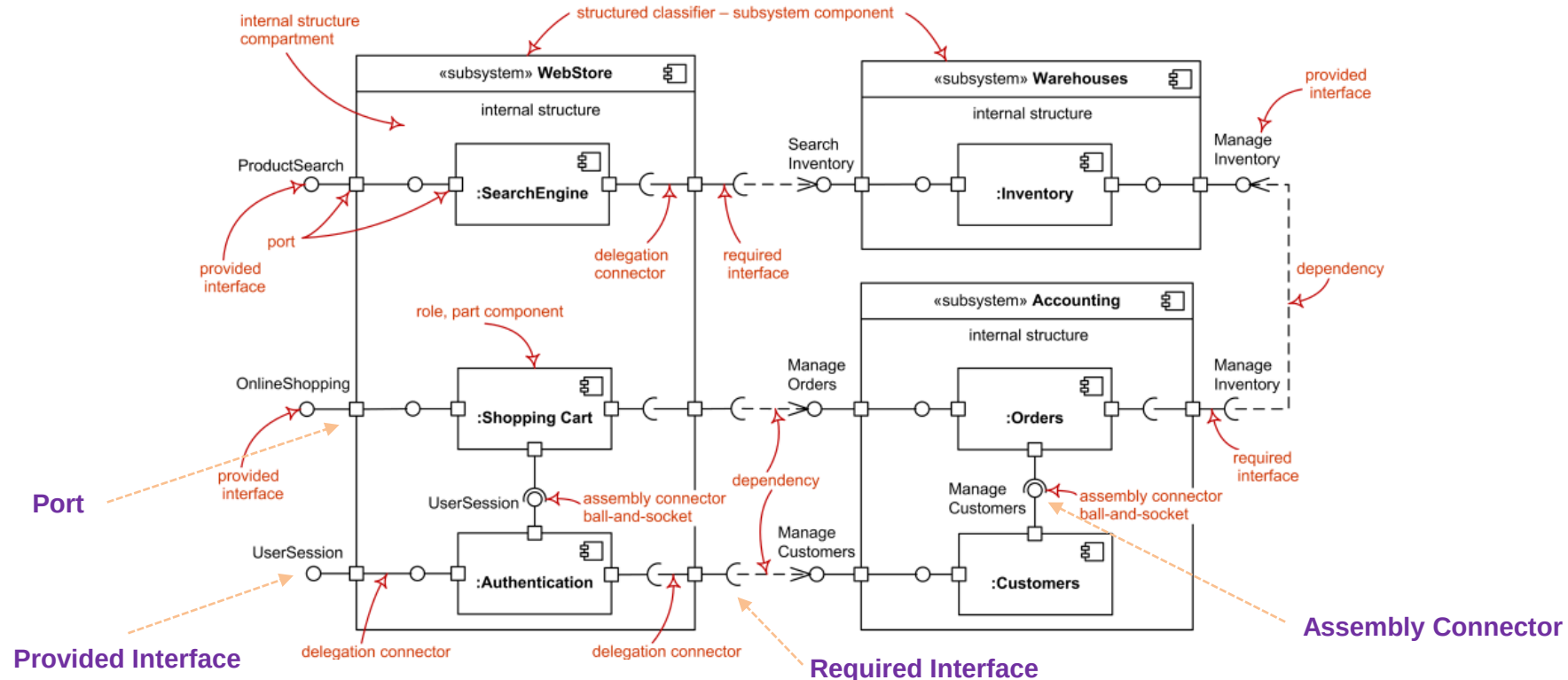
(at the early iterations)



(at the later iterations)

5. Component Diagram (+ 6. Composite Structure Diagram) SEU Samsung Electronics University

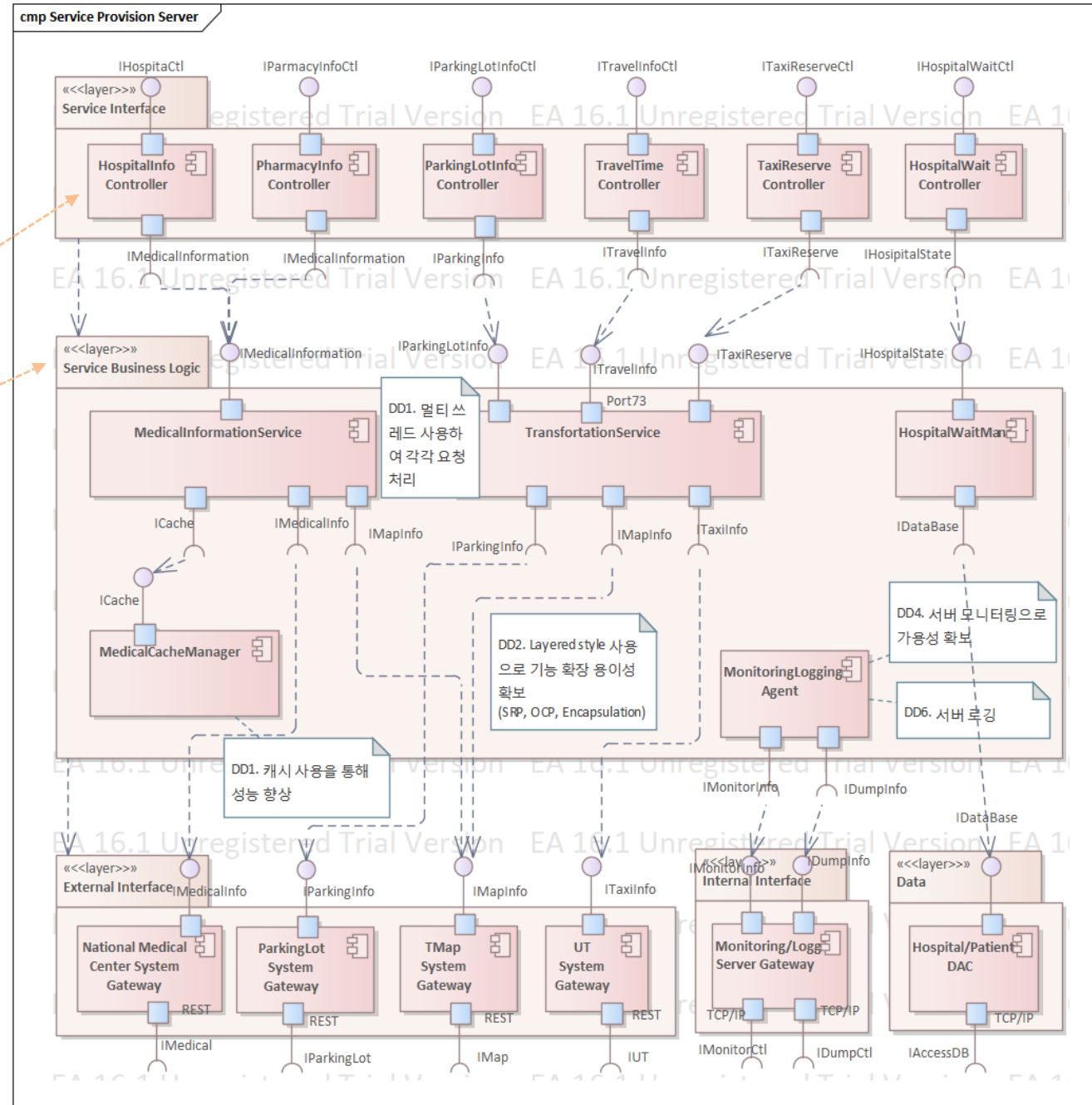
- **Component diagrams** depicts how components are wired together to form larger components or software systems.
 - Illustrate the logical structure and inter-dependency of arbitrarily complex systems
 - Show the internal structure (including parts and connectors) of components
 - Show run-time instances of interconnected instances collaborating over communications links

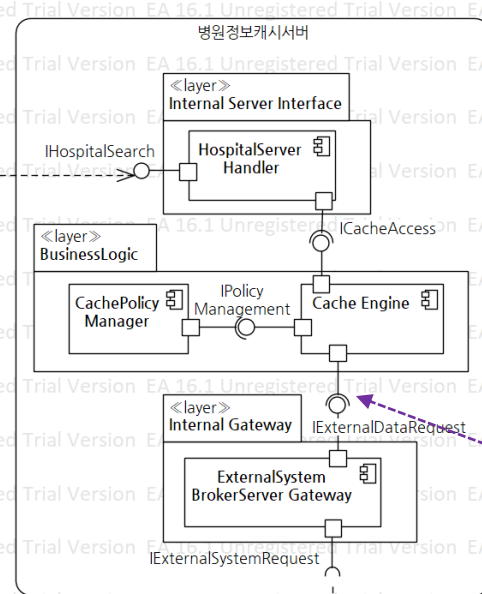
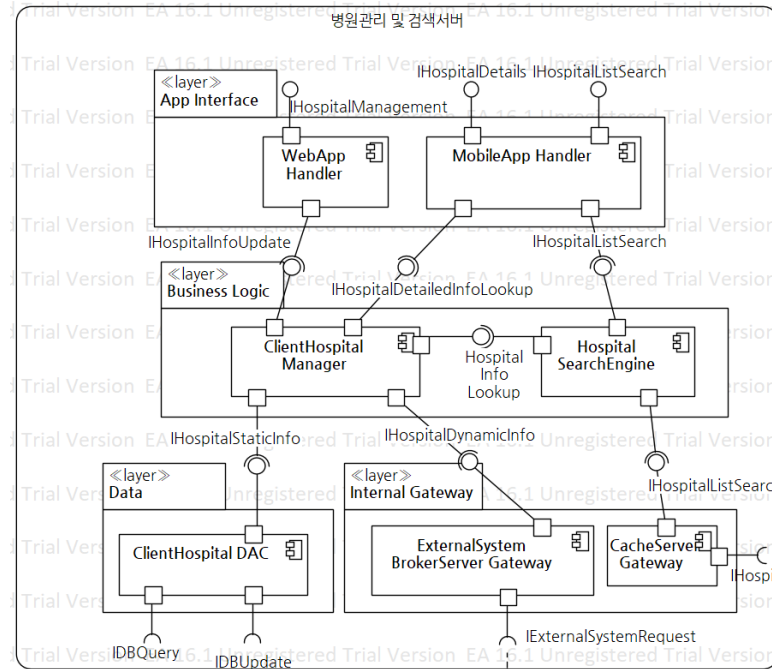


Component Diagram

+

Package Diagram

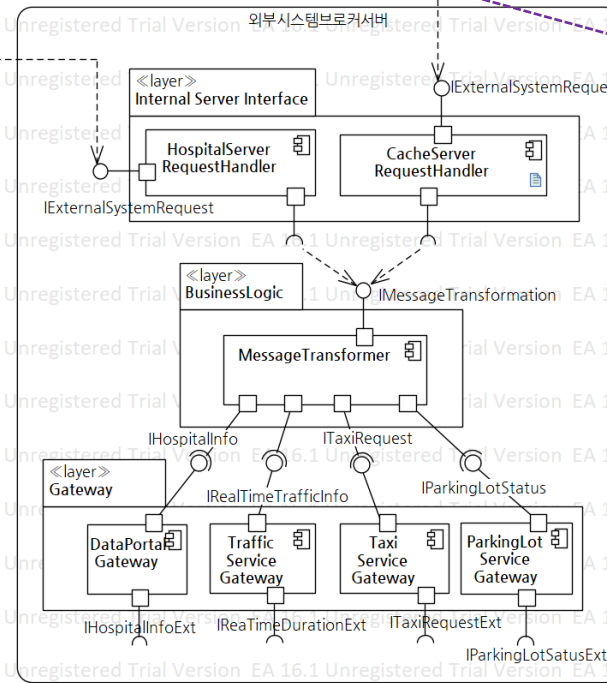
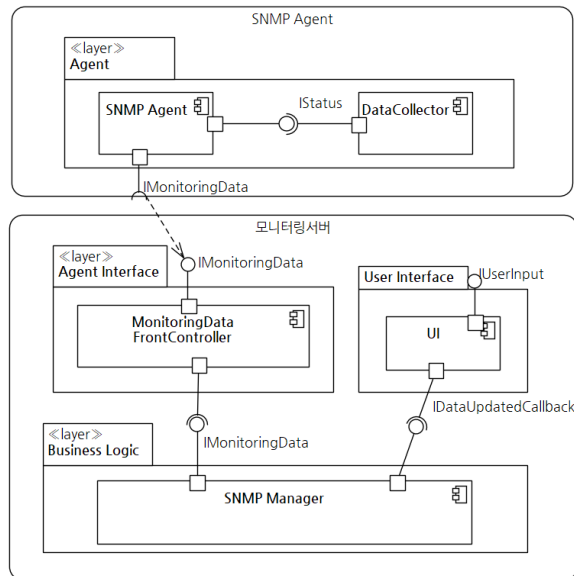




Component Diagram은
논리적 구조 (Logical Structure)를 표현하며,
물리적 배치 (Physical Deployment)는 고려하지 않음.

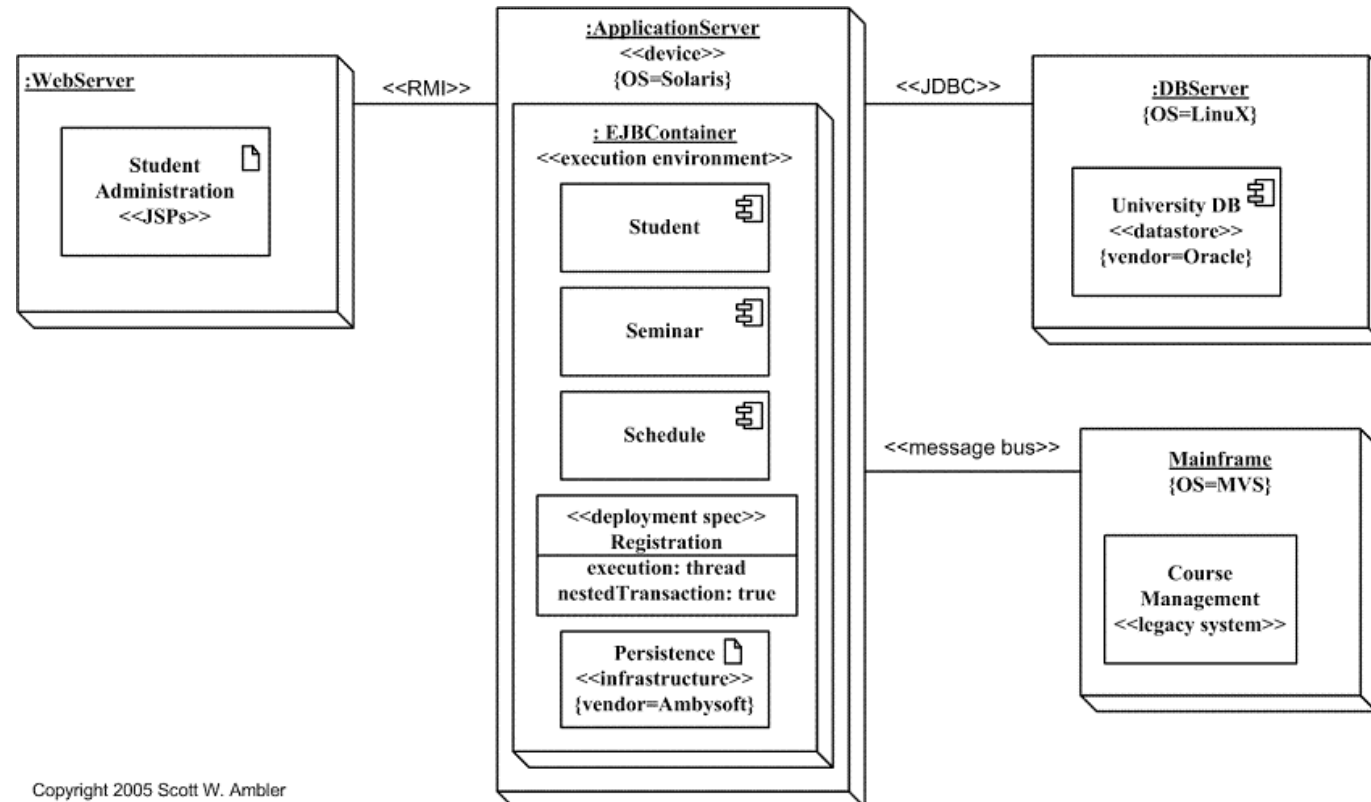
서버 內, layer 간 프로세스 호출

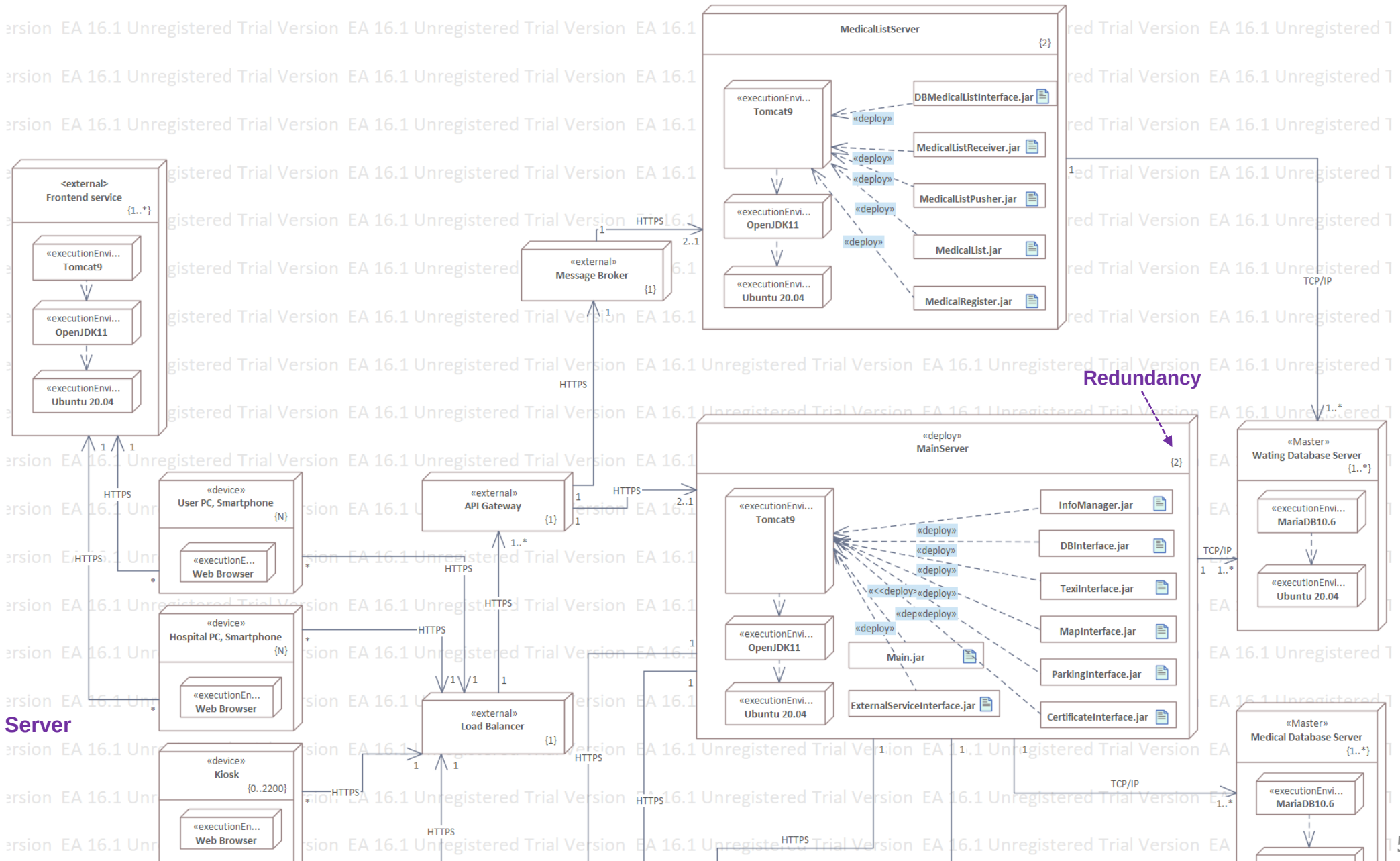
서버 간 네트워크 통신



7. Deployment Diagram

- **Deployment diagrams** depict a static view of the run-time configuration of hardware nodes and the software components running on those nodes.
 - Showing the **reflected QAs** like performance, availability, and scalability
 - Reflecting various deployment patterns and tactics

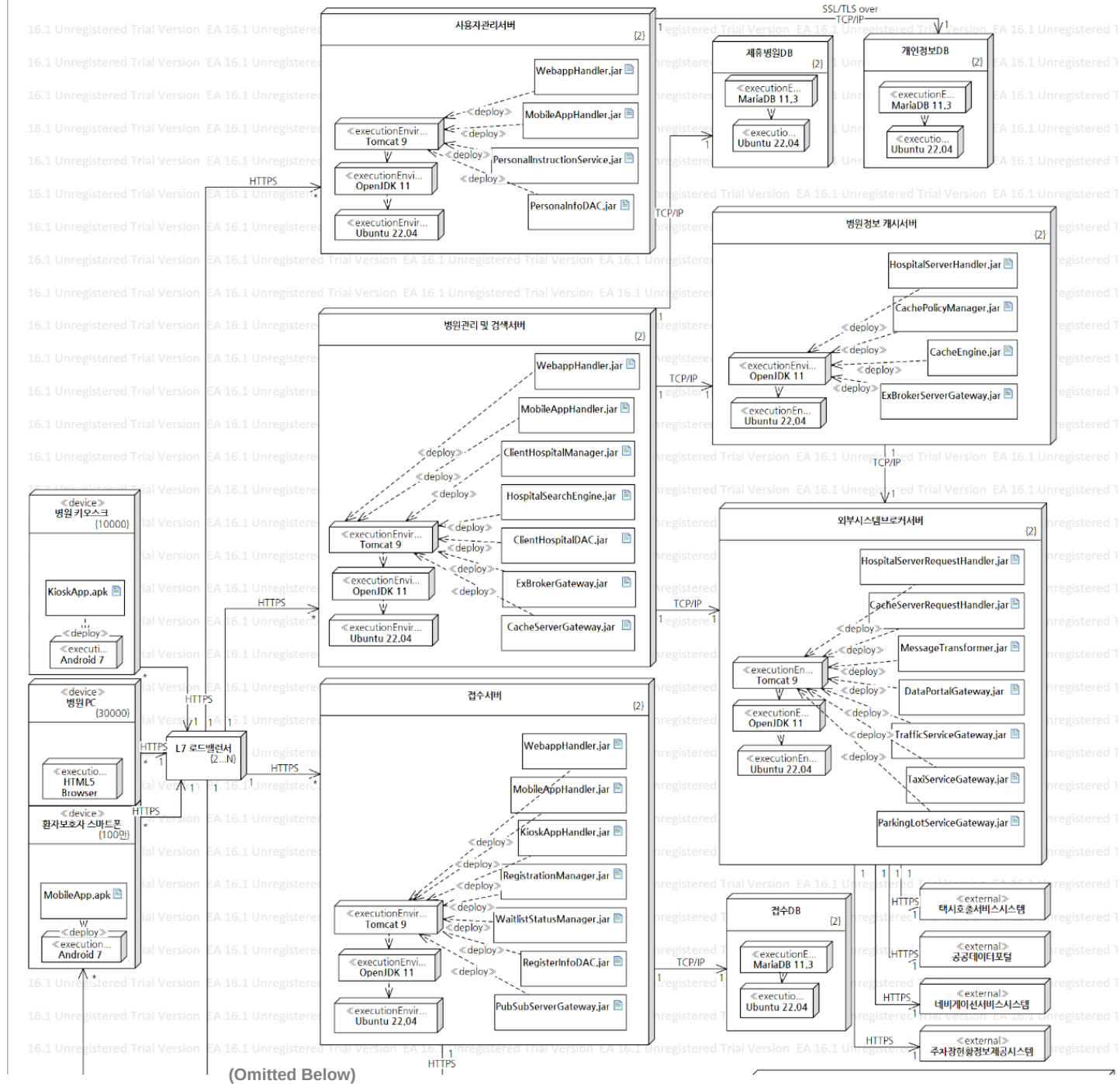


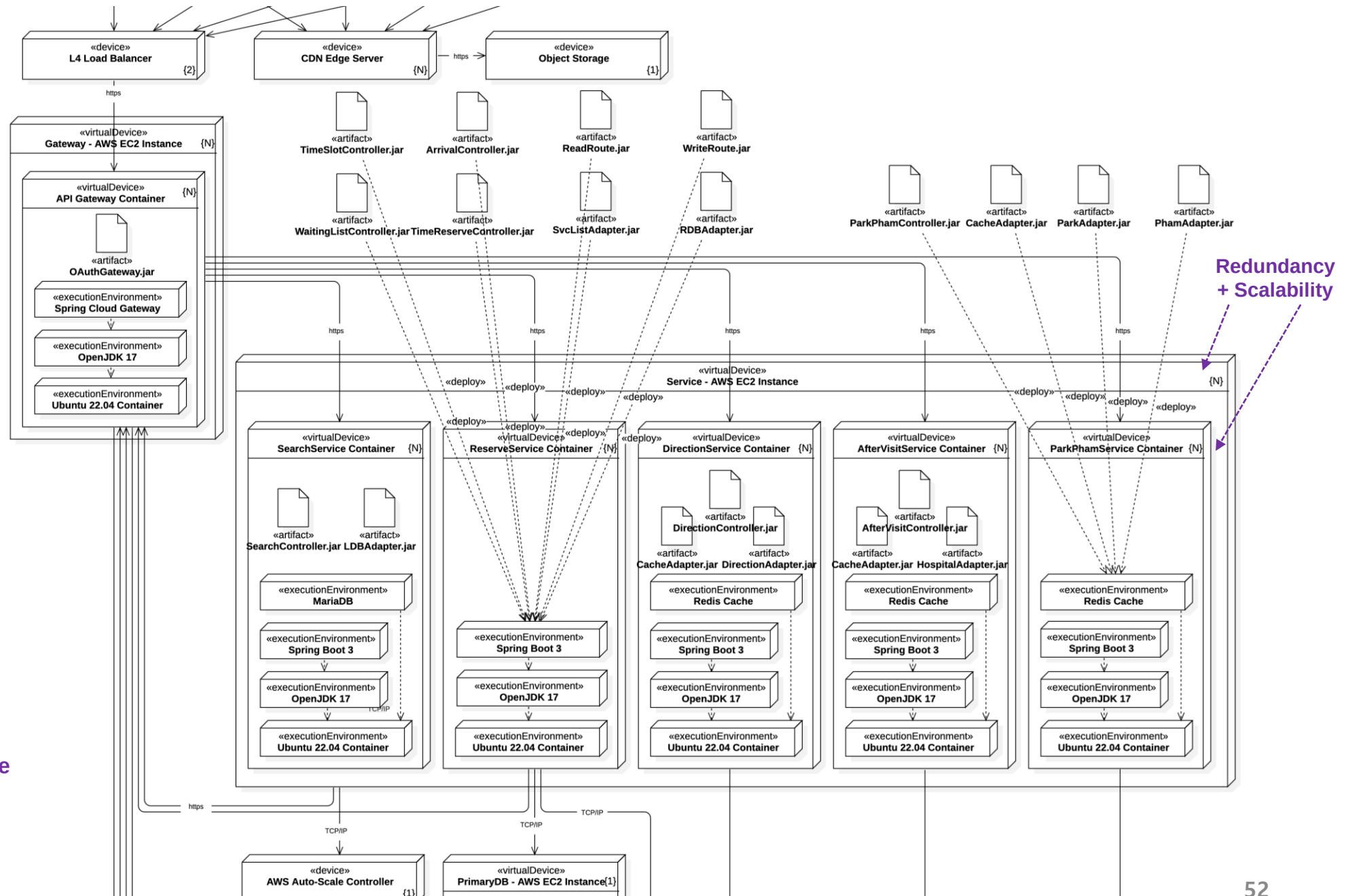


A Single Main Server

(Omitted Below)

Multiple Application Servers





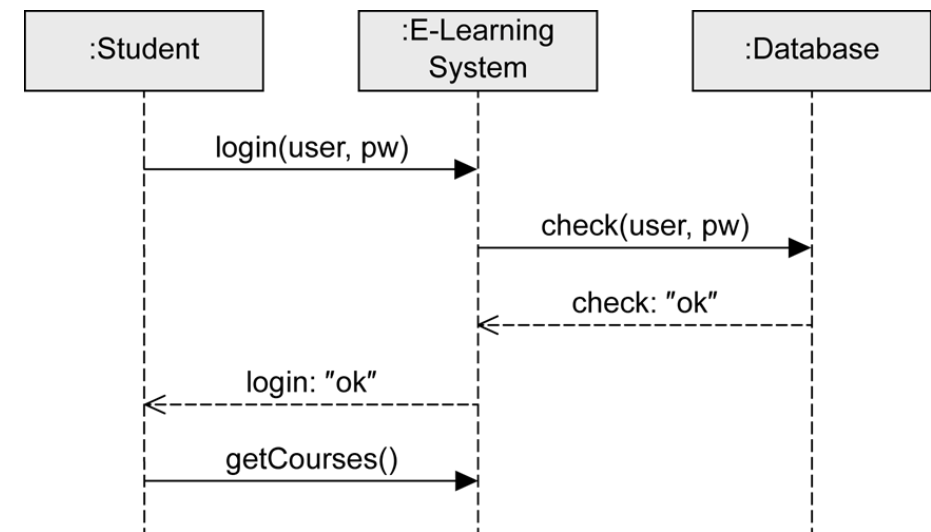
Microservice Architecture
(based on AWS)

(Omitted Below)

8. Sequence Diagram



- **Sequence diagrams** model the collaboration/cimunication of objects based on a time sequence.
 - Show how objects (or components) interact with others in a particular scenario of a use case.
 - **System Sequence Diagram (SSD)**
 - at system level : System is a black box.
 - **Sequence Diagram (SD)**
 - at **object** level : **Objects** communicate
 - at **system components** level : **Components** communicate



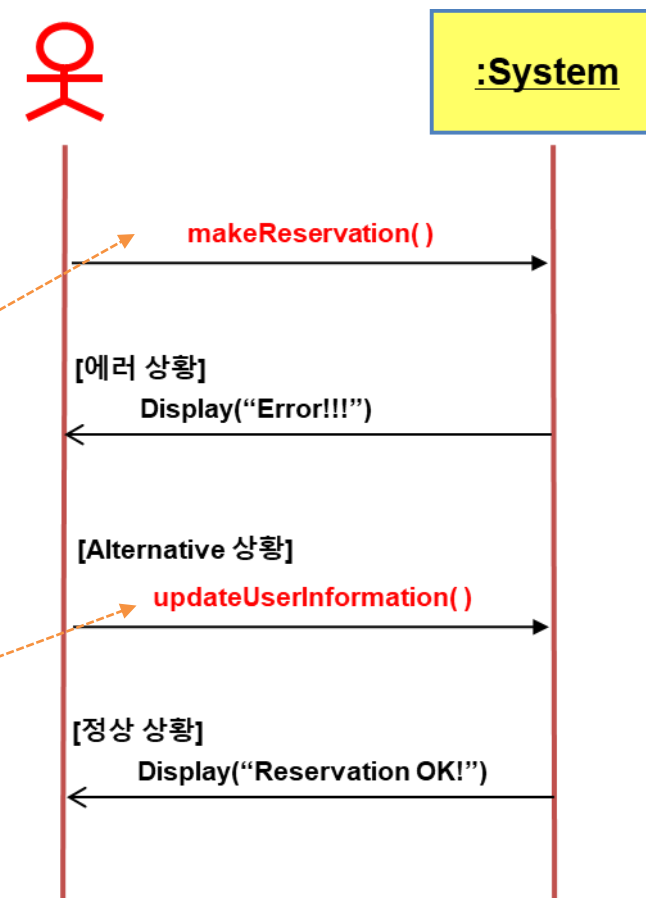
System Sequence Diagram
(at system level)

Use Case → System Sequence Diagram



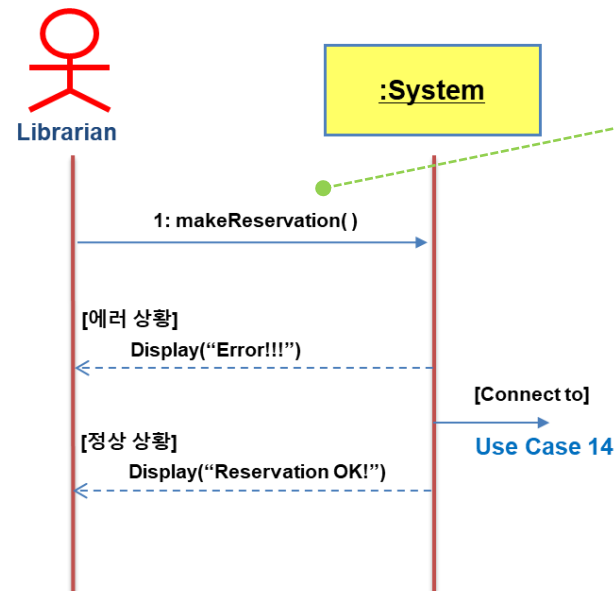
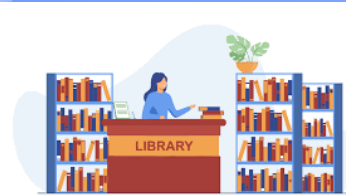
Use Case	1. Make Reservation
Actor	Librarian (Evident)
Purpose	(As in the Inception)
Overview	(As in the Inception)
Type	Primary and Essential
Cross Reference	System Functions: R1.1, R3.1 Use Case: "Add Borrower"
Pre-Requisites	Borrower should have an id_card.
Typical Courses of Events	(A) : Actor, (S) : System 1. (A) A librarian requests the reservation of a title 2. (S) Check if a corresponding title exists 3. (S) Check if a corresponding borrower exists 4. (S) Create a reservation information
Alternative Courses of Events	Line 3: (S) If the borrower's information is out of date, request for the update. (A) A librarian updates up-to-date information of the borrower.
Exceptional Courses of Events	Line 1~3: If invalid reservation information is entered, indicate an appropriate error.

Use Case

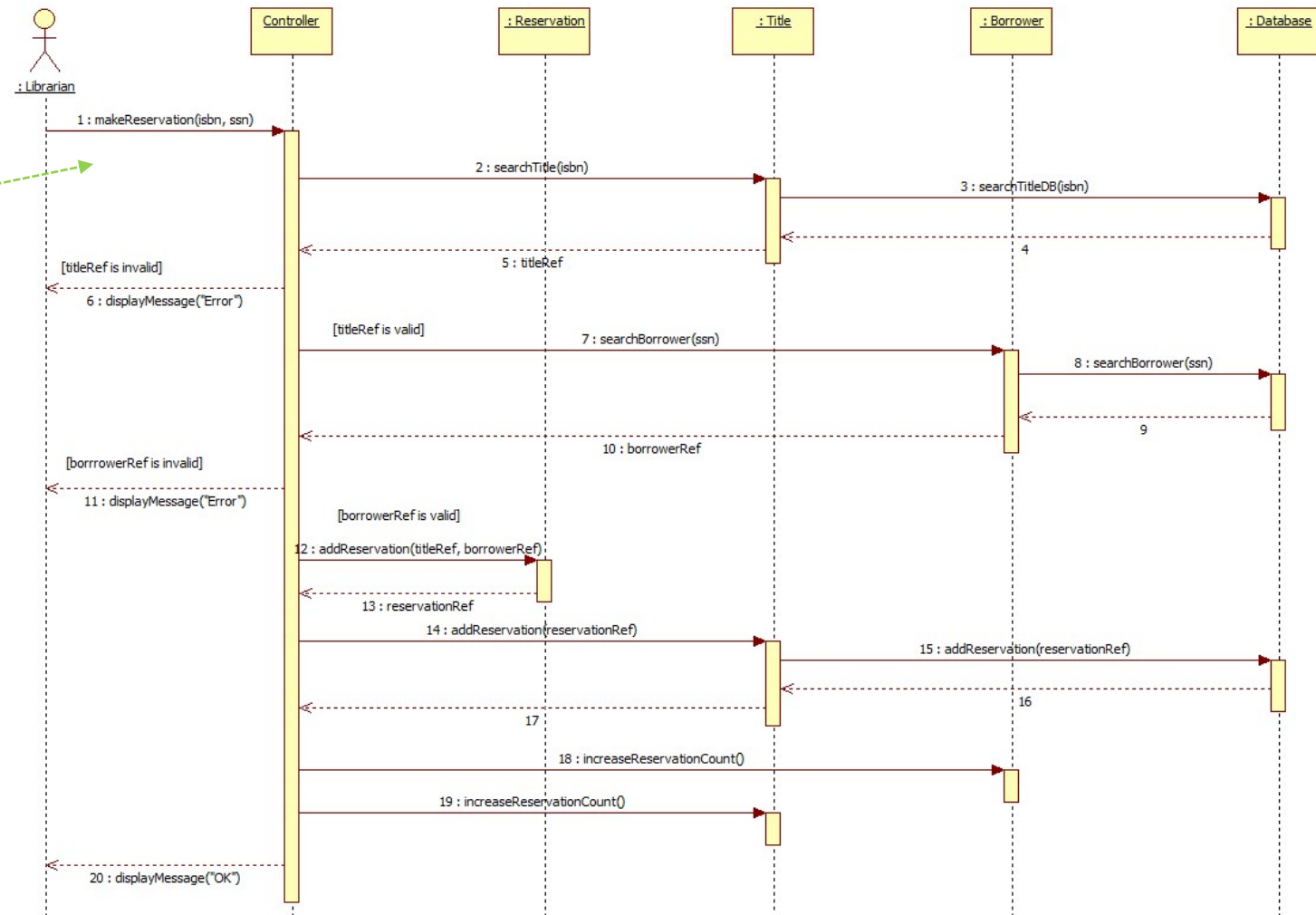


System Sequence Diagram
(at system level)

System Sequence Diagram → Sequence Diagram



System Sequence Diagram (OOA)



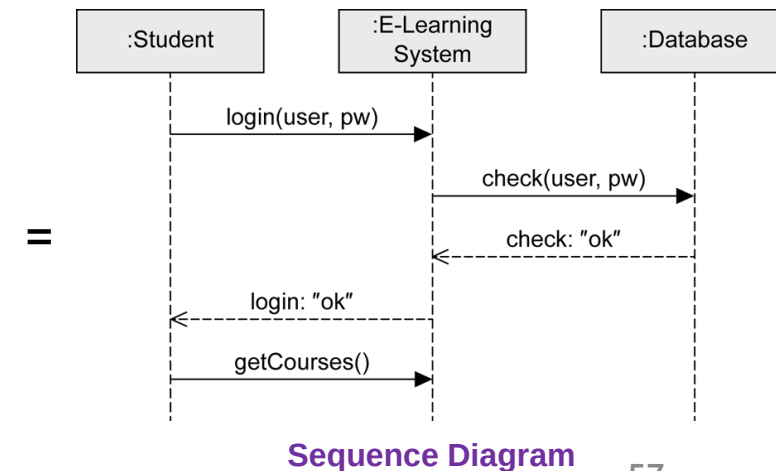
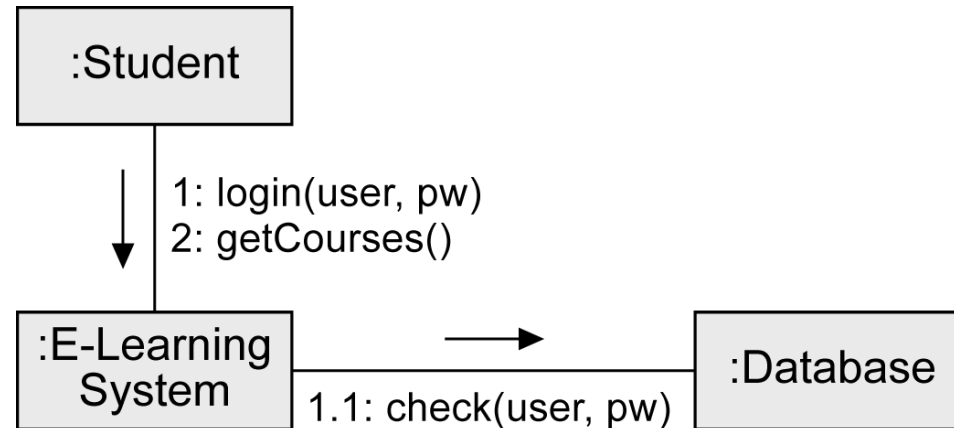
Sequence Diagram (OOD)

External Entities



9. Communication Diagram

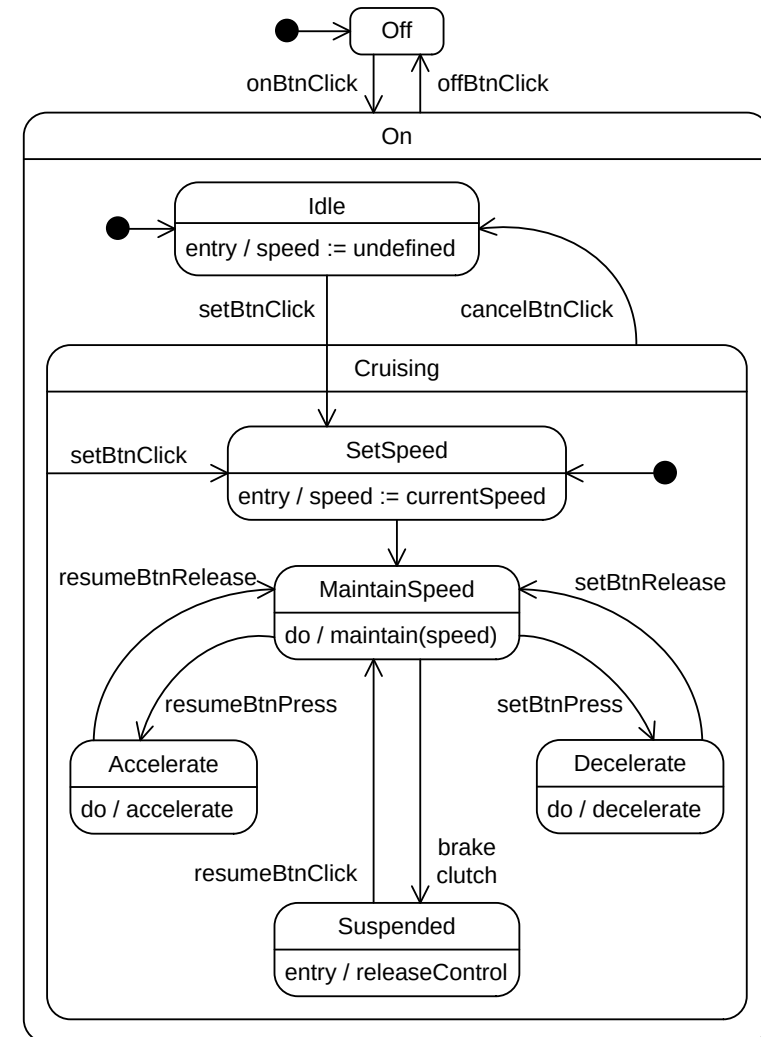
- **Communication diagrams** are used to model the dynamic behavior of a use case.
 - ≈ Sequence diagram
 - Not widely used for designing objects behaviors but explaining and documenting them.
 - More focus on showing the collaboration of objects rather than the time sequence.



10. State (Statechart) Diagram

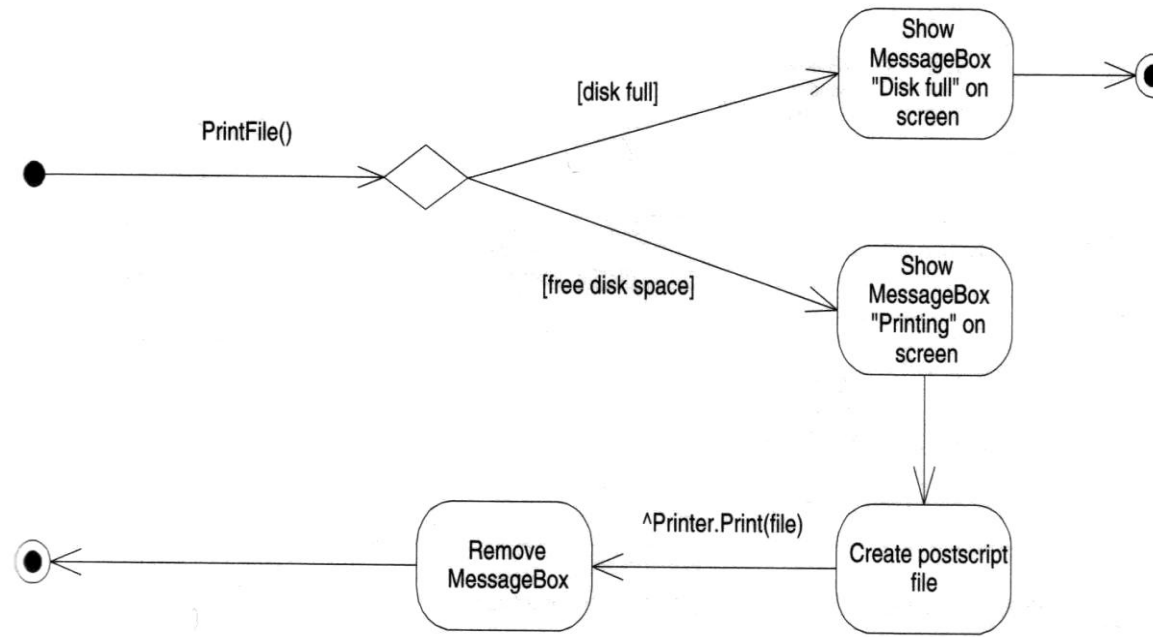


- **State diagrams** can show different states of an entity and how an entity responds to various events by changing from one state to another.
 - Originated from [the Statechart formalism](#)
 - The history of an entity is modeled by a finite state diagram.
 - All events are broadcast.
 - Composite state (OR state)
 - Orthogonal state (AND state)
 - History variables (H, H*)



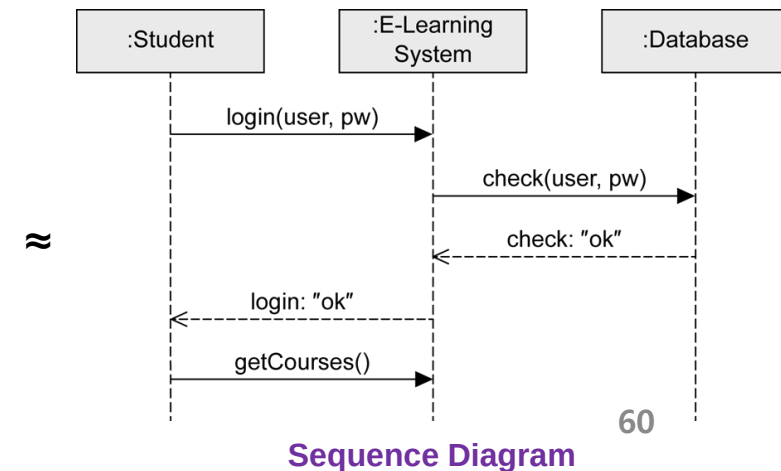
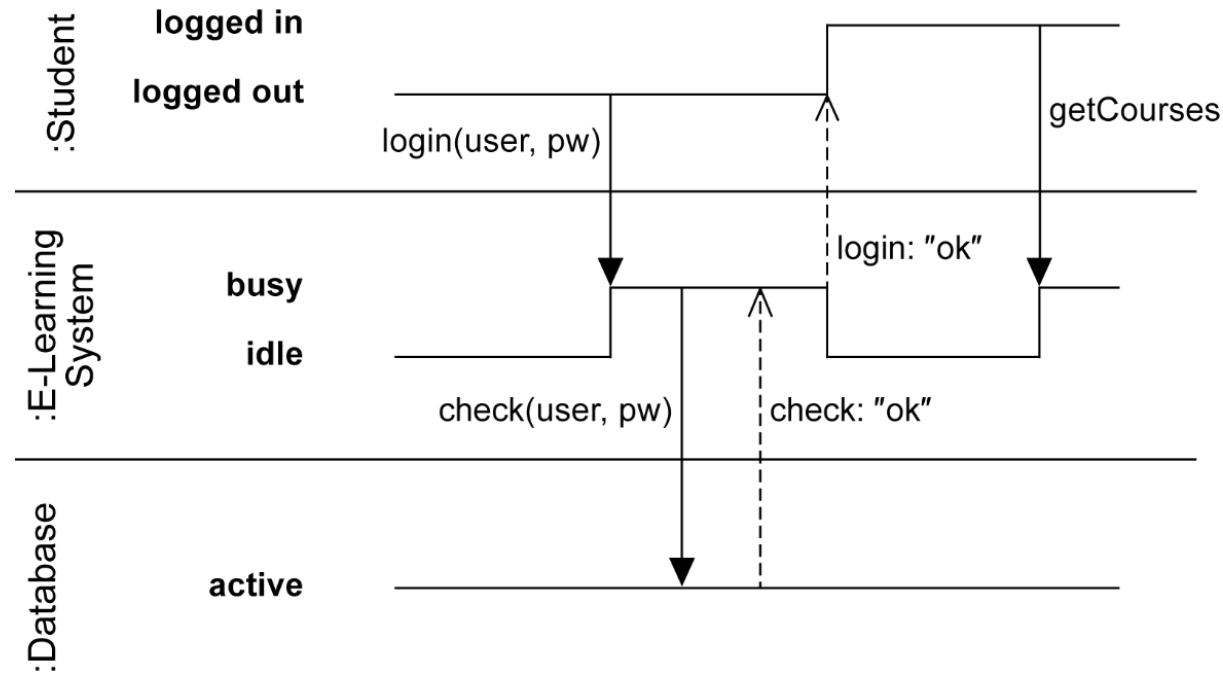
11. Activity Diagram

- **Activity diagrams** help to describe the flow of control of the target system.
 - Exploring complex business rules and operations, describing the use case and the business process.
 - It is an object-oriented equivalent of flow-charts and DFDs (data flow diagrams).



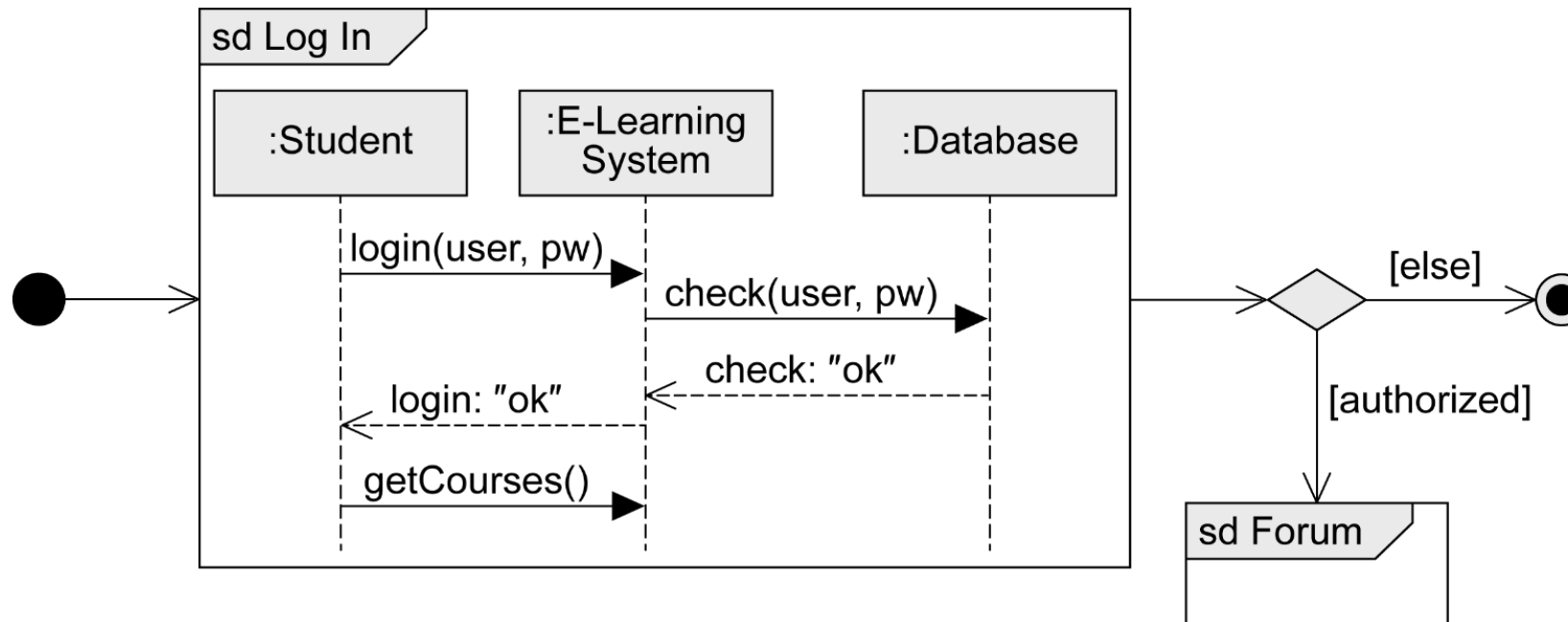
12. Timing Diagram

- **Timing diagrams** show the behavior of the objects in a given period of time.
 - A special form of a sequence diagram
 - The time increases from left to right and the lifelines are shown in separate compartments arranged vertically.



13. Interaction Overview Diagram

- **Interaction overview diagrams** focus on the overview of the flow of control of the interactions.
 - A variant of the **Activity Diagram**, where the nodes are the interactions or interaction occurrences.



A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue shadow above it, a white circle with a blue shadow further up, a circle with diagonal white stripes, and a 3x3 grid of small white dots.

An Introduction to UML

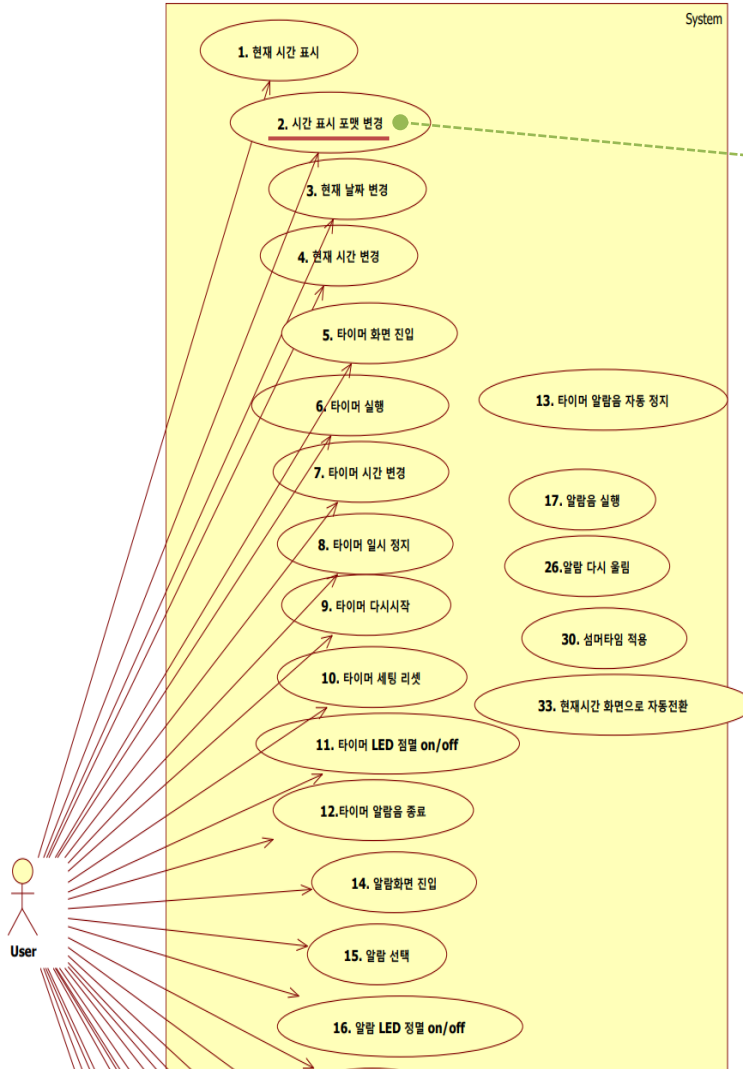
- Case Studies with UML

- Requirements Analysis (Planning and OOA)



Ref. #	Functional Requirements	Use Case Number & Name	Category	Category
R1.1	현재시간 보여주기	1. 현재 시간 표시	Primary	Hidden
R1.2	시간 형식(12hour, 24hour)	2. 시간 표시 포맷 변경	Primary	Evident
R1.3	날짜 설정	3. 현재 날짜를 변경한다	Primary	Evident
R1.4	시간 설정	4. 현재 시간을 변경한다	Primary	Evident
R2.1	타이머 화면	5. 모드 선택에서 타이머 기능으로 들어온다	Primary	Hidden
R2.2	타이머 시작	6. 타이머 시작 버튼을 누른다	Primary	Evident
R2.3	타이머 시간 설정	7. 타이머 시간을 변경한다	Primary	Evident
R2.4	타이머 일시 정지	8. 타이머를 일시 정지한다	Primary	Evident
R2.5	타이머 다시 시작	9. 타이머를 다시 시작한다.	Primary	Evident
R2.6	타이머 reset	10. 타이머 세팅을 리셋한다	Primary	Evident
R2.8	타이머 종료음 울림	11. 타이머 종료 시간 도달시 종료음 발생	Primary	Hidden
R2.9	타이머 종료음 정지	12. 타이머 종료음 울림중 종료한다	Primary	Evident
R2.10	타이머 종료음 자동 정지	13. 타이머 종료음 발생후 1분간 멈추지 않으면 자동 정지	Primary	Hidden
R3.1	알람 화면	14. 모드 선택에서 알람 기능으로 들어온다	Primary	Hidden
R3.2	알람 선택	15. 알람을 선택한다	Primary	Evident
R3.4	알람음 울림	16. 알람이 울린다	Primary	Hidden
R3.5	알람음 중지	17. 알람을 끝낸다	Primary	Evident
R3.6	알람음 자동 정지	18. 알람후 1분간 멈추지 않으면 자동 정지	Primary	Hidden
R3.6	알람 추가	19. 알람을 추가한다	Primary	Evident
R3.7	알람 삭제	20. 알람을 삭제한다	Primary	Evident
R3.8	알람 수정	21. 알람을 수정한다	Primary	Evident
R3.9	알람 리스트 보기	22. 알람리스트를 조회한다	Primary	Evident
R3.10	알람 세팅여부 표시	23. 알람존재 여부를 표시한다	Primary	Hidden
R3.11	알람 알람음 미리재생	24. 알람 알람음을 선택하여 미리 듣기한다	Primary	Evident
R3.12	알람 알람음 설정	25. 알람 알람음을 선택한다	Primary	Evident
R3.13	알람 다시 울림	26. 알람 다시 울림을 실행한다	Primary	Hidden
R4.1	도시 설정	27. 모드 선택에서 세계시간 기능으로 들어온다	Primary	Hidden
R4.2	도시 설정	28. 도시 설정 버튼을 누른다	Primary	Evident
R4.3	도시 리스트 보기	29. 도시 리스트를 조회한다	Primary	Evident
R4.4	썸머타임	30. 썸머타임이 날짜에 맞춰 적용 된다	Primary	Hidden
R5.1	모드 선택 기능	31. 모드 전환한다	Primary	Evident
R5.2	취소 기능	32. 취소를 실행한다	Primary	Evident
R5.3	모드 자동 전환	33. 현재시간 모드로 자동전환된다.(30초간 외부입력 없을시)	Primary	Hidden

Use Cases (OOA → OOD)



Use Case Diagram

Use Case Description (OOA)

Use Case	2. 시간 표시 포맷 변경
Actors	사용자
Description	12시간 표기법 or 24시간 표기법중에 사용자가 선택한다 12시간은 AM, PM등으로 구분되고 24시간은 없다(Hidden)



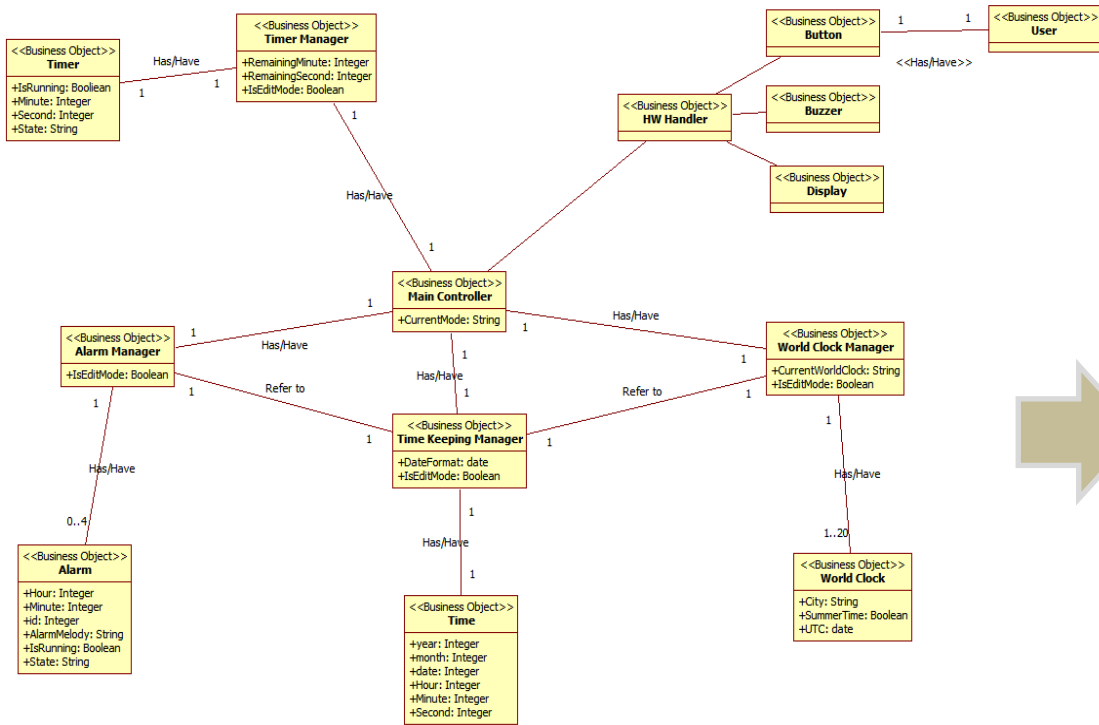
Use Case	2. 시간 표시 포맷 변경
Actor	Actor (Evident)
Purpose	-
Overview	12시간 표기법 or 24시간 표기법중에 사용자가 선택한다12시간은 AM, PM등으로 구분되고 24시간은 없다(Hidden)
Type	-
Cross Reference	-
Pre-Requisites	현재시간이 보여지고 있을 때, Default 화면일 때
Typical Courses of Events	1. (A) 사용자가 R1 버튼을 길게 누른다. 2. (S) 현재 시간 표시 포맷 상태를 가져온다. 3. (S) 24시간 포맷이라면 12시간 포맷으로, 12시간 포맷이라면 24시간 포맷으로 상태를 update 한다. 4. (S) 상태값이 Update 되었다면, 변경된 포맷에 따라 시간의 값을 변경한다 5. (S) 변경된 포맷으로 현재시간 표시 모드로 전환된다
Alternative Courses of Events	Line 4 : (S) 시간이 변경되어 update 되는 시점에 Line 4에 의한 시간값 변경 시
Exceptional Courses of Events	(S) 시간 Update 후, 포맷 변경 변경에 따른 시간 변경 N/A

Use Case Description (OOD)

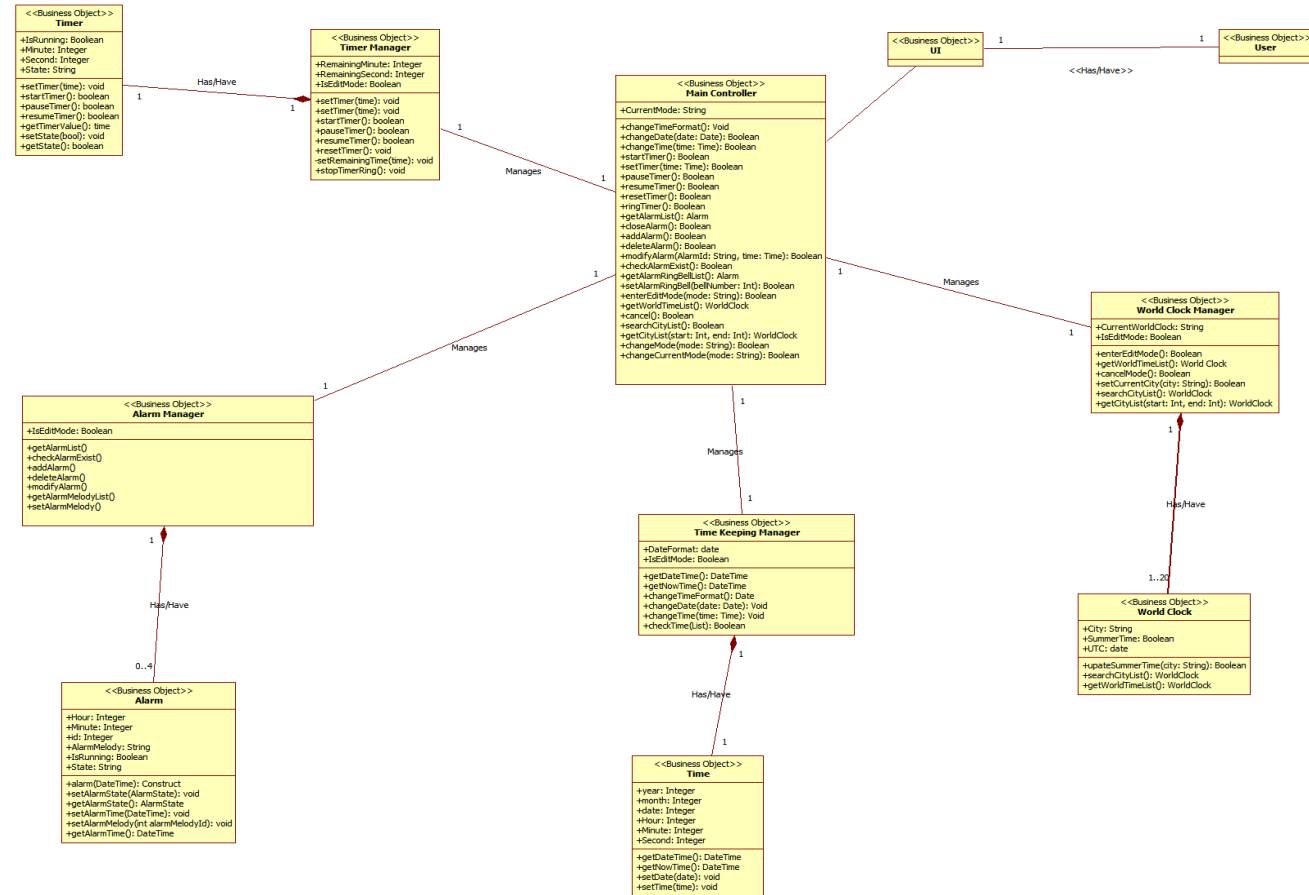
Class Diagram (OOA → OOD)



SE



Domain Model (OOA)



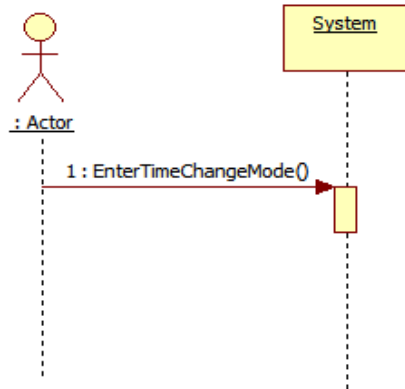
Design Class Diagram (OOD)

Sequence Diagram (OOA → OOD)



2. 시간 표시 포맷 변경

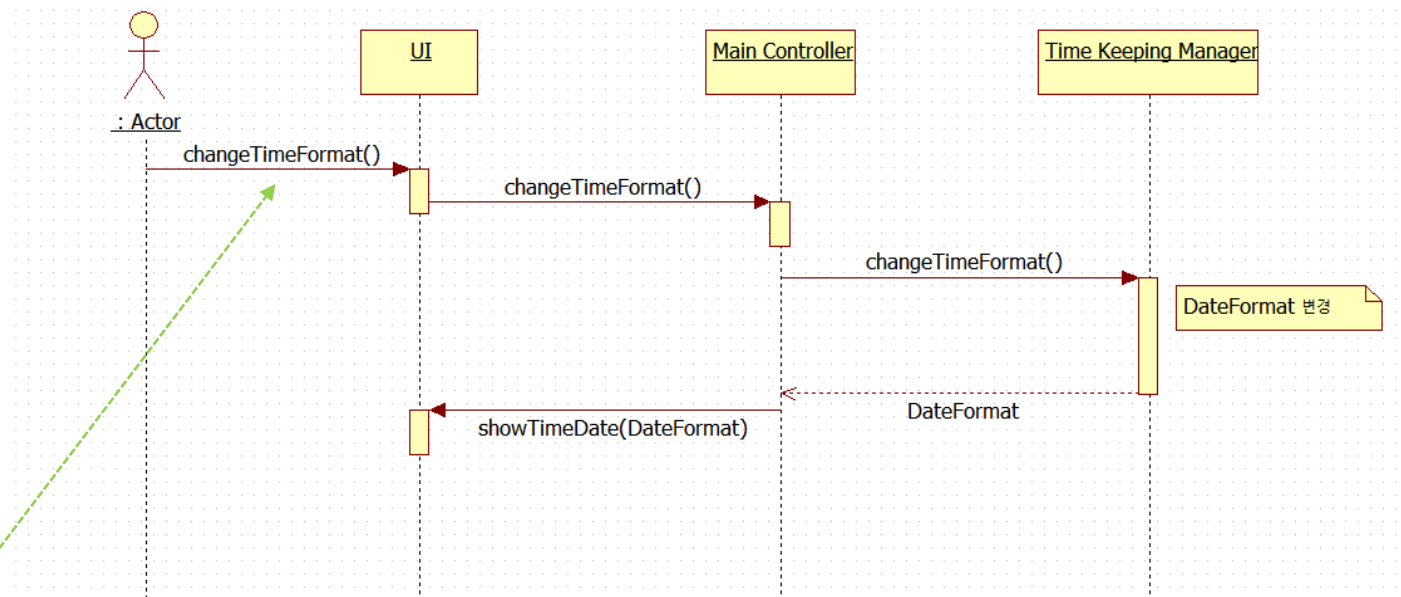
Use Case	2. 시간 표시 포맷 변경
Actor	Actor (Evident)
Purpose	-
Overview	12시간 표기법 or 24시간 표기법중에 사용자가 선택한다12시간은 AM, PM등으로 구분되고 24시간은 없다(hidden)
Type	-
Cross Reference	-
Pre-Requisites	현재시간이 보여지고 있을 때, Default 화면일 때
Typical Courses of Events	1. (A) 사용자가 R1 버튼을 길게 누른다. 2. (S) 현재 시간 표시 포맷 상태를 가져온다. 3. (S) 24시간 포맷이라면 12시간 포맷으로, 12시간 포맷이라면 24시간 포맷으로 상태를 update 한다. 4. (S) 상태값이 Update 되었다면, 변경된 포맷에 따라 시간의 값을 변경한다 5. (S) 변경된 포맷으로 현재시간 표시 모드로 전환된다
Alternative Courses of Events	Line 4 : (S) 시간이 변경되어 update 되는 시점에 Line 4에 의한 시간값 변경 시 (S) 시간 Update 후, 포맷 변경 변경에 따른 시간 변경
Exceptional Courses of Events	N/A



System Sequence Diagram (OOA)



The name of system operation is more clarified !!



Sequence Diagram (OOD)

Sequence Diagram



SE

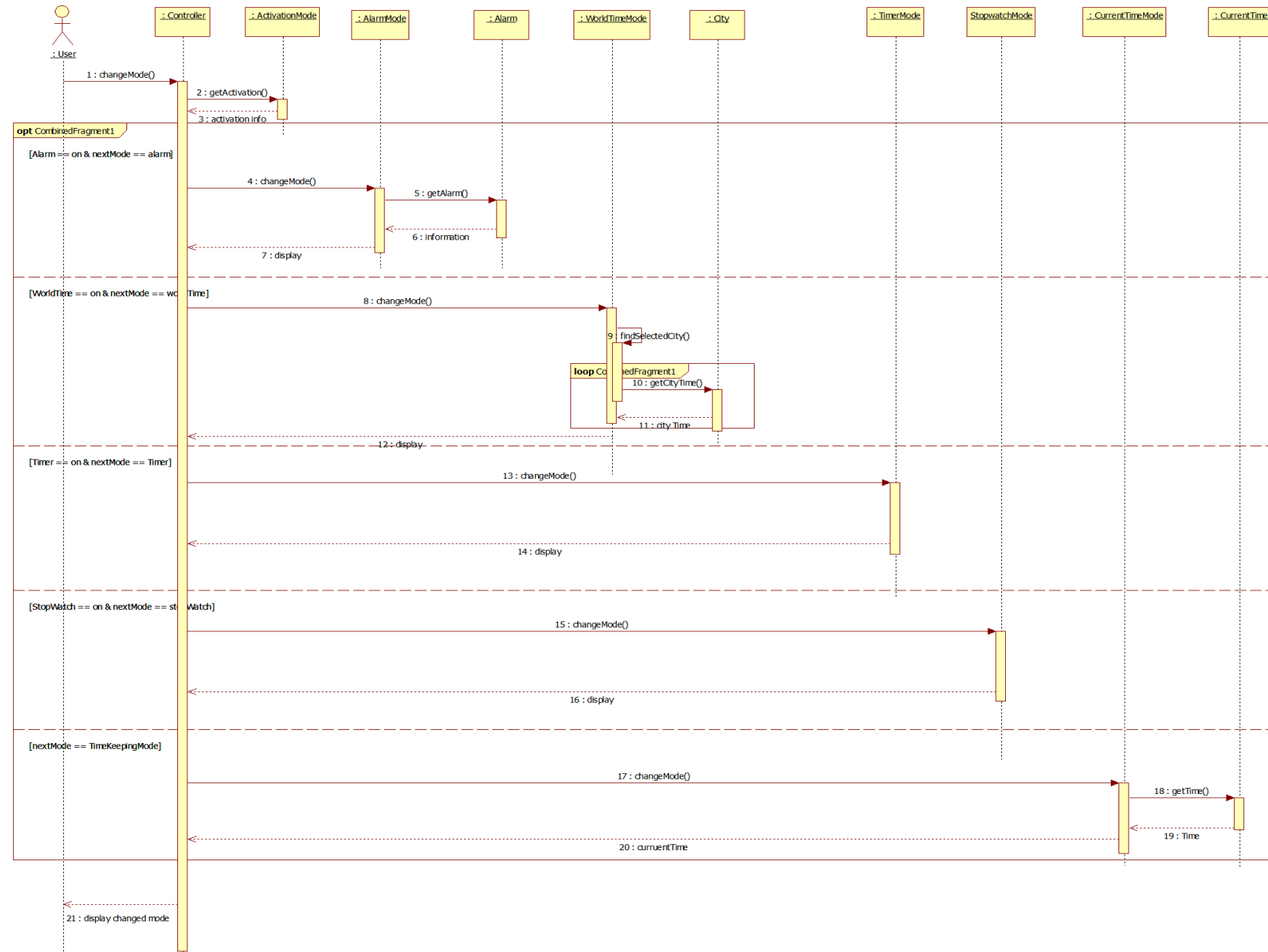


UC-31: Change Mode

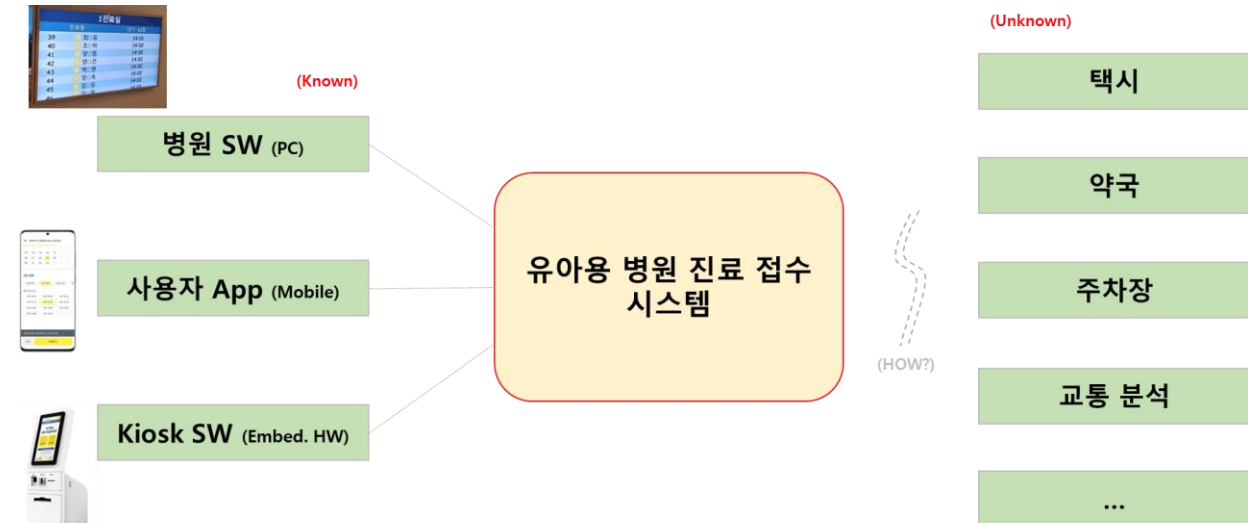


Timekeeping

- Alarm → World Time → Timer
- Stopwatch → Timekeeping



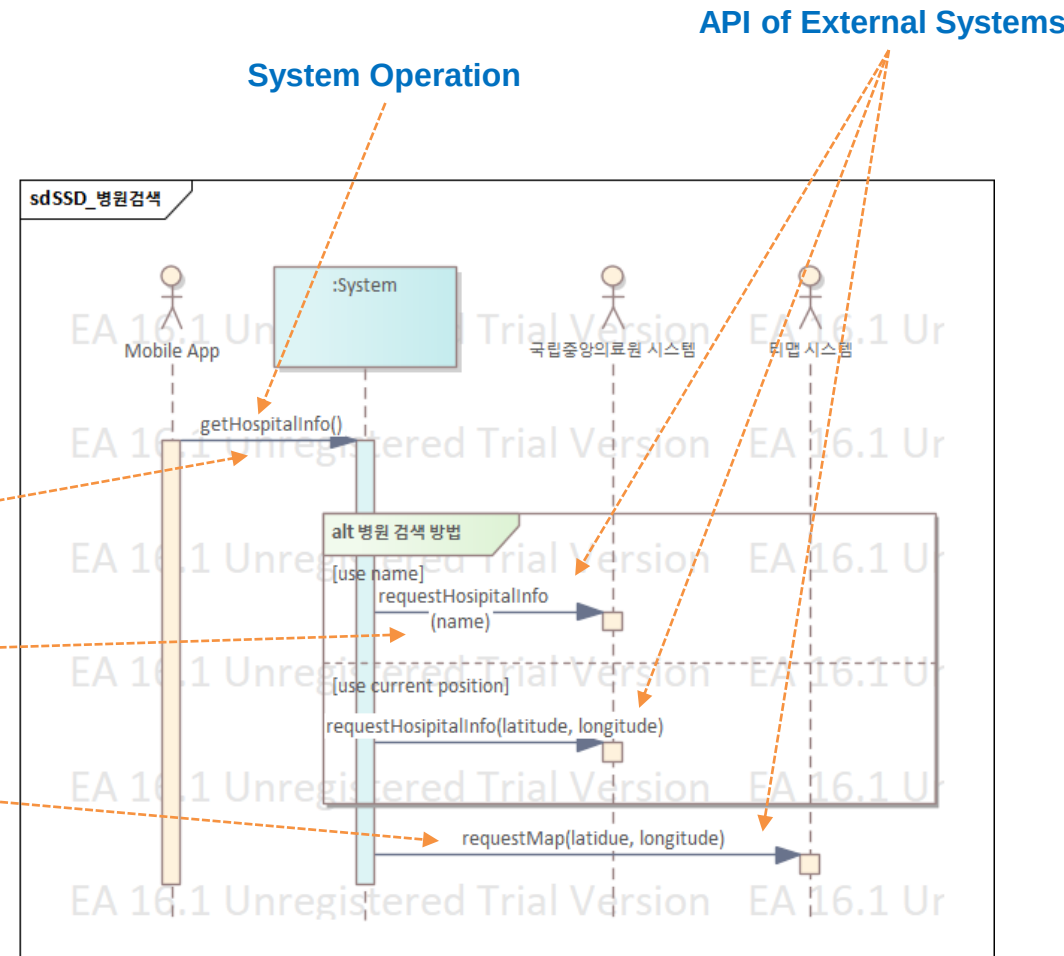
System Description



Use Cases (OOA at System Level)

Use Case	병원 검색
Actor	Mobile App, 국립중앙의료원 시스템, 티맵 시스템
Description	Mobile App은 병원 검색을 요청하고 System은 검색 결과를 Mobile App으로 전달한다.
Stakeholders	환자, 국립중앙의료원 공공데이터 관리자, 티맵모빌리티 관리자
Preconditions	로그인 상태 위치 정보 권한을 부여한 상태 스마트폰 네트워크 사용 가능한 상태
Main Scenario	(A₁) : Mobile App, (A₂) : 국립중앙의료원 시스템, (A₃) : 티맵 시스템, (S) : System 1. (A₁) 검색하고 싶은 병원을 입력하여 검색 요청 2. (S) 검색 요청된 결과 확인 3. (S) 국립중앙의료원 시스템에 검색 요청 4. (A₂) 검색 결과 반환 5. (S) 티맵 시스템에 지도 이미지 요청 6. (A₃) 지도 이미지 반환 7. (S) Mobile App에 검색 결과 전달
Alternative Scenario	[현재 위치 기반 검색을 요청한 경우] 2. (S) 현재 위치 정보를 사용

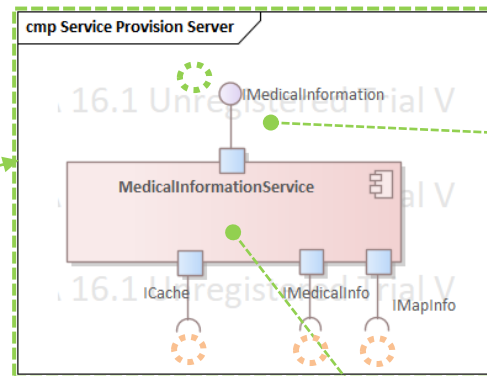
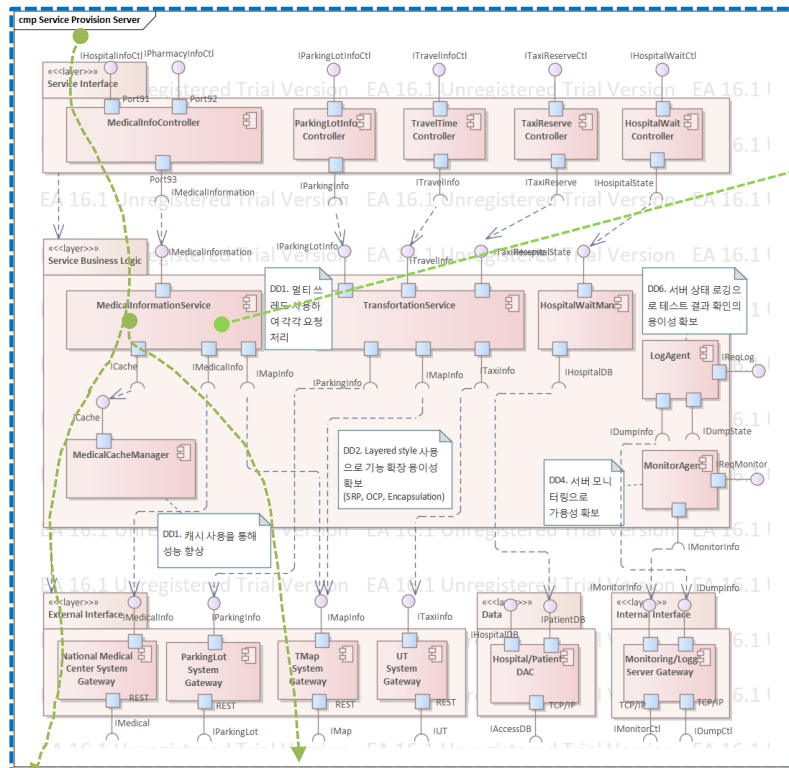
Use Case



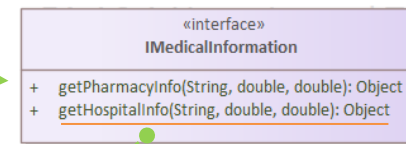
System Sequence Diagram

Component Diagram Multiple App Server Architecture

병원 예약 / 접수 필수 앱

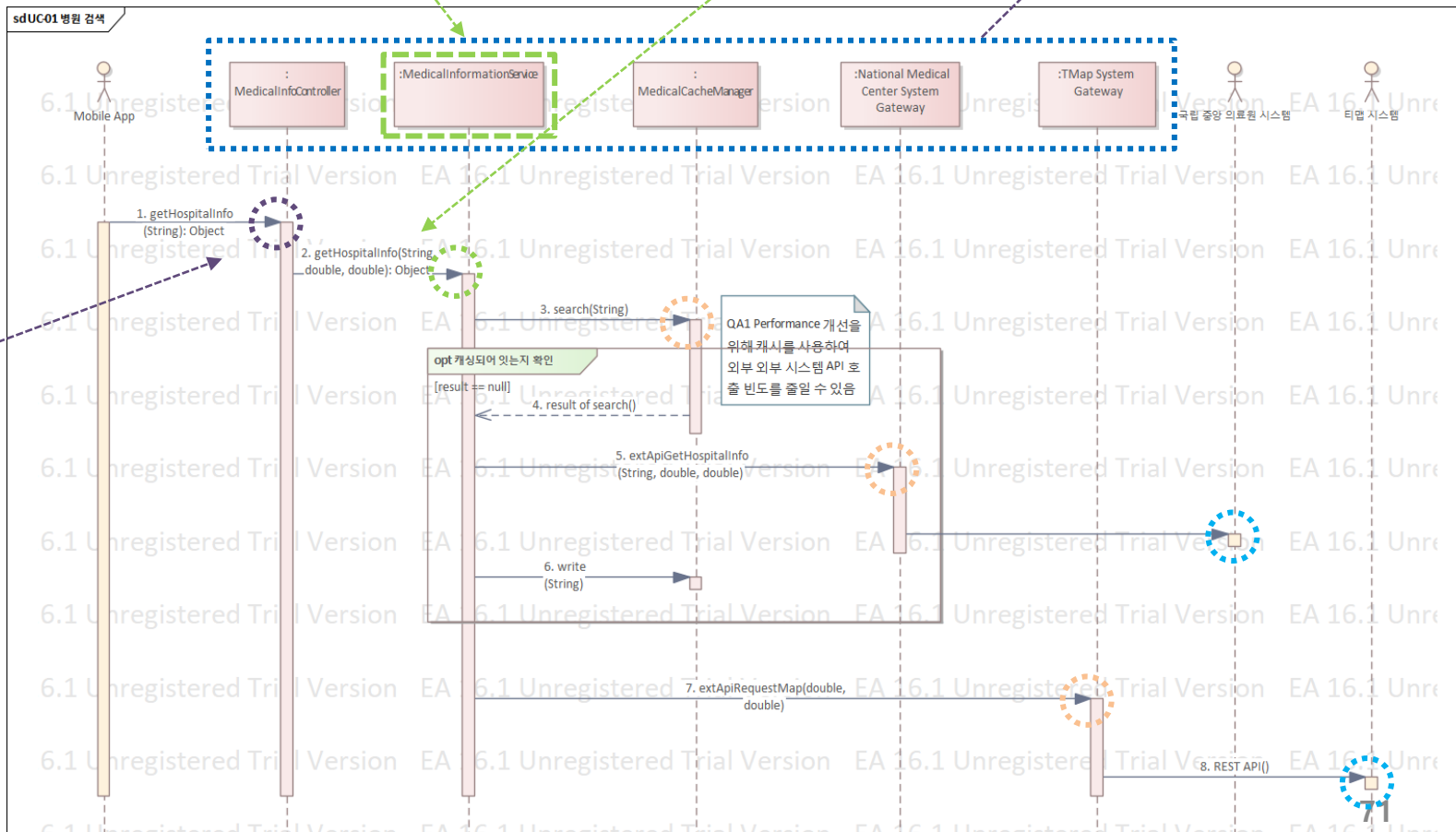


Provided Interface

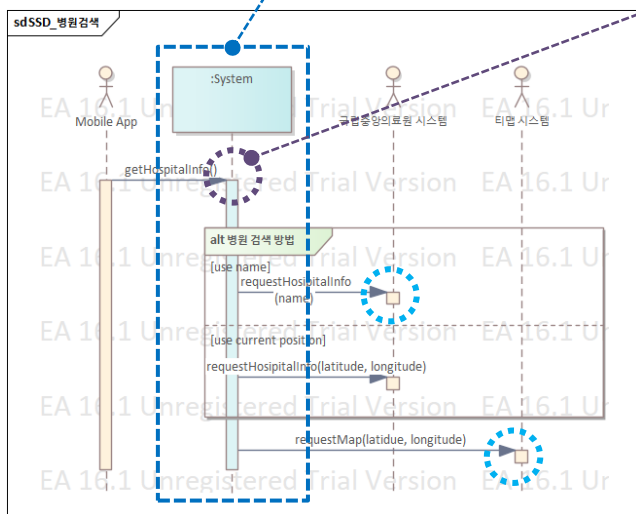


Component

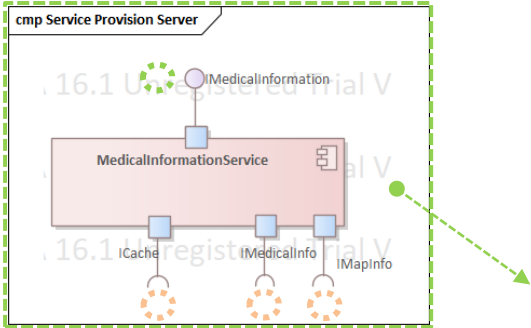
System Boundary



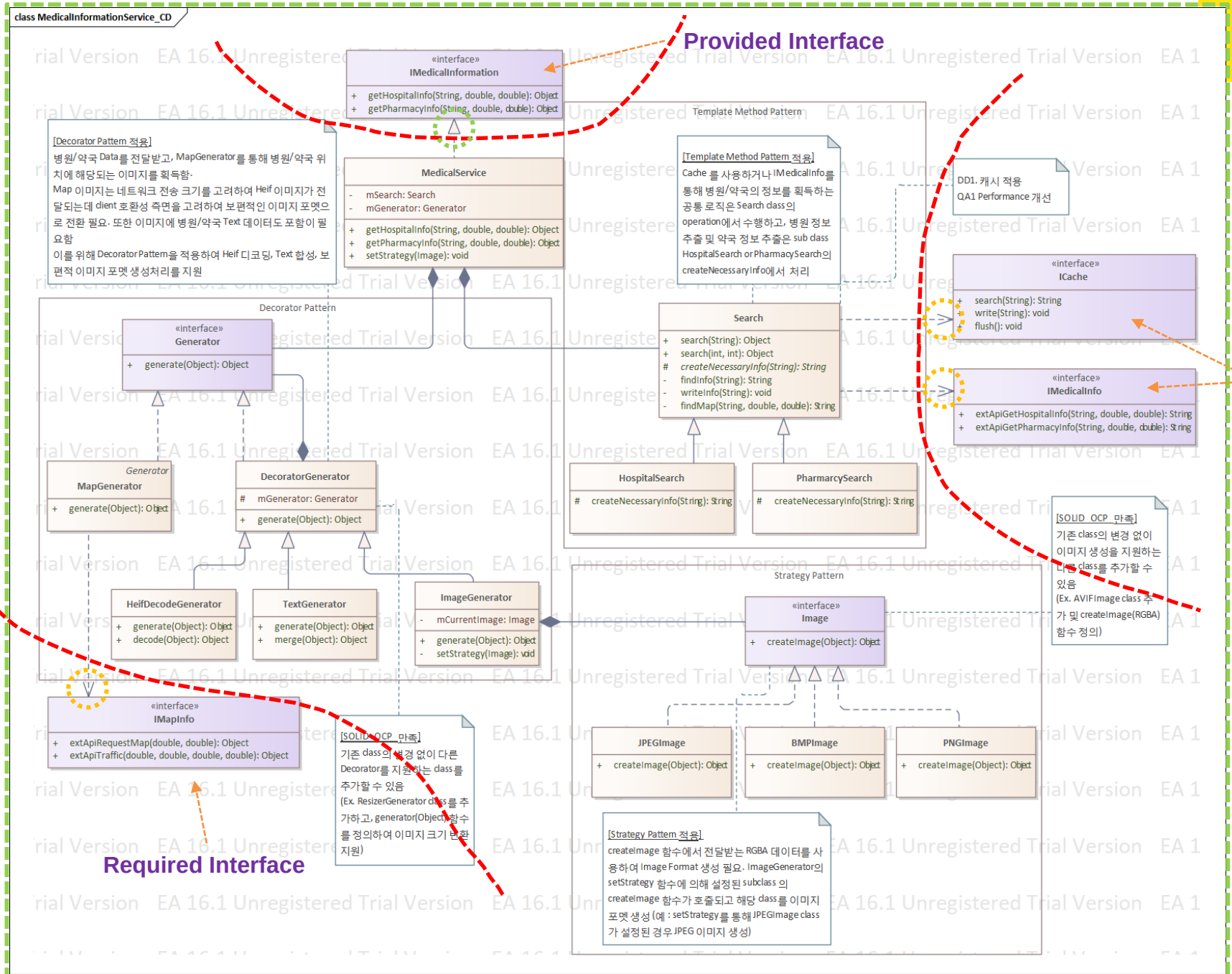
(Omitted Below)



OOAD (Static Model)

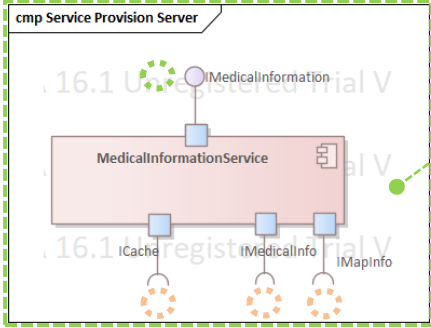


Class Diagram for the Component (MedicalInformationService)

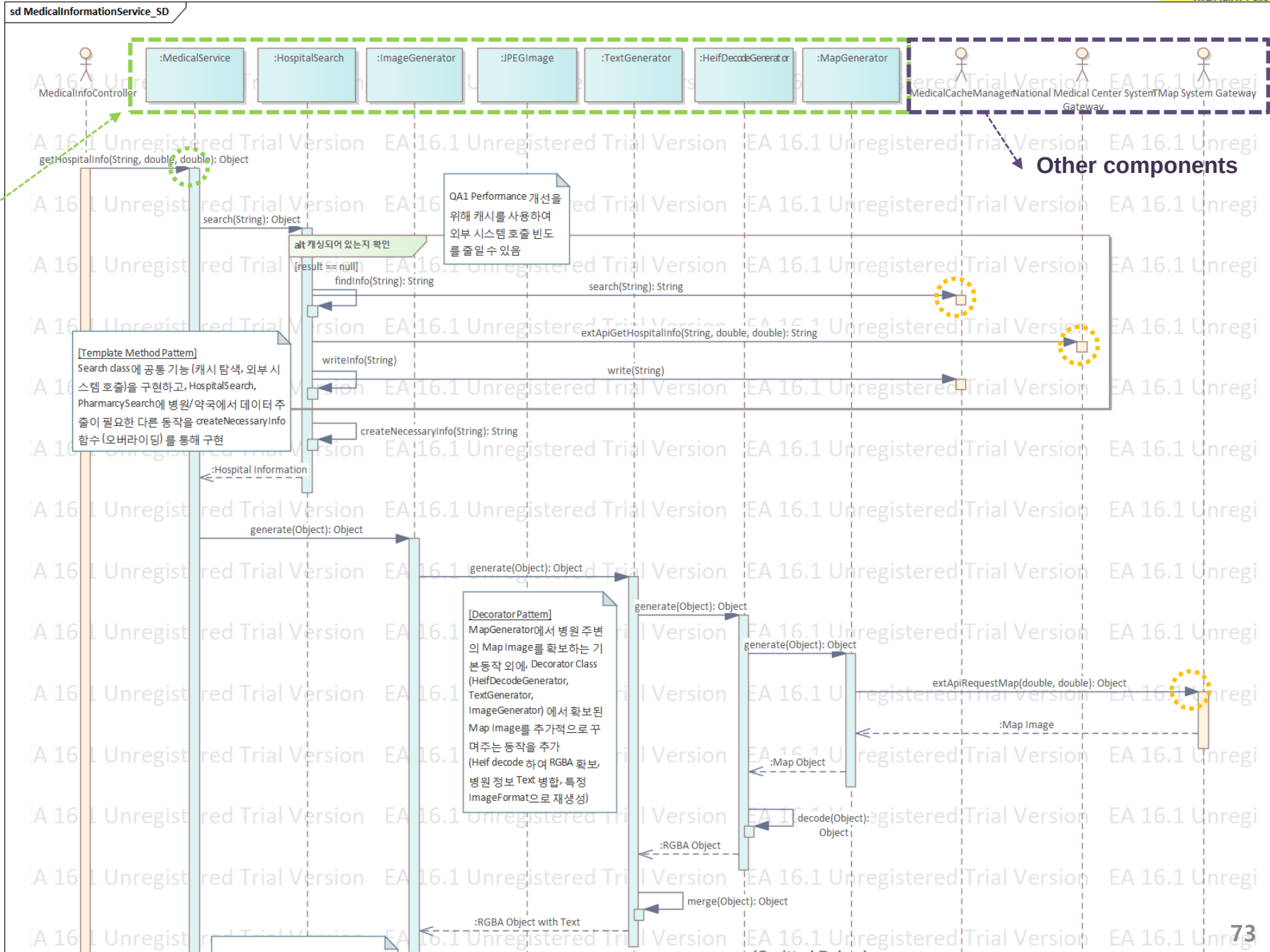


Required Interface

OOAD (Dynamic Model)



Sequence Diagram for the Component *(MedicalInformationService)*



A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a white circle with a blue outline further up, a dark blue circle with a white diagonal stripe pattern at the top right, and a small white circle with a blue outline near the top left. There is also a small white circle with a blue outline near the bottom right.

OOA

- Use Case

An OOAD Example - Dice Game



Define use cases

Define domain model

Define interaction diagrams

Define design class diagrams

OOA

Use Case : **Play a Dice Game**

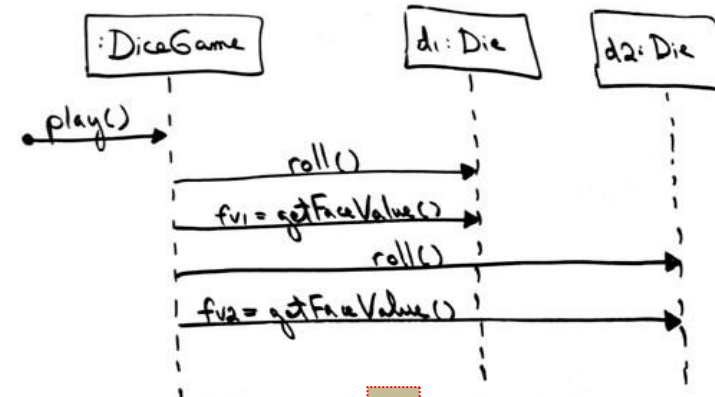
- Player requests to roll the dice.
- System presents results.
- If the dice's face value totals seven, player wins; otherwise, player loses.



Domain Model

OOD

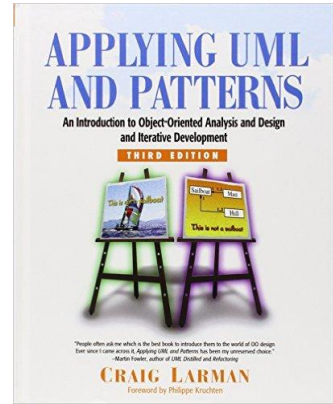
Interaction Diagram (Sequence Diagram)



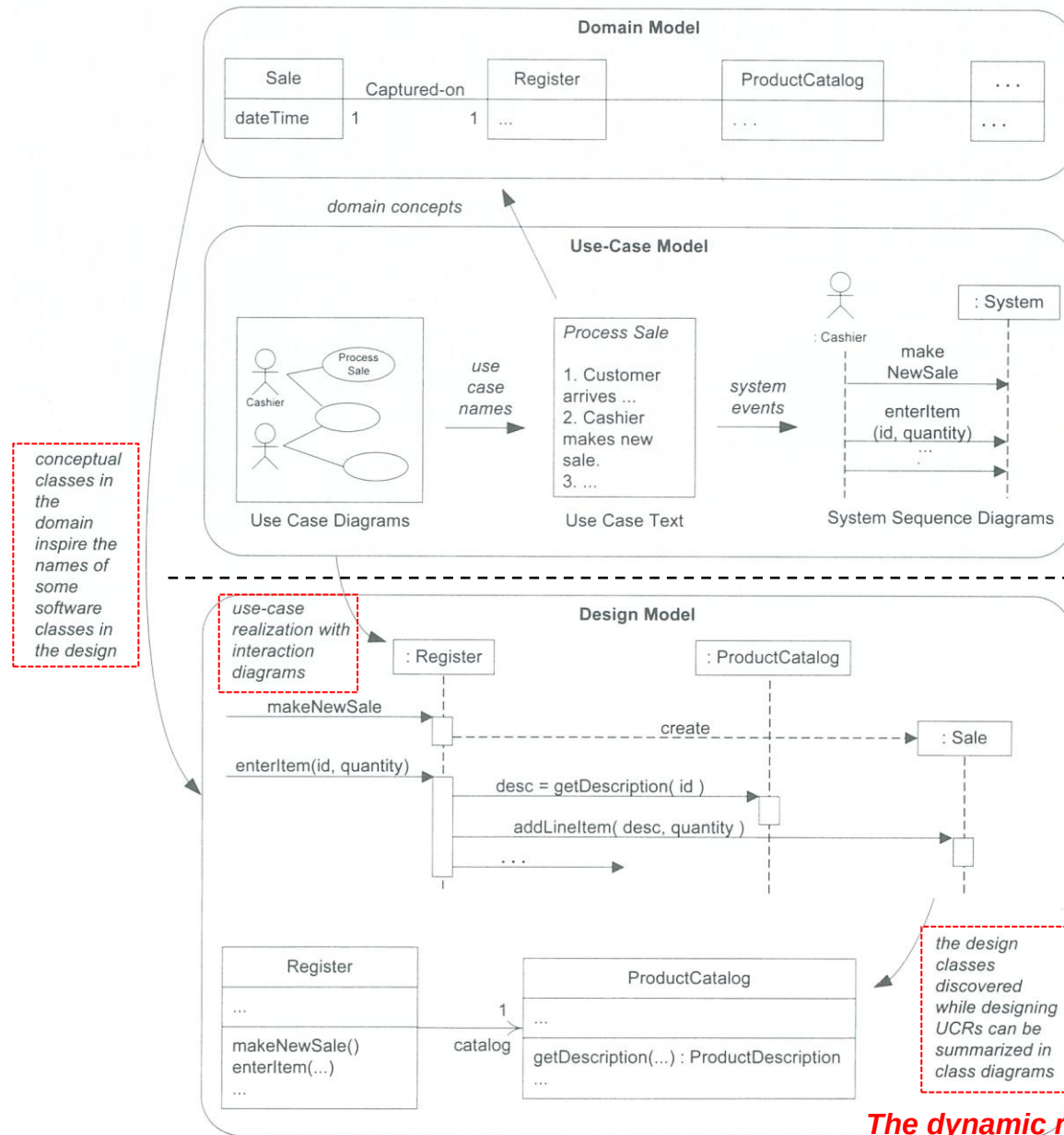
Dynamic model-driven !!!



Design Class Diagram



Sample Unified Process Artifact Relationships



Domain Model

Use-Case Model
(+System Sequence Diagram)

OOA

OOD

Sequence Diagrams

Class Diagram

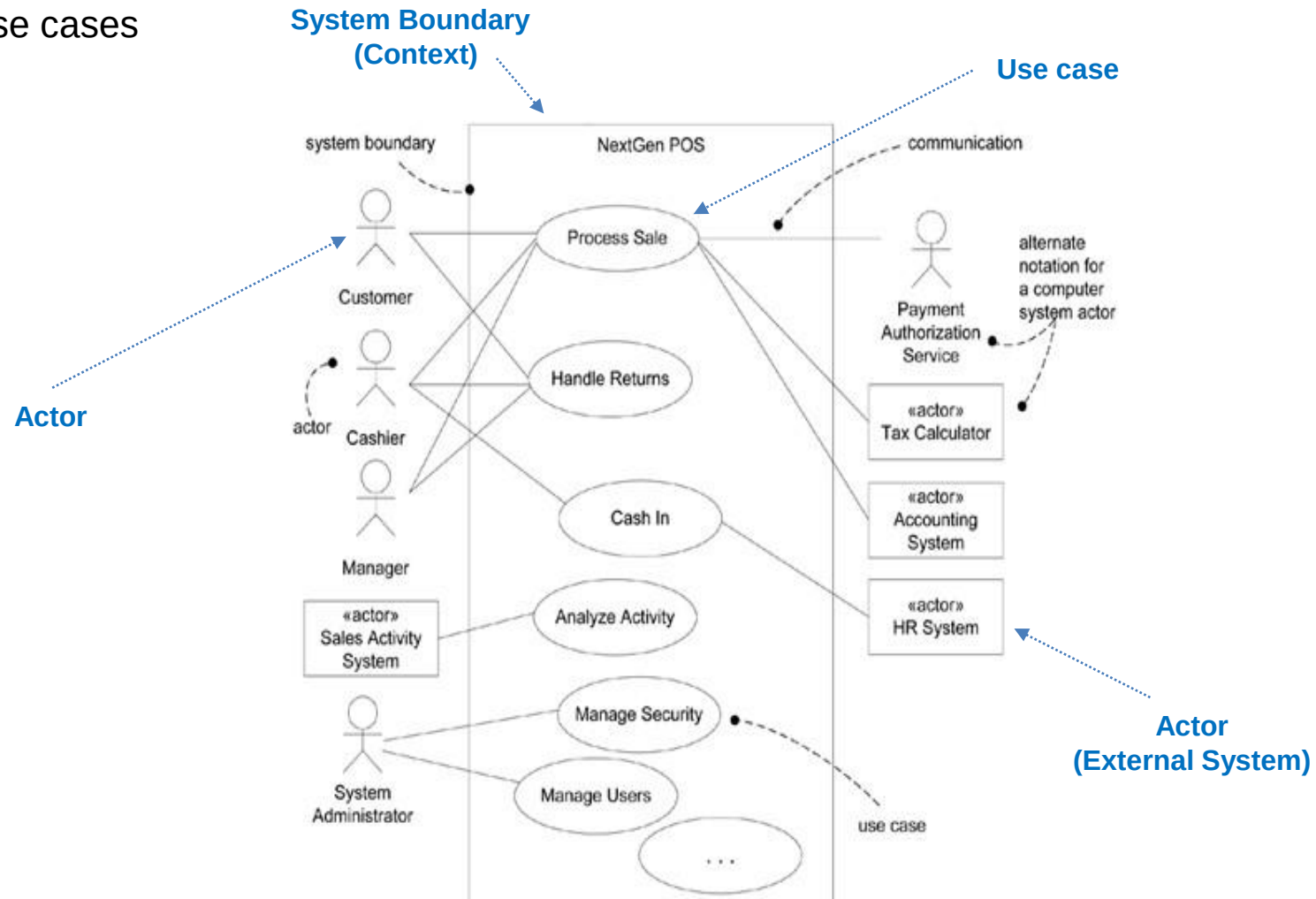
The dynamic model-driven approach!!!

OOI

- **Use cases** are **text stories** of some actors using a system to meet goals.
 - A mechanism to capture (analyzes) requirements
 - An example (Brief format):
 - **Process Sale:** *A customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each purchased item. The system presents a running total and line-item details. The customer enters payment information, which the system validates and records. The system updates inventory. The customer receives a receipt from the system and then leaves with the items.*
- Use case is not a diagram, but a **text**.
 - Primarily functional requirements (FR)

Use Case Section	Comment
Use Case Name	Start with a verb.
Scope	The system under design.
Level	"user-goal" or "subfunction"
Primary Actor	Calls on the system to deliver its services.
Stakeholders and Interests	Who cares about this use case, and what do they want?
<u>Preconditions</u>	What must be true on start, <i>and</i> worth telling the reader?
<u>Success Guarantee</u>	What must be true on successful completion, <i>and</i> worth telling the reader.
<u>Main Success Scenario</u>	A typical, unconditional happy path scenario of success.
<u>Extensions</u>	Alternate scenarios of success or failure.
Special Requirements	Related non-functional requirements.
Technology and Data Variations List	Varying I/O methods and data formats.
Frequency of Occurrence	Influences investigation, testing, and timing of implementation.
Miscellaneous	Such as open issues.

- **Use case diagram** illustrates the name of use cases and actors, and the relationships between them.
 - System context diagram
 - A **summary** of all use cases



3 Common Formats of Use Case

- **Brief**

- Terse one paragraph summary
- Usually, the main success scenario

- **Casual**

- Multiple paragraphs that cover various scenarios.

- **Fully Dressed**

- Includes all steps, variations and supporting sections

Handle Returns

Main Success Scenario: A customer arrives at a checkout with items to return. The cashier uses the POS system to record each returned item ...

Alternate Scenarios:

If the customer paid by credit, and the reimbursement transaction to their credit account is rejected, inform the customer and pay them with cash.

If the item identifier is not found in the system, notify the Cashier and suggest manual entry of the identifier code (perhaps it is corrupted).

If the system detects failure to communicate with the external accounting system, ...

Use Case Section	Comment
Use Case Name	Start with a verb.
Scope	The system under design.
Level	"user-goal" or "subfunction"
Primary Actor	Calls on the system to deliver its services.
Stakeholders and Interests	Who cares about this use case, and what do they want?
Preconditions	What must be true on start, <i>and</i> worth telling the reader?
Success Guarantee	What must be true on successful completion, <i>and</i> worth telling the reader.
Main Success Scenario	A typical, unconditional happy path scenario of success.
Extensions	Alternate scenarios of success or failure.
Special Requirements	Related non-functional requirements.
Technology and Data Variations List	Varying I/O methods and data formats.
Frequency of Occurrence	Influences investigation, testing, and timing of implementation.
Miscellaneous	Such as open issues.

Alternatives & Exceptions

Example: (UC1) Process Sale



Stakeholders' needs and interests
→ Really important to architecture design

Use Case UC1: Process Sale

Scope: NextGen POS application

Level: user goal

Primary Actor: Cashier

Stakeholders and Interests:

- Cashier: Wants accurate, fast entry, and no payment errors, as cash drawer shortages are deducted from his/her salary.
- Salesperson: Wants sales commissions updated.
- Customer: Wants purchase and fast service with minimal effort. Wants easily visible display of entered items and prices. Wants proof of purchase to support returns.
- Company: Wants to accurately record transactions and satisfy customer interests. Wants to ensure that Payment Authorization Service payment receivables are recorded. Wants some fault tolerance to allow sales capture even if server components (e.g., remote credit validation) are unavailable. Wants automatic and fast update of accounting and inventory.
- Manager: Wants to be able to quickly perform override operations, and easily debug Cashier problems.
- Government Tax Agencies: Want to collect tax from every sale. May be multiple agencies, such as national, state, and county.
- Payment Authorization Service: Wants to receive digital authorization requests in the correct format and protocol. Wants to accurately account for their payables to the store.

Preconditions: Cashier is identified and authenticated.

Success Guarantee (or Postconditions): Sale is saved. Tax is correctly calculated. Accounting and Inventory are updated. Commissions recorded. Receipt is generated. Payment authorization approvals are recorded.

Main Success Scenario (or Basic Flow):

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total.
Price calculated from a set of price rules.
Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting system (for accounting and commissions) and Inventory system (to update inventory).
9. System presents receipt.
10. Customer leaves with receipt and goods (if any).

Extensions (or Alternative Flows):

*a. At any time, Manager requests an override operation:

1. System enters Manager-authorized mode.
2. Manager or Cashier performs one Manager-mode operation. e.g., cash balance change, resume a suspended sale on another register, void a sale, etc.
3. System reverts to Cashier-authorized mode.

*b. At any time, System fails:

To support recovery and correct accounting, ensure all transaction sensitive state and events can be recovered from any step of the scenario.

1. Cashier restarts System, logs in, and requests recovery of prior state.
2. System reconstructs prior state.

2a. System detects anomalies preventing recovery:

1. System signals error to the Cashier, records the error, and enters a clean state.
2. Cashier starts a new sale.

1a. Customer or Manager indicate to resume a suspended sale.

1. Cashier performs resume operation, and enters the ID to retrieve the sale.
2. System displays the state of the resumed sale, with subtotal.

2a. Sale not found.

1. System signals error to the Cashier.
2. Cashier probably starts new sale and re-enters all items.

3. Cashier continues with sale (probably entering more items or handling payment).

2-4a. Customer tells Cashier they have a tax-exempt status (e.g., seniors, native peoples)

1. Cashier verifies, and then enters tax-exempt status code.
2. System records status (which it will use during tax calculations)

3a. Invalid item ID (not found in system):

1. System signals error and rejects entry.

2. Cashier responds to the error:

2a. There is a human-readable item ID (e.g., a numeric UPC):

1. Cashier manually enters the item ID.
2. System displays description and price.

2a. Invalid item ID: System signals error. Cashier tries alternate method.

2b. There is no item ID, but there is a price on the tag:

1. Cashier asks Manager to perform an override operation.

2. Manager performs override.

3. Cashier indicates manual price entry, enters price, and requests standard taxation for this amount (because there is no product information, the tax engine can't otherwise deduce how to tax it)

2c. Cashier performs Find Product Help to obtain true item ID and price.

2d. Otherwise, Cashier asks an employee for the true item ID or price, and does either manual ID or manual price entry (see above).

3b. There are multiple of same item category and tracking unique item identity not important (e.g., 5 packages of veggie-burgers):

1. Cashier can enter item category identifier and the quantity.

3c. Item requires manual category and price entry (such as flowers or cards with a price on them):

1. Cashier enters special manual category code, plus the price.

3-6a: Customer asks Cashier to remove (i.e., void) an item from the purchase:

This is only legal if the item value is less than the void limit for Cashiers, otherwise a Manager override is needed.

1. Cashier enters item identifier for removal from sale.

2. System removes item and displays updated running total.

2a. Item price exceeds void limit for Cashiers:

1. System signals error, and suggests Manager override.
2. Cashier requests Manager override, gets it, and repeats operation.

3-6b. Customer tells Cashier to cancel sale:

1. Cashier cancels sale on System.

3-6c. Cashier suspends the sale:

1. System records sale so that it is available for retrieval on any POS register.
2. System presents a "suspend receipt" that includes the line items, and a sale ID used to retrieve and resume the sale.

4a. The system supplied item price is not wanted (e.g., Customer complained about something and is offered a lower price):

1. Cashier requests approval from Manager.
2. Manager performs override operation.
3. Cashier enters manual override price.
4. System presents new price.

5a. System detects failure to communicate with external tax calculation system service:

1. System restarts the service on the POS node, and continues.
 - 1a. System detects that the service does not restart.
 1. System signals error.
 2. Cashier may manually calculate and enter the tax, or cancel the sale.

5b. Customer says they are eligible for a discount (e.g., employee, preferred customer):

1. Cashier signals discount request.
2. Cashier enters Customer identification.
3. System presents discount total, based on discount rules.

5c. Customer says they have credit in their account, to apply to the sale:

1. Cashier signals credit request.
2. Cashier enters Customer identification.
3. System applies credit up to price=0, and reduces remaining credit.

6a. Customer says they intended to pay by cash but don't have enough cash:

1. Cashier asks for alternate payment method.
 - 1a. Customer tells Cashier to cancel sale. Cashier cancels sale on System.

7a. Paying by cash:

1. Cashier enters the cash amount tendered.
2. System presents the balance due, and releases the cash drawer.
3. Cashier deposits cash tendered and returns balance in cash to Customer.
4. System records the cash payment.

7b. Paying by credit:

1. Customer enters their credit account information.
2. System displays their payment for verification.
3. Cashier confirms.
 - 3a. Cashier cancels payment step:
 1. System reverts to "item entry" mode.
4. System sends payment authorization request to an external Payment Authorization Service System, and requests payment approval.
 - 4a. System detects failure to collaborate with external system:
 1. System signals error to Cashier.
 2. Cashier asks Customer for alternate payment.
5. System receives payment approval, signals approval to Cashier, and releases cash drawer (to insert signed credit payment receipt).
 - 5a. System receives payment denial:
 1. System signals denial to Cashier.
 2. Cashier asks Customer for alternate payment.
 - 5b. Timeout waiting for response.
 1. System signals timeout to Cashier.
 2. Cashier may try again, or ask Customer for alternate payment.
6. System records the credit payment, which includes the payment approval.
7. System presents credit payment signature input mechanism.
8. Cashier asks Customer for a credit payment signature. Customer enters signature.
9. If signature on paper receipt, Cashier places receipt in cash drawer and closes it.

7c. Paying by check...

7d. Paying by debit...

7e. Cashier cancels payment step:

1. System reverts to "item entry" mode.

7f. Customer presents coupons:

1. Before handling payment, Cashier records each coupon and System reduces price as appropriate. System records the used coupons for accounting reasons.
 - 1a. Coupon entered is not for any purchased item:
 1. System signals error to Cashier.

9a. There are product rebates:

1. System presents the rebate forms and rebate receipts for each item with a rebate.

9b. Customer requests gift receipt (no prices visible):

1. Cashier requests gift receipt and System presents it.

9c. Printer out of paper.

1. If System can detect the fault, will signal the problem.
2. Cashier replaces paper.
3. Cashier requests another receipt.

- Write in an **Essential UI-Free Style**
 - Express **user intentions** and system responsibilities, rather than concrete actions.
 - Concrete use cases are better avoided during early requirements analysis.
 - For example: **Manage Users** use case

Essential Style	Concrete Style
1. Administrator identities self. 2. System authenticates identity. 3. ...	1. Administrator enters ID and PW in dialog box. 2. System authenticates Administrator. 3. System displays the “edit user” window. 4. ...

- Write **Black-Box Use Cases**
 - **Don't** describe **the internal working** of the system, its components or design.
 - Define what the system does (*analysis*), rather than how it does it (*design*).

Black-box style	Not
The system records the sale.	The system writes the sale to a database. ...or (even worse): The system generates a SQL INSERT statement for the sale...

A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a white circle with a blue outline further up, a dark blue circle with a white outline above that, and a white circle with a blue outline at the top. There are also some diagonal lines and a grid of small white dots in the lower right of the sidebar.

OOA

- System Sequence Diagram

- **System sequence diagram (SSD)**

- A picture that shows the events that external actors generate, their order, and inter-system events, for one particular scenario of a use case.
 - the external actors that interact directly with the system,
 - the system (as a black box), and
 - the system events that the actors generate
- Depict system behavior in terms of what the system does.

- The UML does not define something called ‘System Sequence Diagram’.

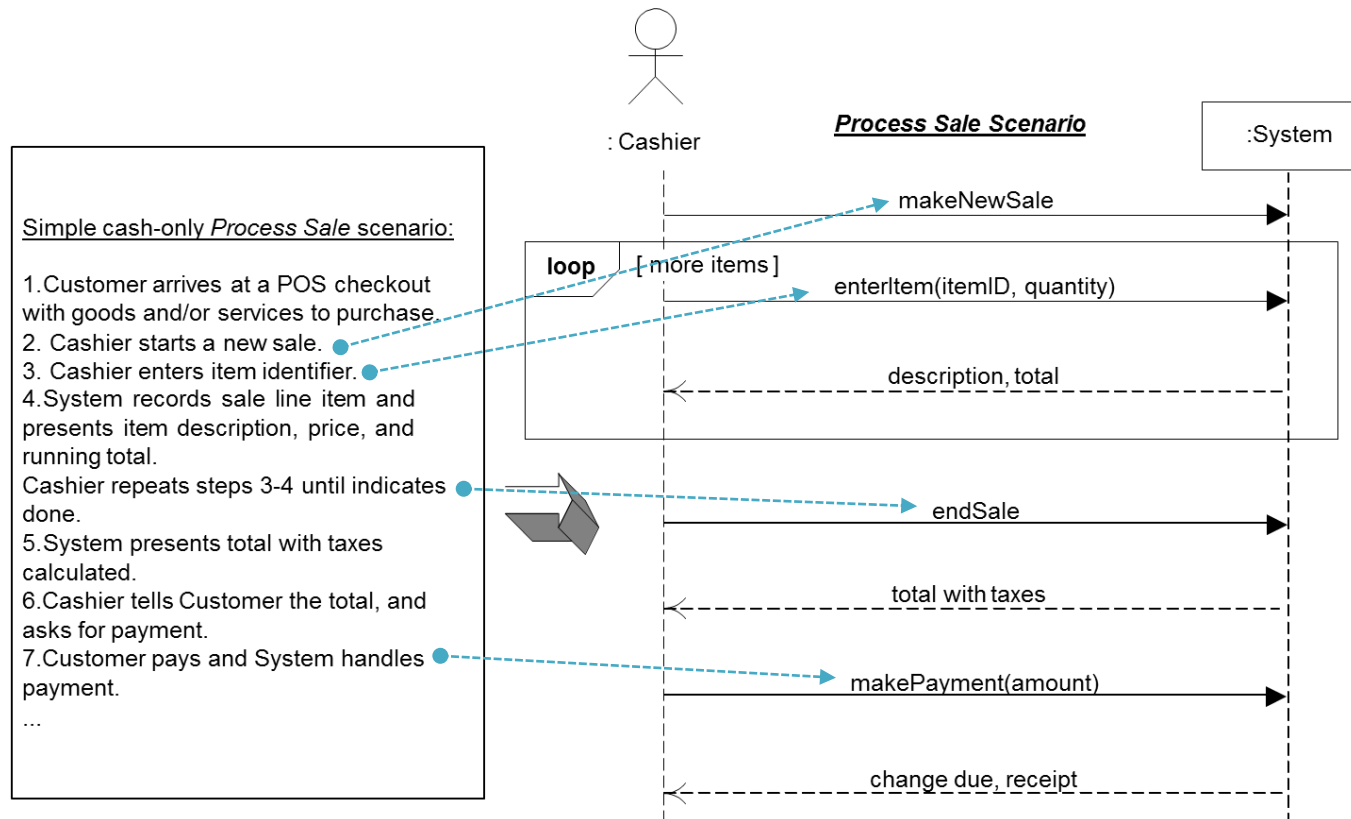
- We use the general UML sequence diagram notation.
- The term ‘system’ in SSDs is used to emphasize that systems are viewed as black boxes.
- An SSD shows system events for each use case.

- **SSD** describes how external actors interact with the software system.

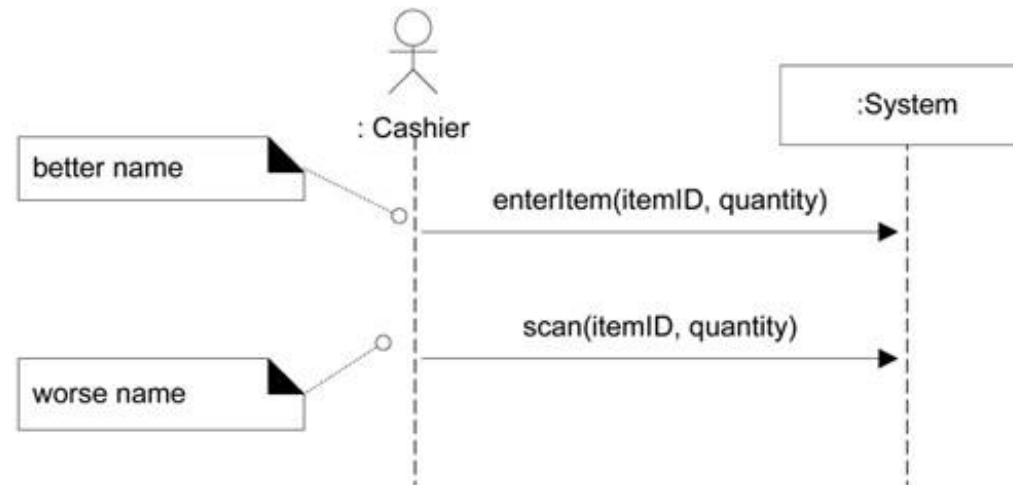
- During this interaction, an actor generates system events to a system, usually requesting some system operations to handle the event.
- System operations are used as input to object design.

- **System operations**

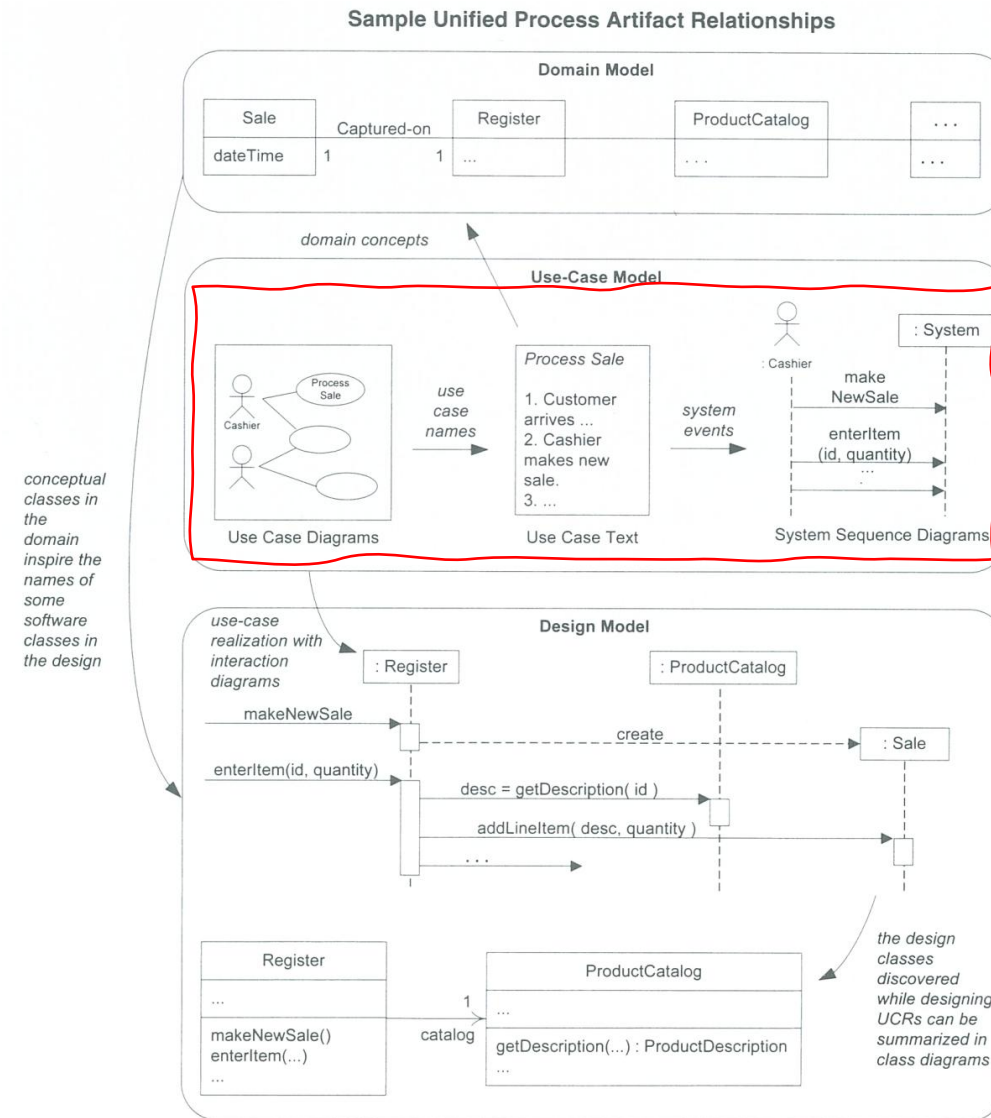
- Operations that the system as a black box component offers in its public interface
- Show **system events**, which the system should have system operations to handle the system events.
- **System Interfaces**: the entire set of system operations across all use cases



- System events should be expressed at the abstract level of intention rather than in terms of the physical input device.
- Example : *scan(itemID)* vs. *enterItem(itemID)*
 - The *enterItem* name is better, since it communicates intention rather than the input device.



The Relationship to Other Artifacts



from "Applying UML and Patterns (3rd)"

A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a white circle with a blue outline further up, a dark blue circle with a white outline above that, and a white circle with a blue outline at the top. There are also some diagonal lines and a grid of small white dots.

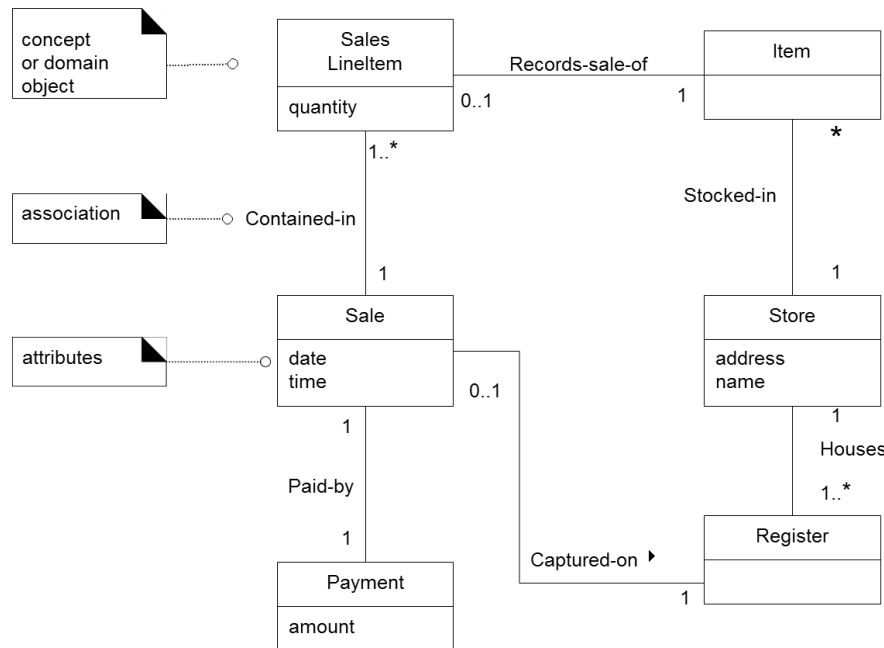
OOA

- Domain Model

- **Domain model**
 - A **visual representation** of conceptual classes or real-situation objects in a **domain**.
 - The most important classic model in object-oriented analysis
 - Can act as a source of inspiration for designing software objects and classes.
 - **Visual dictionary** of the noteworthy abstractions, domain vocabulary, and information contents of the domain
 - Not represents software objects
 - Not an UML diagram
 - A kind of a preliminary version of class diagram in design phase
- Domain model is often illustrated with **class diagrams** with **no operations**.
 - **domain objects** (or conceptual classes)
 - **associations** between conceptual classes (with labels and multiplicities)
 - **attributes** of conceptual classes

Why Create a Domain Model?

- Two reasons to create a domain model:
 - Getting to know the domain** during early elaboration iterations
 - Understanding the internal/external concepts involved and their relationships
 - Inspiring the software classes** of the **domain layer** in the design model
 - This prevents software from being far away from the reality of the domain.
 - Lower representation gap** : Use software class names in the domain layer inspired from names in the domain model, with objects having domain-familiar information and responsibilities.

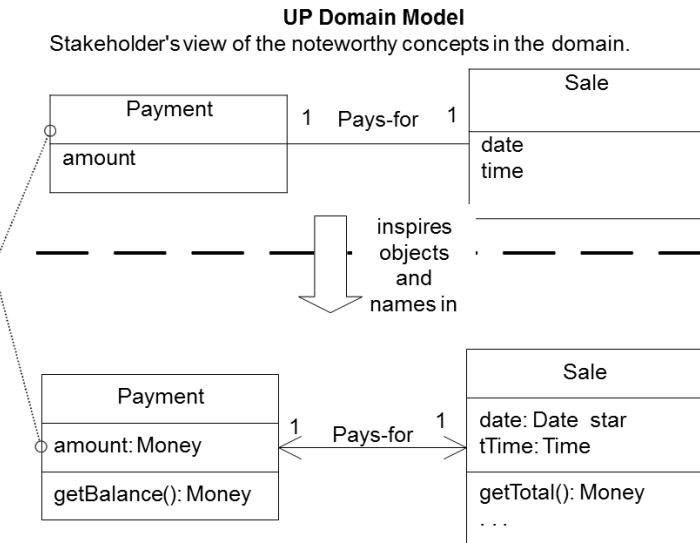


Partial Domain Model for NextGen POS

A Payment in the Domain Model is a concept, but a Payment in the Design Model is a software class. They are not the same thing, but the former *inspired* the naming and definition of the latter.

This reduces the representational gap.

This is one of the big ideas in object technology.



The object-oriented developer has taken inspiration from the real world domain in creating software classes.

Therefore, the representational gap between how stakeholders conceive the domain, and its representation in software, has been lowered.

- Same as the way of creating class diagrams.
 1. Find **conceptual classes** and draw them in a [UML class diagram](#)
 2. Add **associations** and **attributes** to conceptual classes
- Identification of **Noun Phrases**
 - Identify [the nouns and noun phrases](#) in a textual description of the domain and then
 - Consider them as candidate conceptual classes and attributes.

1. **Customer** arrives at a **POS checkout** with **goods** and/or **services** to purchase.

2. **Cashier** starts a new **sale**.

3. **Cashier** enters **item identifier**.

4. System records **sale line item** and presents **item description, price, and running total**.
Price calculated from a set of price rules.

Cashier repeats steps 2-3 until indicates done.

5. System presents total with **taxes** calculated.

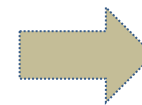
6. Cashier tells Customer the total, and asks for **payment**.

7. Customer pays and System handles payment.

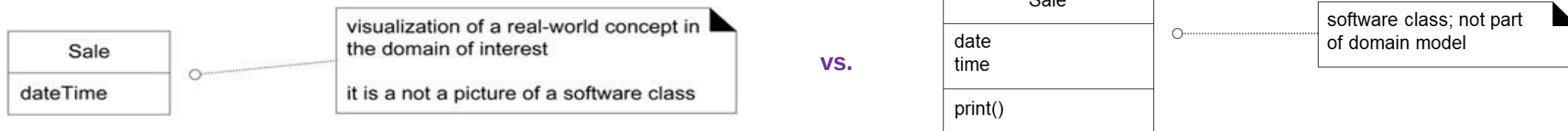
8. System logs the completed **sale** and sends sale and payment information to the external **Accounting** (for accounting and **commissions**) and **Inventory** systems (to update inventory).

9. System presents **receipt**.

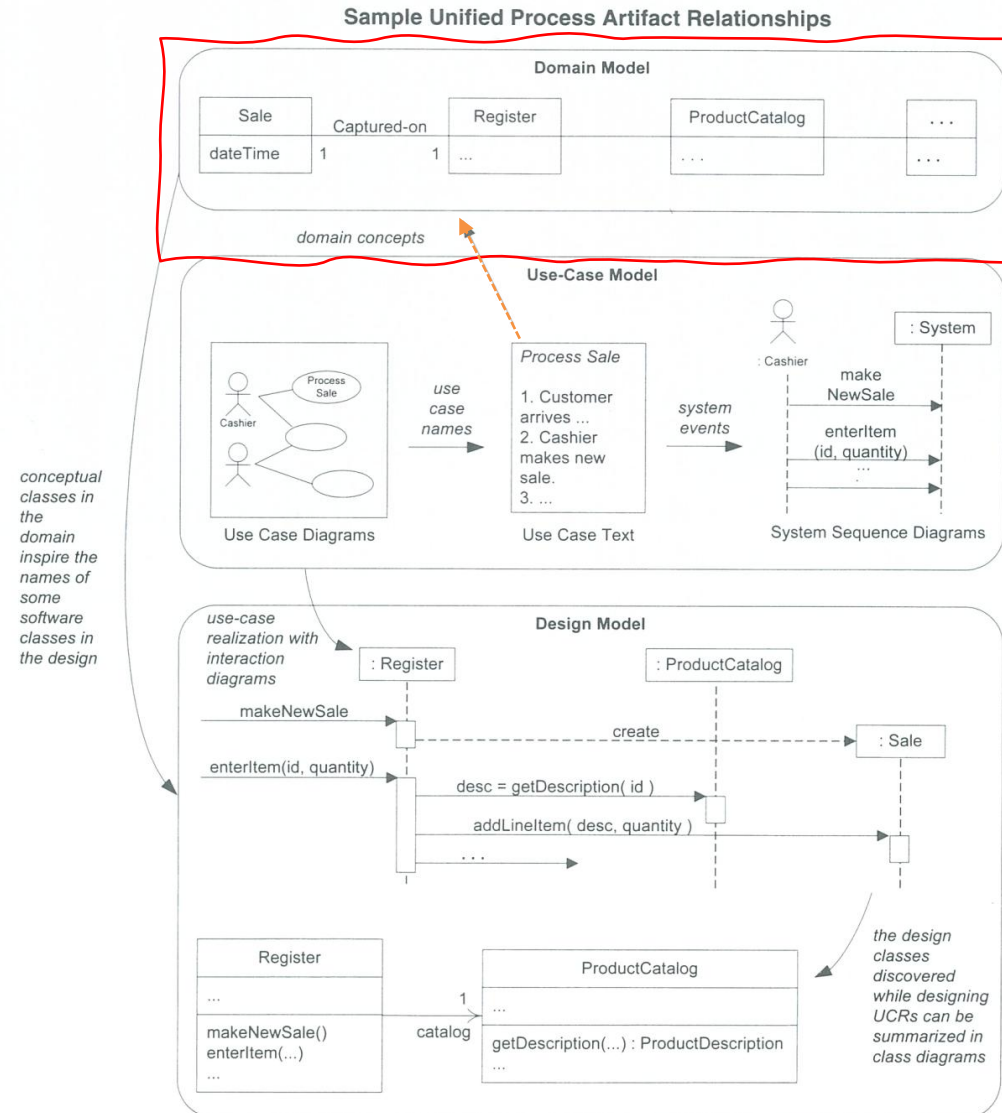
10. Customer leaves with receipt and goods (if any).



- A domain model is not of software objects such as:
 - Software classes (*i.e.*, C++ or Java classes)
 - Elements representing artifacts related to the implementation of the system (*e.g.*, a database or a window)
 - Methods (operations)



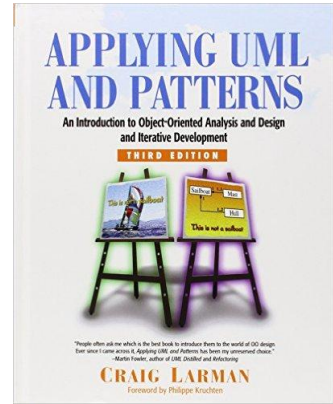
- **Is the Domain Model Correct?**
 - There is **no** such thing as a single correct domain model.
 - All models are approximations of the domain we are attempting to understand.
 - The domain model is a primary tool of **understanding** and **communication** among a particular group.
 - Correctness << Usefulness



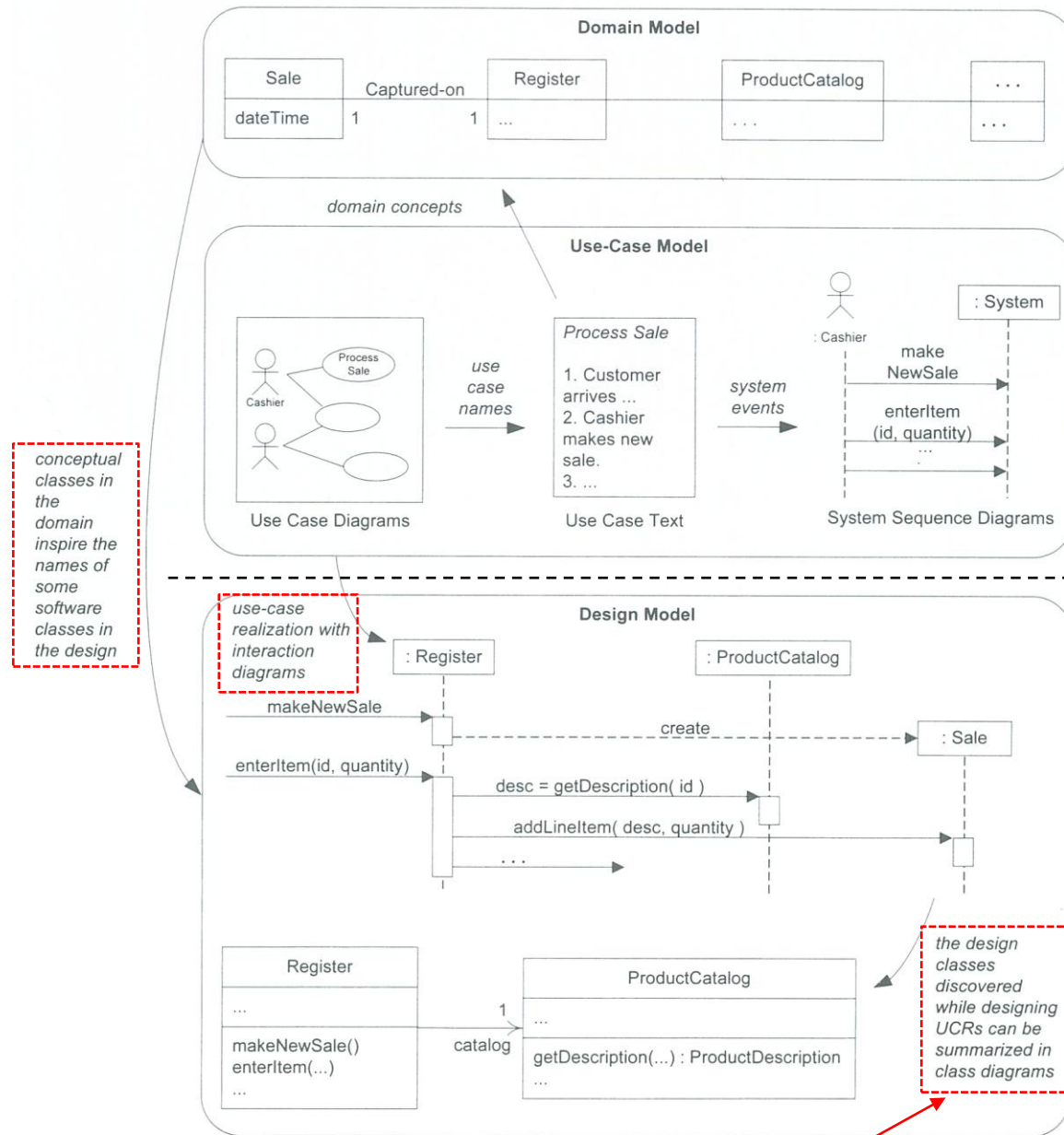
- Artifacts developed in OOA
 - Use-Case Model (Use-Case + SSD)
 - Domain Model

Sample Unified Process Artifacts and Timing (s-start; r-refine)

Discipline	Artifact Iteration→	Incep.	Elab.	Const.	Trans.
		I1	E1..En	C1..Cn	T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	
Implementation	Implementation Model (code, html, ...)		s	r	r



Sample Unified Process Artifact Relationships



Domain Model

Use-Case Model
(+System Sequence Diagram)

OOA

OOD

Sequence Diagrams

Class Diagram

The dynamic model-driven approach!!!

OOI

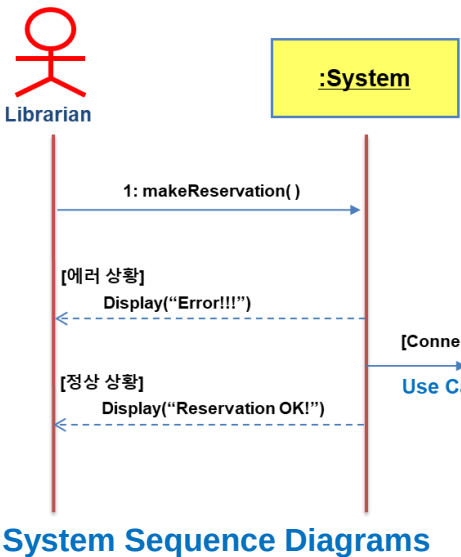
• OOA (Object-Oriented Analysis)

USE CASE: 1. Make Reservation

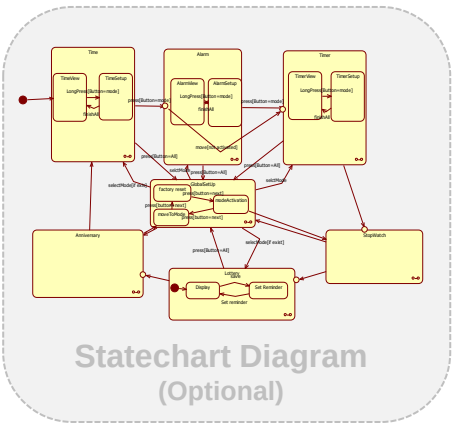
- 1. (A) A librarian requests the reservation of title.
- 2. (S) Check if corresponding title exist.
- 3. (S) Check if corresponding borrower exist.
- 4. (S) If the borrower does not exist, invoke "Add Borrower".
(→ connect to other Use Case)
- 5. (S) Create reservation information.

Use Cases
(Casual)

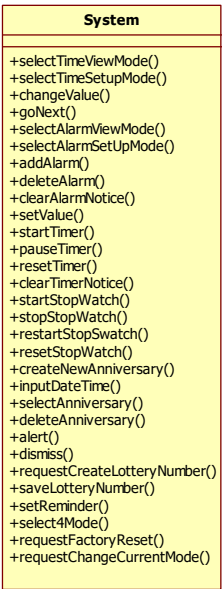
for each use case



System Sequence Diagrams



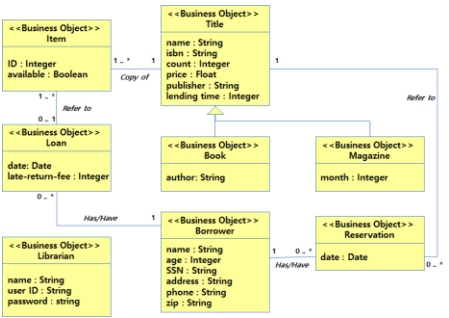
Statechart Diagram
(Optional)



System Operations
(System Interface)

System Function	Source Use Case	Destination in sequence diagram
Make reservation	→ Make Reservation	→ makeReservation()
Remove reservation	→ Remove Reservation	→ removeReservation()
Land item	→ Land Item	→ landItem()
Return item	→ Return Item	→ returnItem()
Calculate Late-Return Fee	→ Calculate Late-Return Fee	→ getReservationFee()
Calculate Reservation Fee	→ Calculate Reservation Fee	→ calculateReservationFee()
Apply Availability	→ Apply Availability	→ applyAvailability()
Apply Item	→ Apply Item	→ applyItem()
Update title	→ Update Title	→ updateTitle()
Update item	→ Update Item	→ updateItem()
Remove item	→ Remove Item	→ removeItem()
Update item	→ Update Item	→ updateItem()
Add borrower	→ Add Borrower	→ addBorrower()
Remove borrower	→ Remove Borrower	→ removeBorrower()
Update borrower	→ Update Borrower	→ updateBorrower()
Log in	→ Log In	→ login()
Log out	→ Log Out	→ logout()
Complete state of items checked out	→ Complete State	→ completeState()

Traceability Table



Domain Model

A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a white circle with a blue outline further up, a dark blue circle with a white outline above that, and a white circle with a blue outline at the top. There are also some diagonal lines and a grid of small white dots in the lower right of the sidebar.

OOD

- Sequence Diagram

An OOAD Example - Dice Game



Define use cases

Define domain model

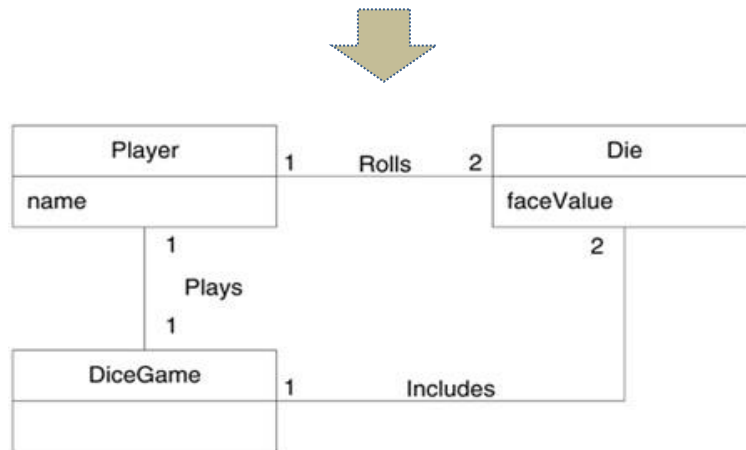
Define interaction diagrams

Define design class diagrams

OOA

Use Case : **Play a Dice Game**

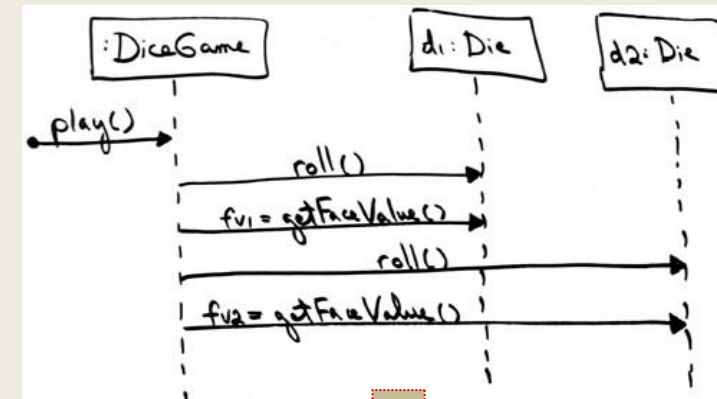
- Player requests to roll the dice.
- System presents results.
- If the dice's face value totals seven, player wins; otherwise, player loses.



Domain Model

OOD

Interaction Diagram (Sequence Diagram)

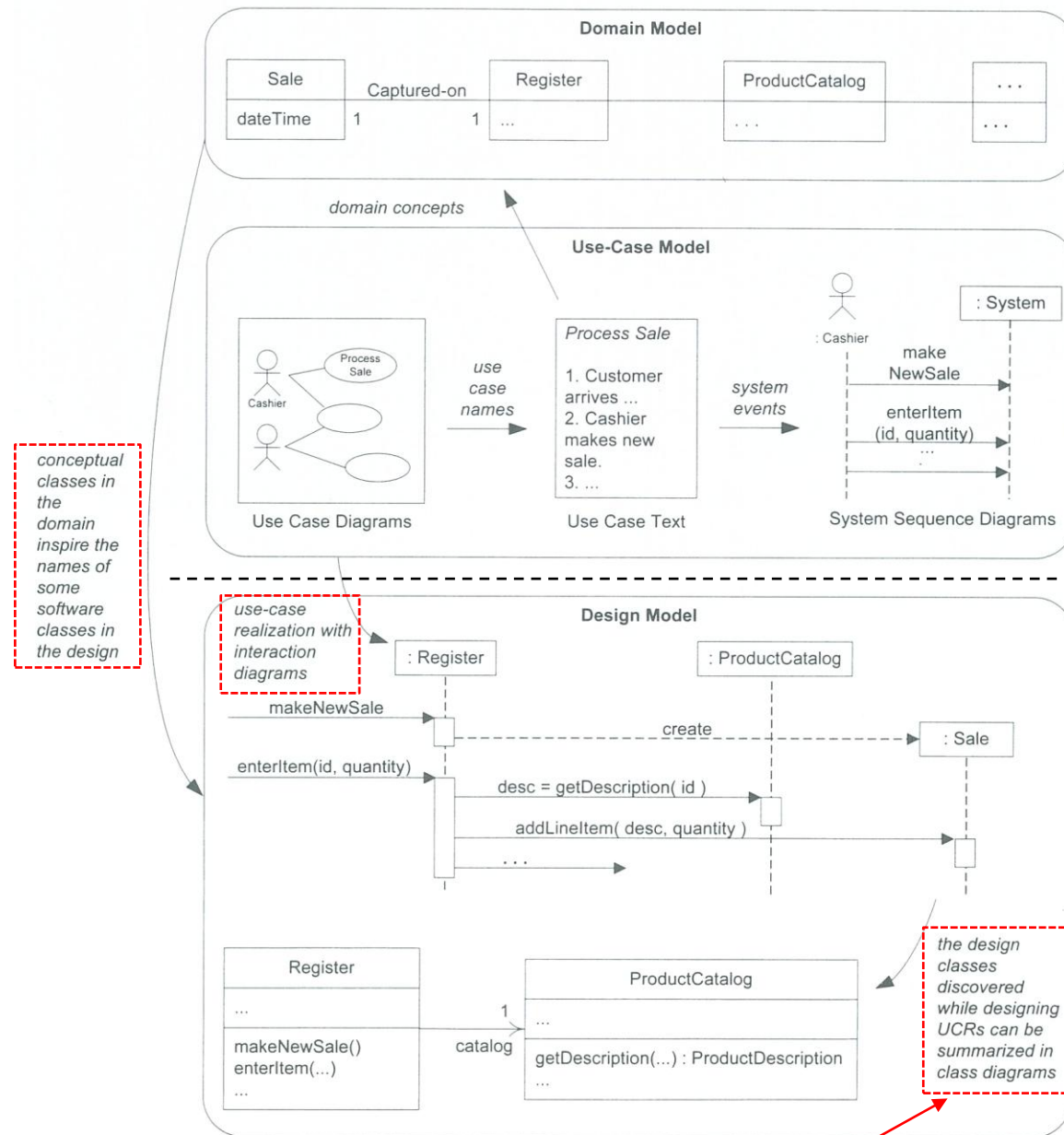


Dynamic model-driven !!!



Design Class Diagram

Sample Unified Process Artifact Relationships



Domain Model

Use-Case Model
(+System Sequence Diagram)

OOA

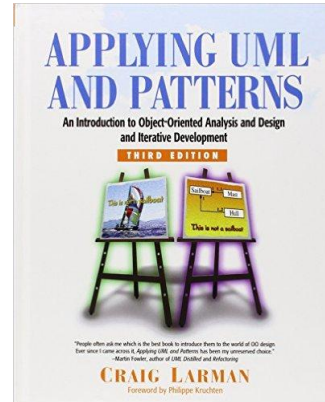
OOD

Sequence Diagrams

Class Diagram

The dynamic model-driven approach!!!

OOI

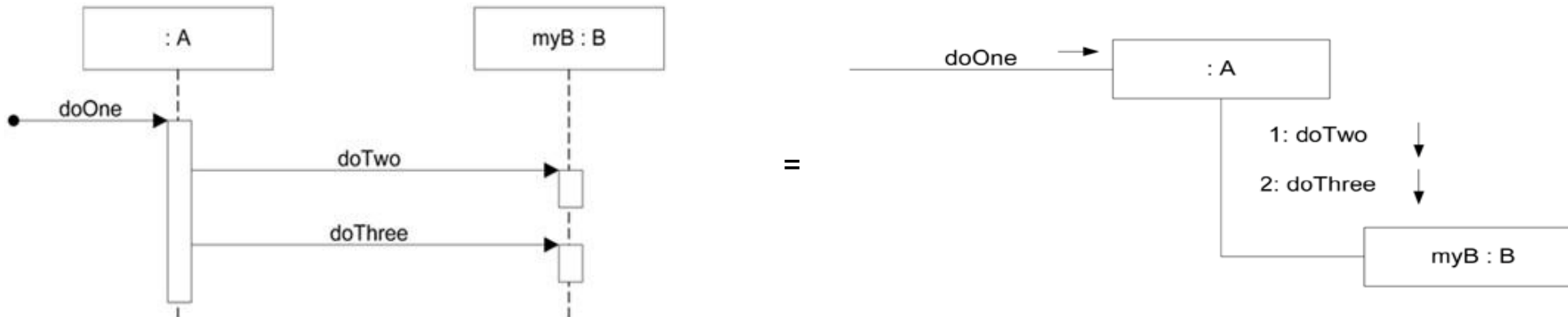


- **Sequence diagrams**
 - Modeling the collaboration of objects based on a time sequence
- **Communication diagrams**
 - Focusing on showing the collaboration of objects rather than the time sequence

```
public class A
{
    private B myB = new B();

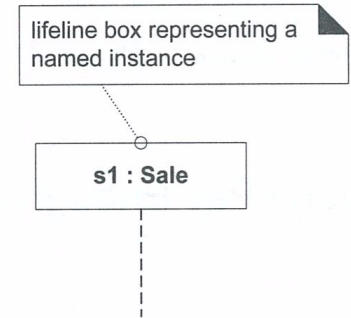
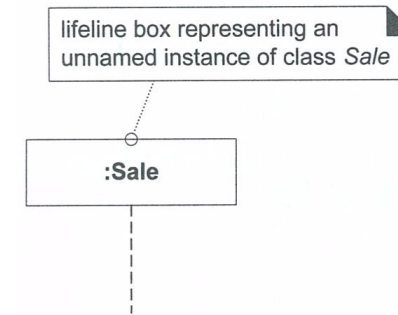
    public void doOne()
    {
        myB.doTwo();
        myB.doThree();
    }

    // ...
}
```



- **Lifeline boxes**

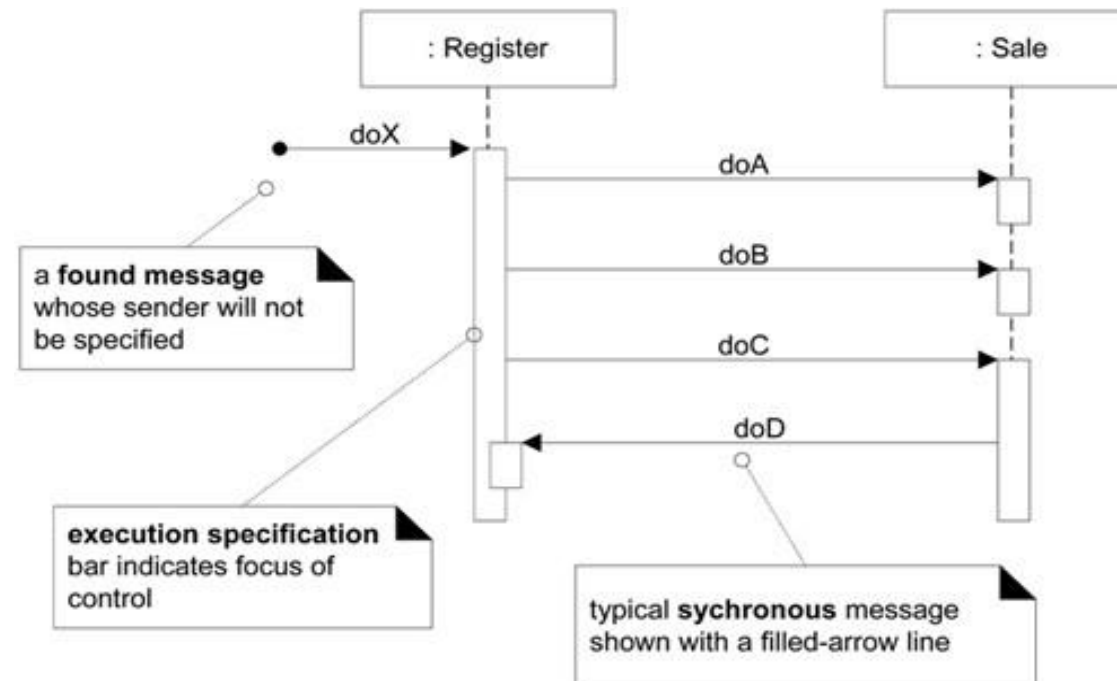
- Represent the participants in the interaction, informally and practically
 - object(s), class, subsystem, component, etc.
- Usually called *roles*



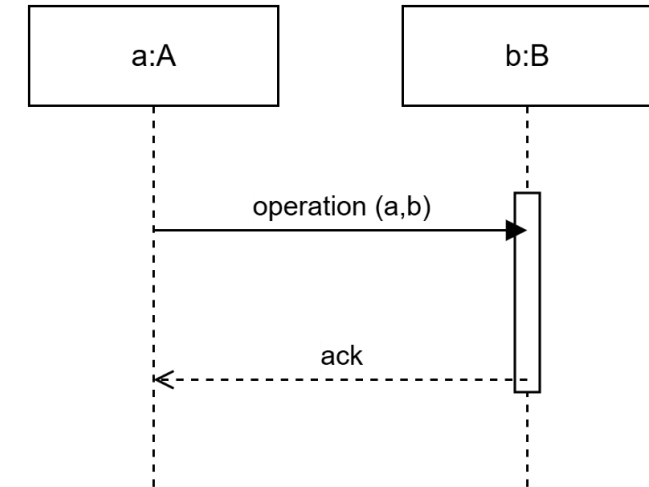
- **Lifelines**

- **Messages**

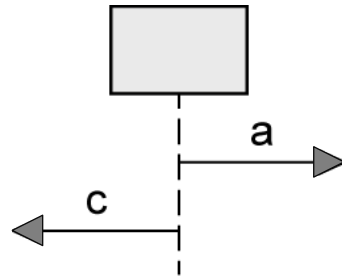
- **Execution specification**



- **Synchronous message**
 - Sender waits until it has received a response message before continuing.
 - An execution specification is inserted at target.
- **Response message**
 - Corresponding to a synchronous message like acknowledgement
 - May be omitted if content and location are obvious
- **Asynchronous message**
 - Sender continues without waiting for a response message.

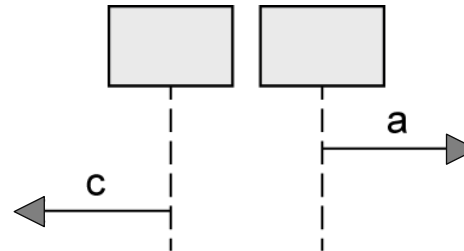


on one lifeline



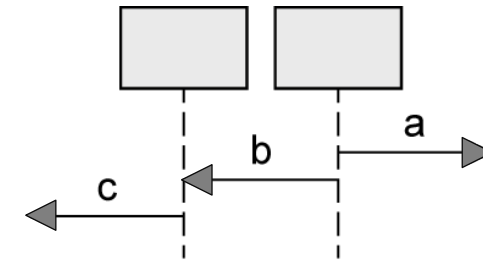
Traces:
T01: $a \rightarrow c$

on different lifelines



Traces:
T01: $a \rightarrow c$
T02: $c \rightarrow a$

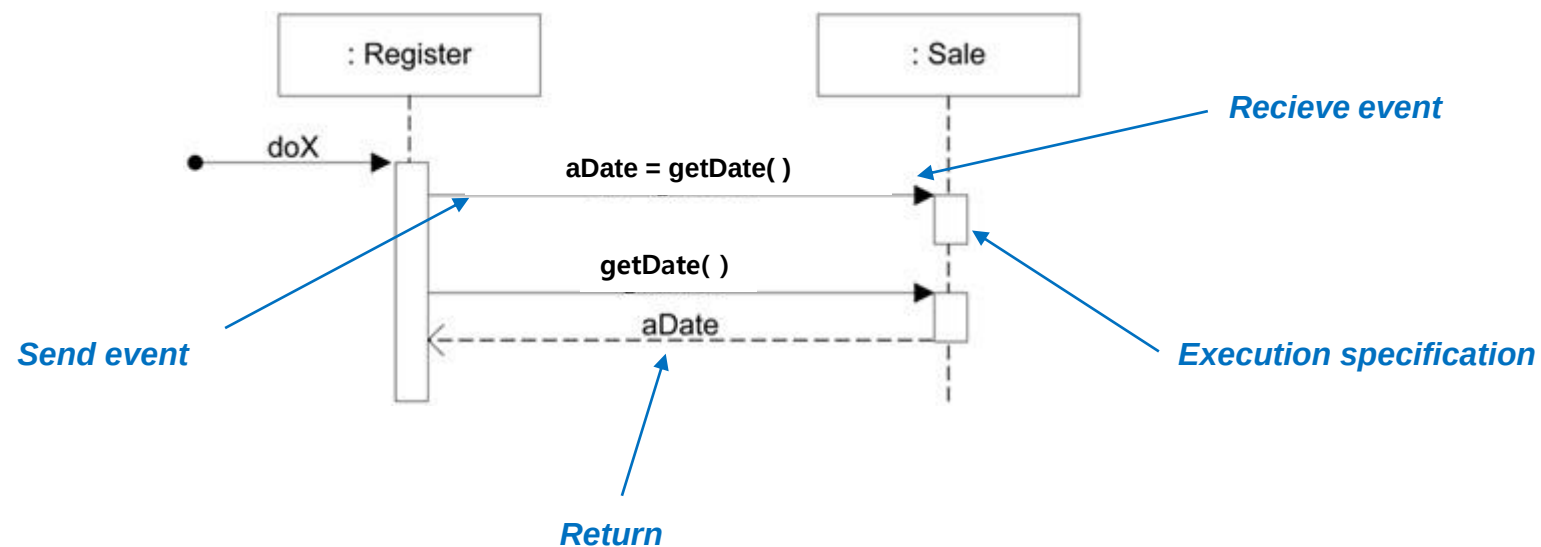
on different lifelines
which exchange messages



Traces:
T01: $a \rightarrow b \rightarrow c$

return = message (parameter: parameterType) : returnType

- For example:
 initialize(code)
 initialize
 d = getProductDescription(id)
 d = getProductDescription(id:ItemID)
 d = getProductDescription(id:ItemID) : ProductDescription



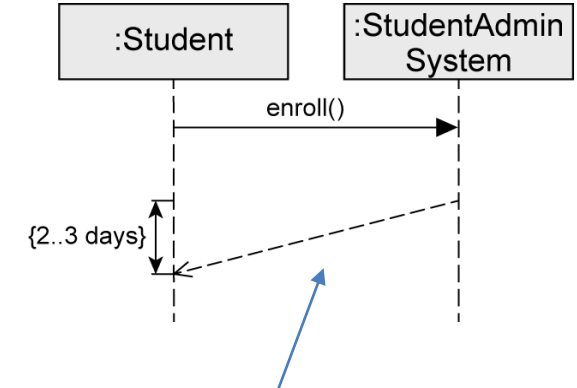
- **Singleton object**

- There is only one instance of a class instantiated.
- Implying to **the Singleton design pattern**

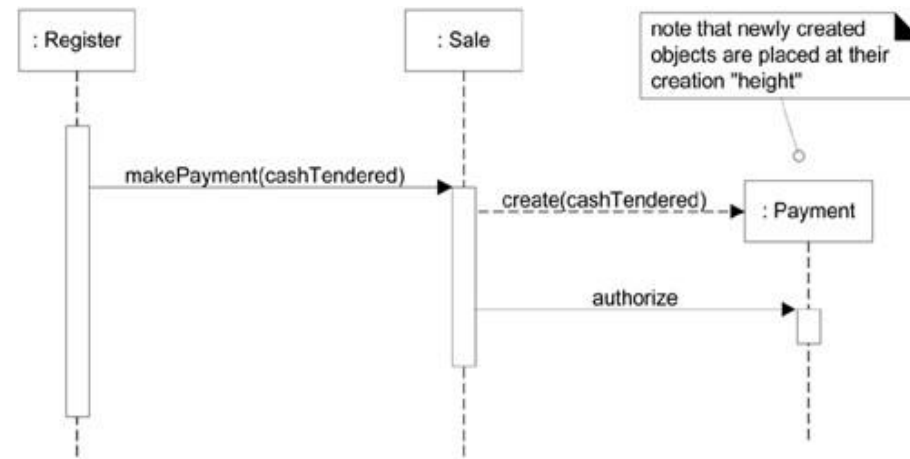


- **Time-consuming message**

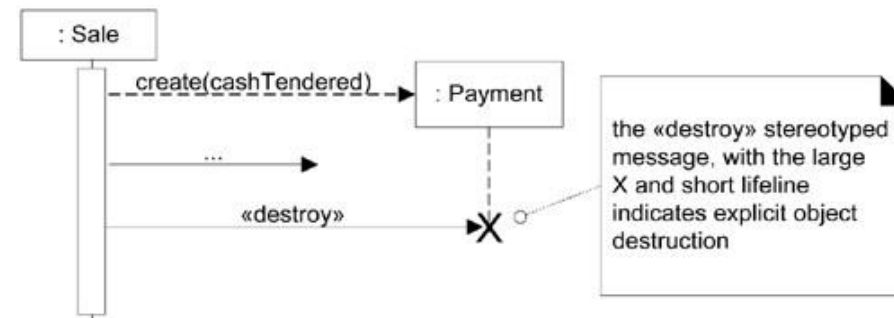
- Message with duration
- Express that time elapses between the sending and the receipt of a message
- Usually, messages are assumed to be transmitted without any loss of time.



- To **create** an instance of a class
 - The UML mandates dashed line.
 - The message name *create* is not required.
 - Anything is legal, but it's a UML idiom.



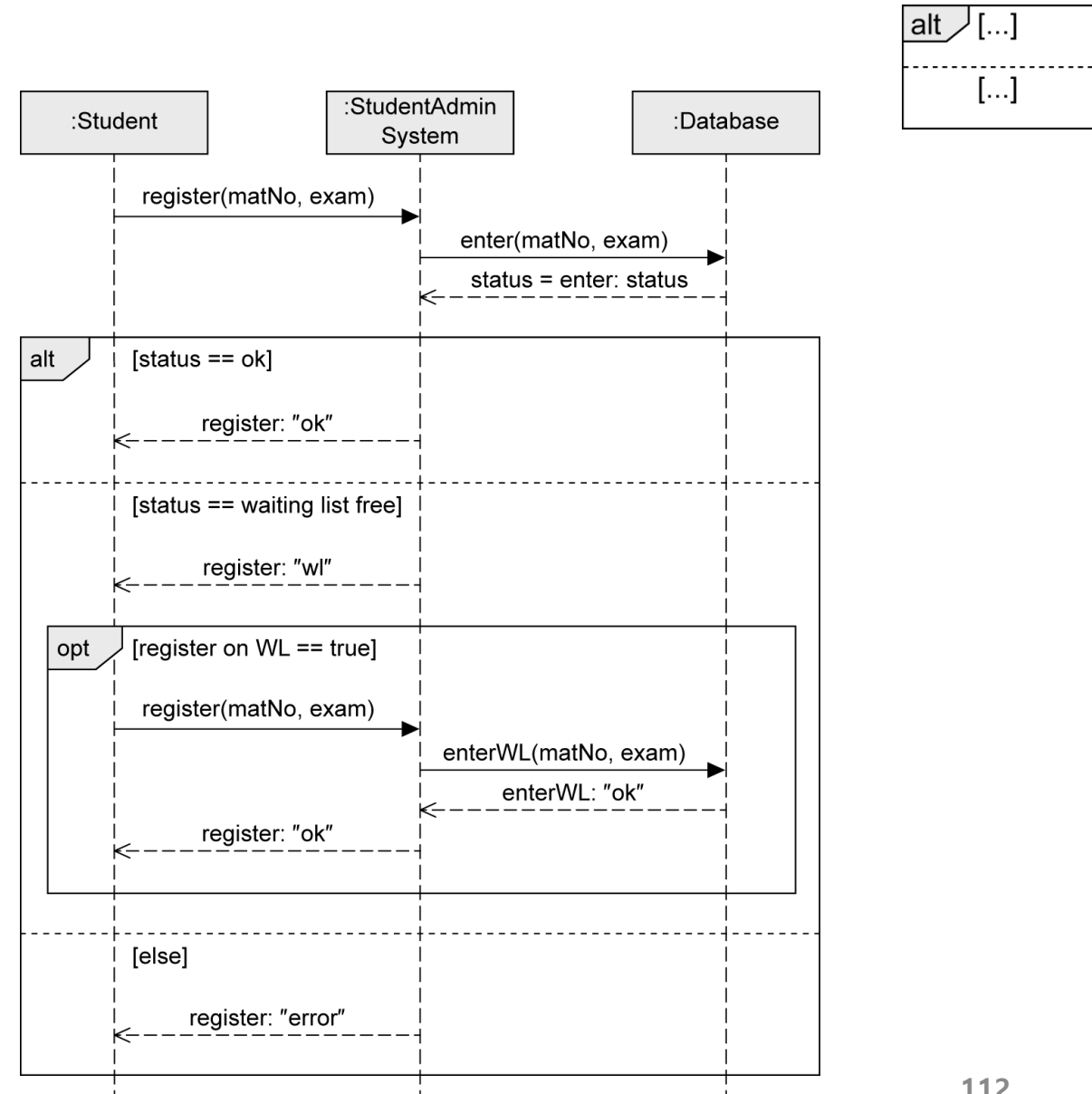
- To show explicit **destruction** of an object
 - The `<<destroy>>` stereotyped message, with the large X and short lifeline indicates explicit object destruction



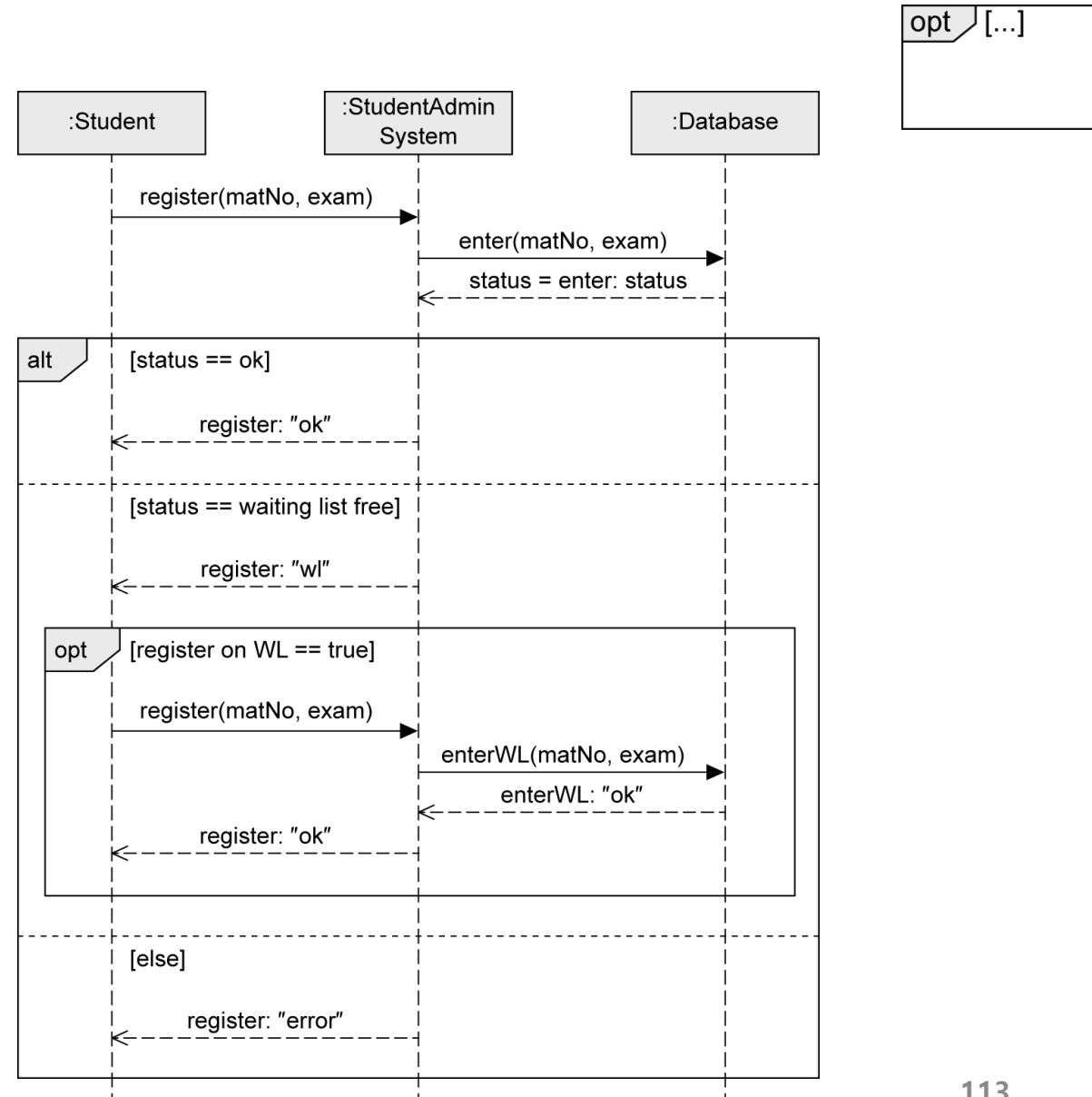
- 12 predefined types of **operators**
 - Model various control structures with frames
 - **Frames** : regions or fragments of the diagrams, which has an operator and a guard
 - Frames are nested freely.

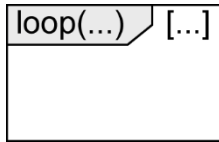
	Operator	Purpose
Branches and loops	alt	Alternative interaction
	opt	Optional interaction
	loop	Repeated interaction
	break	Exception interaction
Concurrency and order	seq	Weak order
	strict	Strict order
	par	Concurrent interaction
	critical	Atomic interaction
Filters and assertions	ignore	Irrelevant interaction
	consider	Relevant interaction
	assert	Asserted interaction
	neg	Invalid interaction

- To model alternative sequences
- Similar to **switch statement** in Java / C++
 - Guards are used to select the one path to be executed.
 - Multiple operands
- Guards
 - Modeled in square brackets
 - default: true
 - predefined: [else]
- Guards should be disjoint to avoid non-deterministic behavior.



- To model an optional sequence
- Similar to **if statement without else branch**
 - Exactly one operand
 - Actual execution at runtime is dependent on the guard.





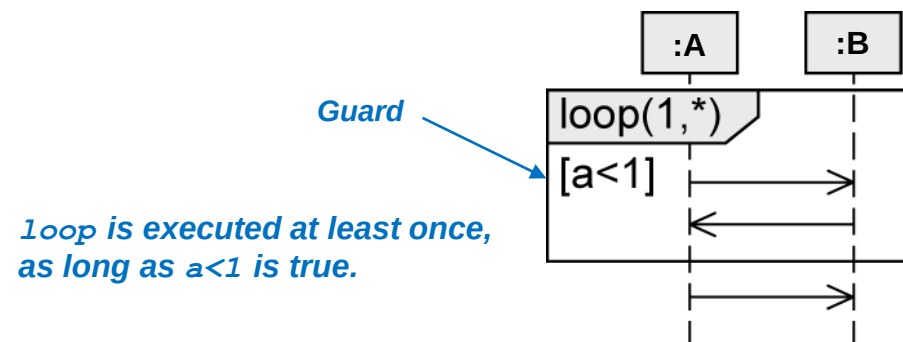
- To model repeatedly-executed sequences
 - Exactly one operand
- Similar to **for/while loop statements** in Java / C++
 - Keyword **loop** followed by the minimal/maximal number of iterations
 - **(min..max)** or **(min,max)**
 - default: **(*)** .. no upper limit
- Guard
 - Evaluated as soon as the minimum number of iterations has taken place
 - Checked for each iteration within the **(min,max)** limits
 - If the guard evaluates to false, the execution of the loop is terminated.

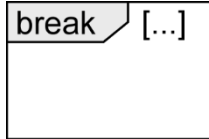
Notation alternatives:

loop(3,8) = loop(3..8)

loop(8,8) = loop(8)

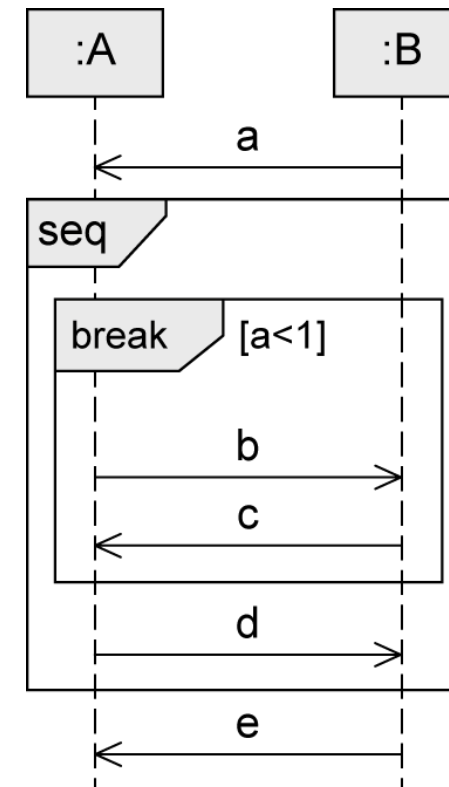
loop = loop() = loop(0,*)*



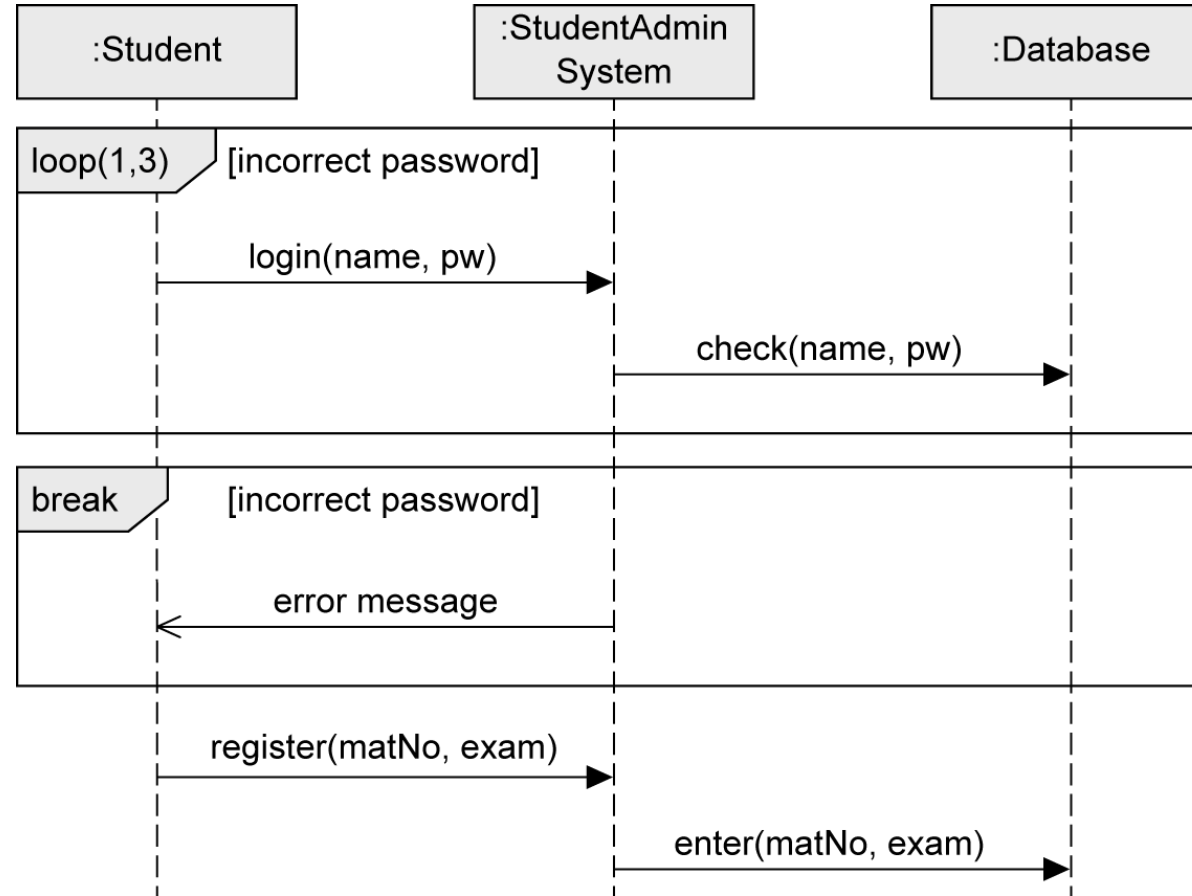


- Similar to exception handling
 - Exactly one operand with a guard
- If the guard is true,
 - Interactions within this operand are executed.
 - Remaining operations of the surrounding fragment are omitted.
 - Interaction continues in the next higher level fragment.

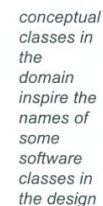
Not executed if break is executed



loop and break Fragment - Example



Domain Model

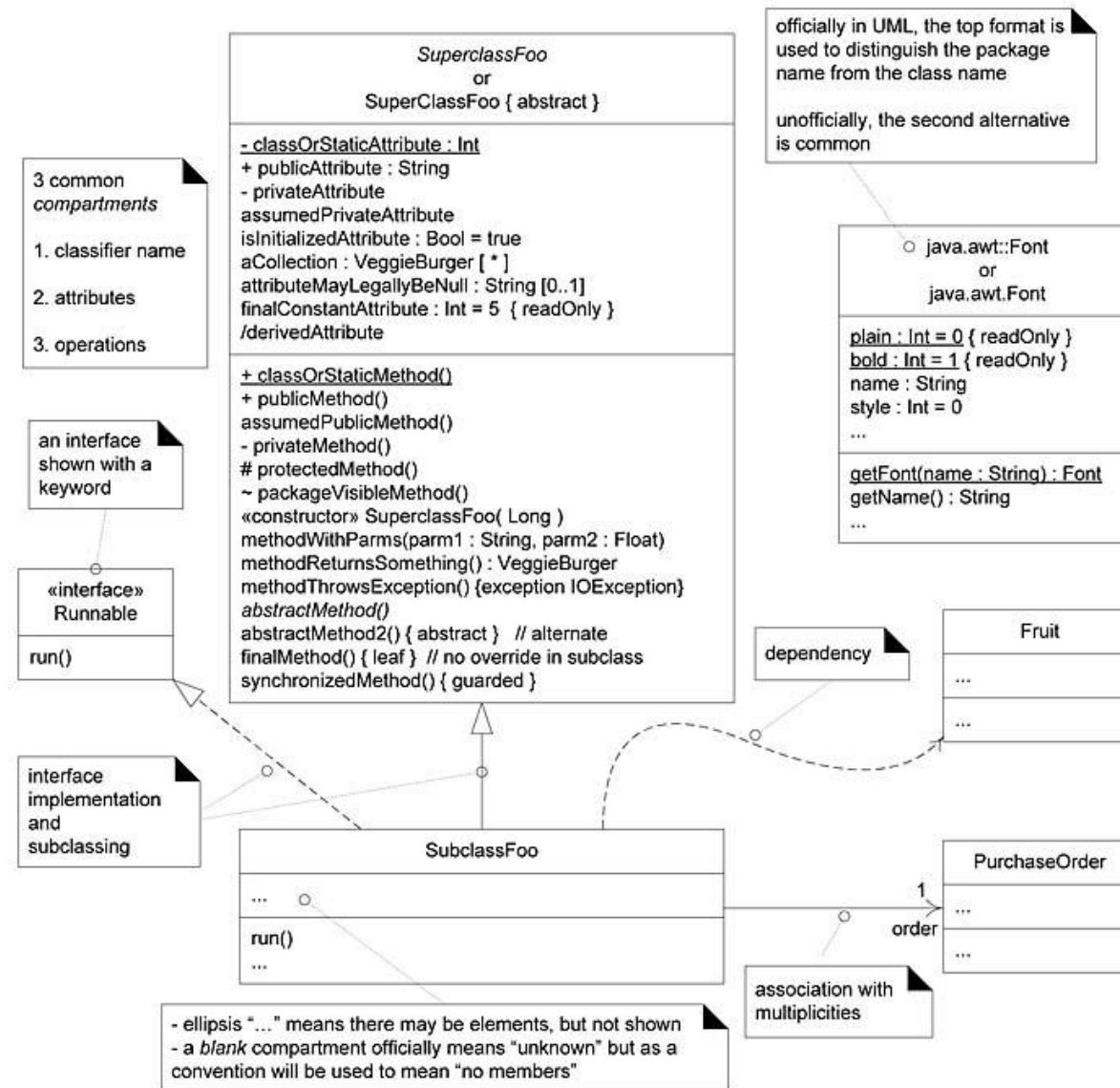


117

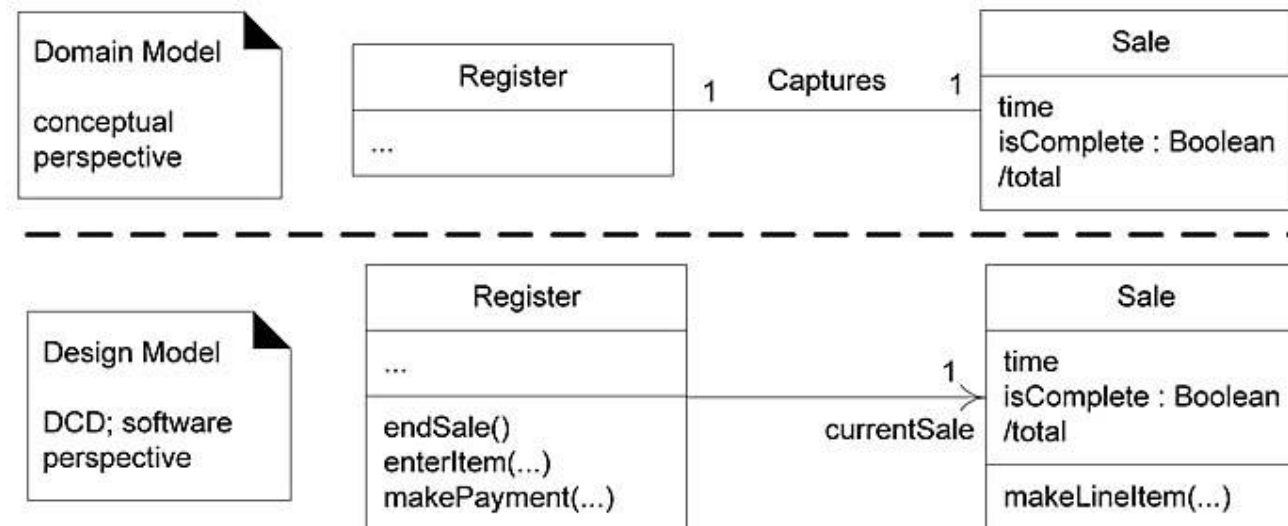
A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a white circle with a blue outline further up, a white circle with a blue outline at the top, and a white circle with a blue outline at the top right. There are also some white dots arranged in a grid pattern near the bottom right of the sidebar.

OOD

- Class Diagram

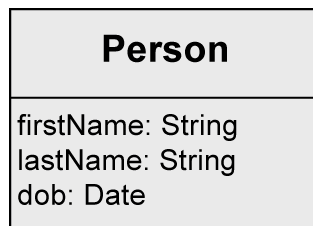
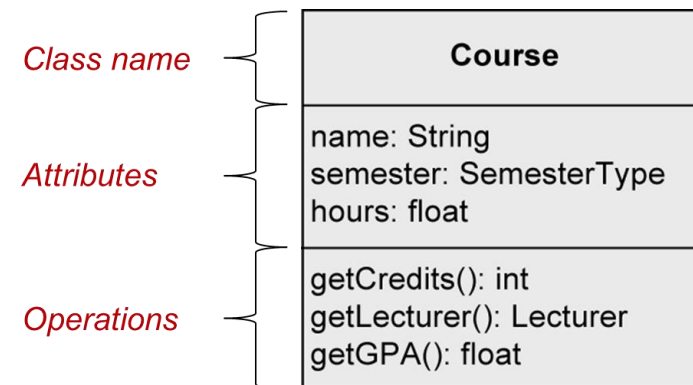


- The UML class diagrams are used in multiple perspectives.
 - In a conceptual perspective, at OOA : **Domain model**
 - In a design perspective, at OOD : **Design Class Diagram (DCD)**

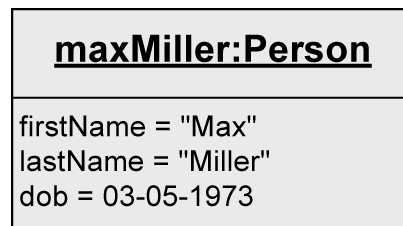


- A **class** is a construction plan for a set of similar objects of a system.
 - Objects are instances of classes.

- **Attributes:** structural characteristics of a class
 - Different value for each instance (object)
- **Operations:** behavior of a class
 - Identical for all objects of a class → Not depicted in object diagram



Class



An Object of the Person class

- **Visibility : Who is permitted to access the attribute.**

- + ... public: everybody
- - ... private: only the object itself
- # ... protected: class itself and subclasses
- ~ ... package: classes that are in the same package

- **Derived Attribute**

- Value is derived from other attributes or associations.
- /age: calculated from the date of birth

- **Multiplicity**

- Number of values which an attribute may contain
- Notation: [min..max]
- Default value: 1 , No upper limit: [*] or [0..*]

- **Default value**

- Used if the attribute value is not set explicitly by the user

- **Types of attributes**

- User-defined classes
- Data types
 - Primitive data type
 - Pre-defined: Boolean, Integer, Unlimited Natural, String
 - User-defined: «primitive»
 - Composite data type: «datatype»
 - Enumerations: «enumeration»

«primitive» Float
round(): void

«datatype» Date
day month year

«enumeration» AcademicDegree
bachelor master phd

Person
+ firstName: String + lastName: String - dob: Date # address: String[1..*] {unique, ordered} - ssNo: String {readOnly} - /age: int - ppassword: String = "pw123" - <u>personsNumber</u> : int

- **Pre-defined properties**

- {readOnly} ... value cannot be changed
- {unique} ... no duplicates permitted
- {non-unique} ... duplicates permitted
- {ordered} ... fixed order of the values
- {unordered} ... no fixed order of the values

- **Attribute specification**

- Set: {unordered, unique}
- Multi-set (Bag): {unordered, non-unique}
- Ordered set: {ordered, unique}
- List: {ordered, non-unique}

Person
+ firstName: String + lastName: String - dob: Date # address: String[1..*] {unique, ordered} - ssNo: String {readOnly} - /age: int - ppassword: String = "pw123" - <u>personsNumber: int</u>
+ getName(out fn:String, out gn: String): void + updateLastName(newName: String): boolean + <u>getPersondNumber(): int</u>

- **Notation similar to attributes**

- *visibility name (parameter-list) {property-string}*
- *visibility name (parameter-list) : return-type {property-string}* ← UML1.X
- Guidelines
 - Assume that the new version includes a return type.
 - Operations are usually assumed public if no visibility is shown.

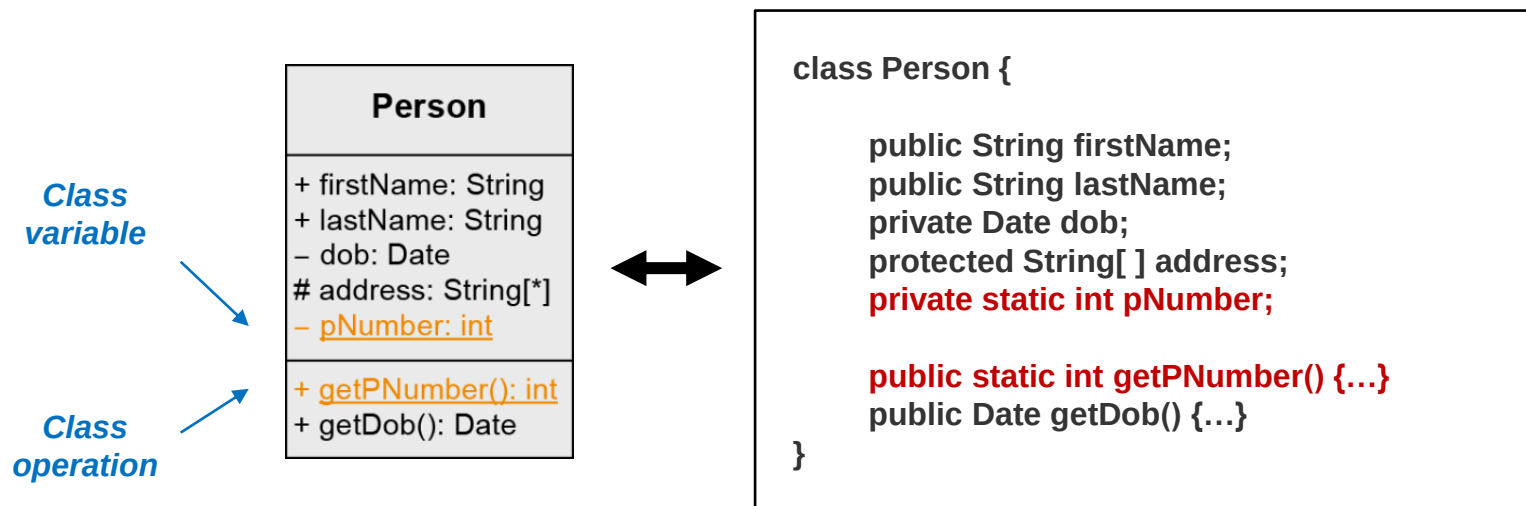
- **Direction of the parameter**

- in ... input parameter
 - When the operation is used, a value is expected from this parameter
- out ... output parameter
 - After the execution of the operation, the parameter has adopted a new value
- inout : combined input/output parameter

- **An operation is not a method.**

- **A UML operation is a declaration**, with a name, parameters, return type, exceptions list, and possibly a set of constraints of pre-and post-conditions.
- Not an implementation - rather, **methods are implementations**.

- **Instance variable** (= instance attribute) : attributes defined on instance level
- **Class variable** (= class attribute, static attribute)
 - Defined only once per class, *i.e.*, shared by all instances of the class.
 - Example: counters for the number of instances of a class
- **Class operation** (= static operation)
 - Can be used, if no instance of the corresponding class was created.
 - Example: constructors, counting operations, etc.



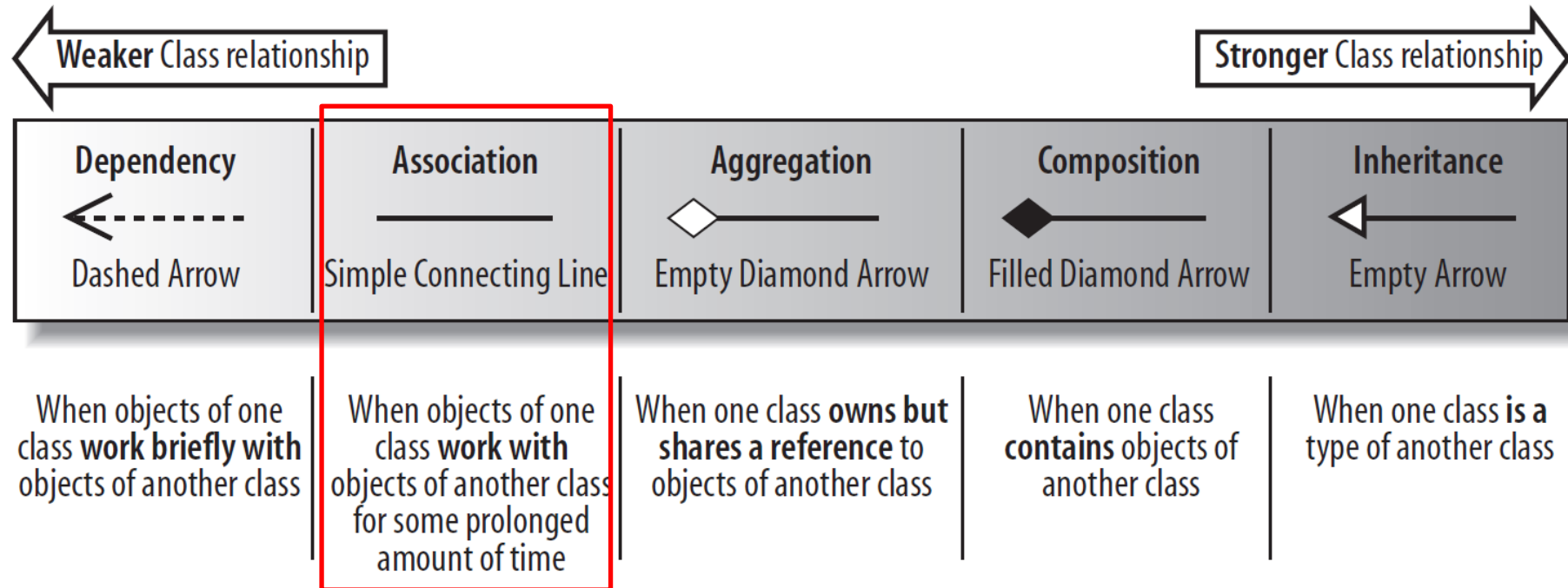
- **Accessing operations** to retrieve or set (private) attributes
 - Example: `getPNumber()` and `setPNumber()`
 - Often excluded (or filtered) from the class diagram, since they are too many.
 - For n attributes, there may be $2n$ uninteresting **getter** and **setter** operations.

Person
+ firstName: String + lastName: String - dob: Date # address: String[1..*] {unique, ordered} - ssNo: String {readOnly} - /age: int - ppassword: String = "pw123" - <u>personsNumber: int</u>
+ getName(out fn:String, out gn: String): void + updateLastName(newName: String): boolean + <u>getPersonNumber(): int</u>

- Getter/Setter are a big **obstacle** for securing “*Object Data Encapsulation*” and “*Information Hiding*”.
 - The fastest and easiest way to summon the “**Maintenance Nightmare**”

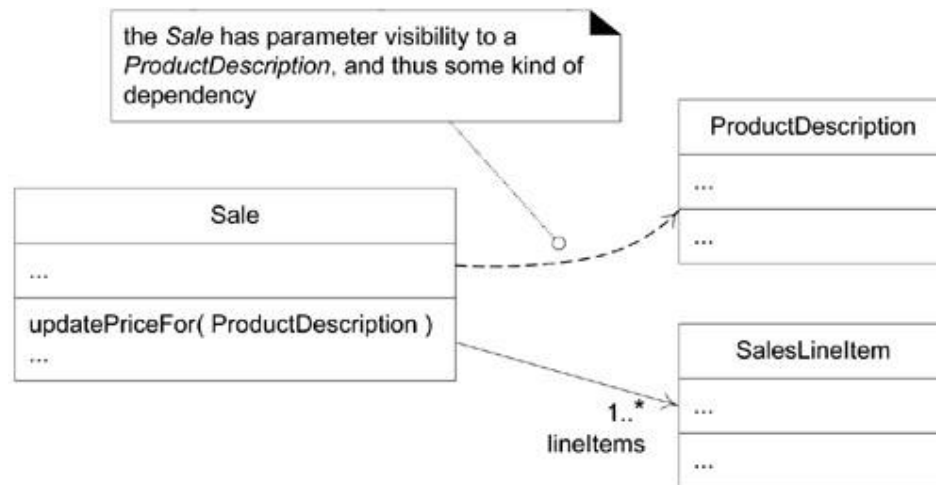


Types of Class Relationship



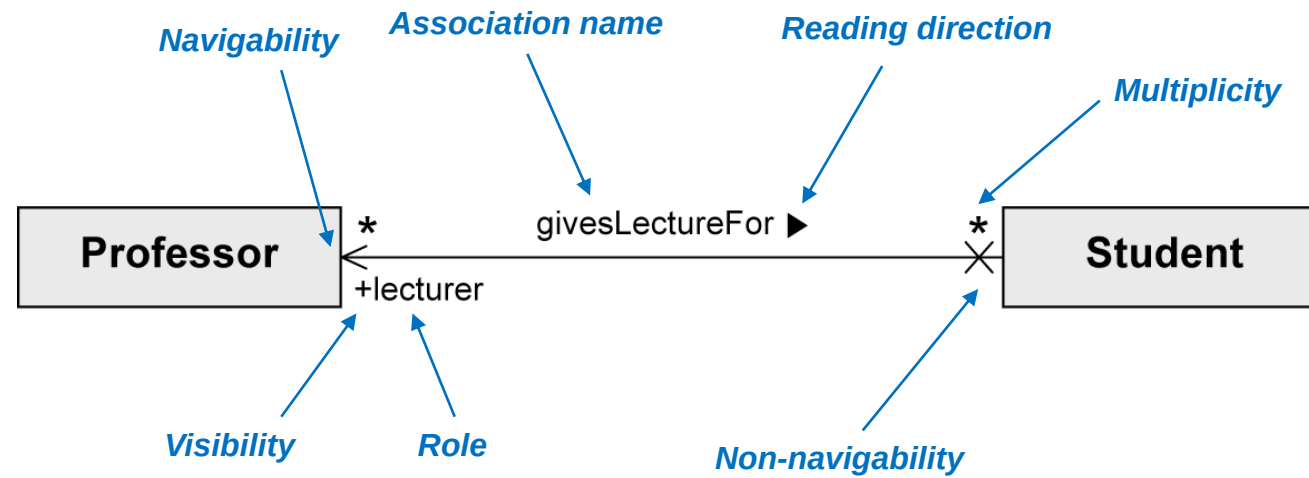
This is the default basic relationship !!!

- **Dependency** models weakest possible relationships between classes.
 - A class needs to know about another class to use objects of that class **briefly**.
 - Not often used in class diagram but does in component diagram.
- Example:
 - The `updatePriceFor` method receives a `ProductDescription` parameter object and then sends it a `getPrice` message.
 - Therefore, the `Sale` object has parameter visibility to the `ProductDescription`, and message-sending coupling, and thus a dependency on the `ProductDescription`.



```
public class Sale
{
    public void updatePriceFor( ProductDescription description )
    {
        Money basePrice = description.getPrice();
        //...
    }
    // ...
}
```

- **Association** models possible relationships between instances of classes
 - When objects of one class work with objects of another class for some prolonged amount of time.
 - A basic and default relationship

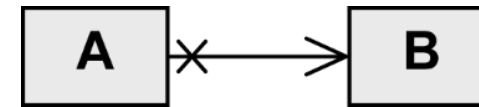


- **Navigability**

- An object knows its partner objects and therefore can access their visible attributes and operations.
- Indicated by open arrow-head or cross

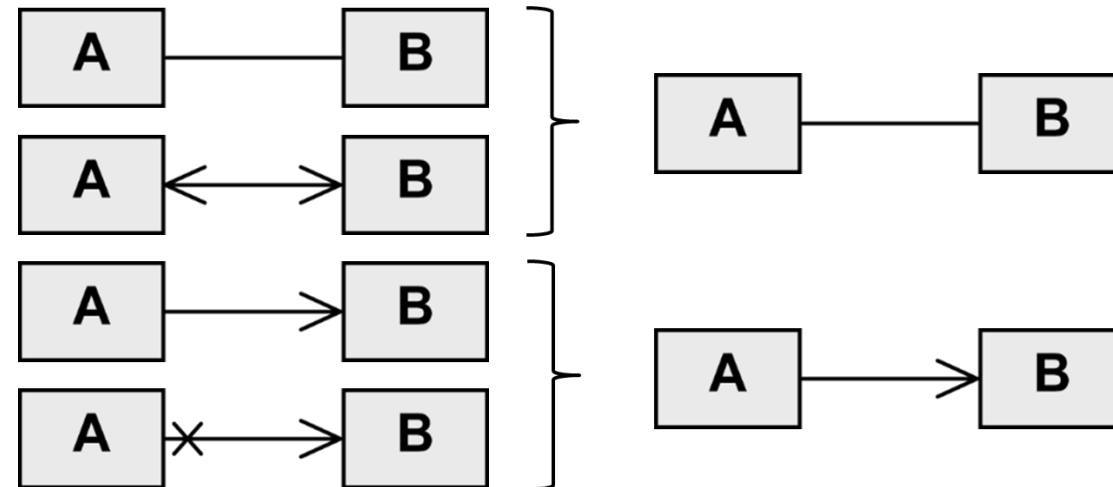
- Example:

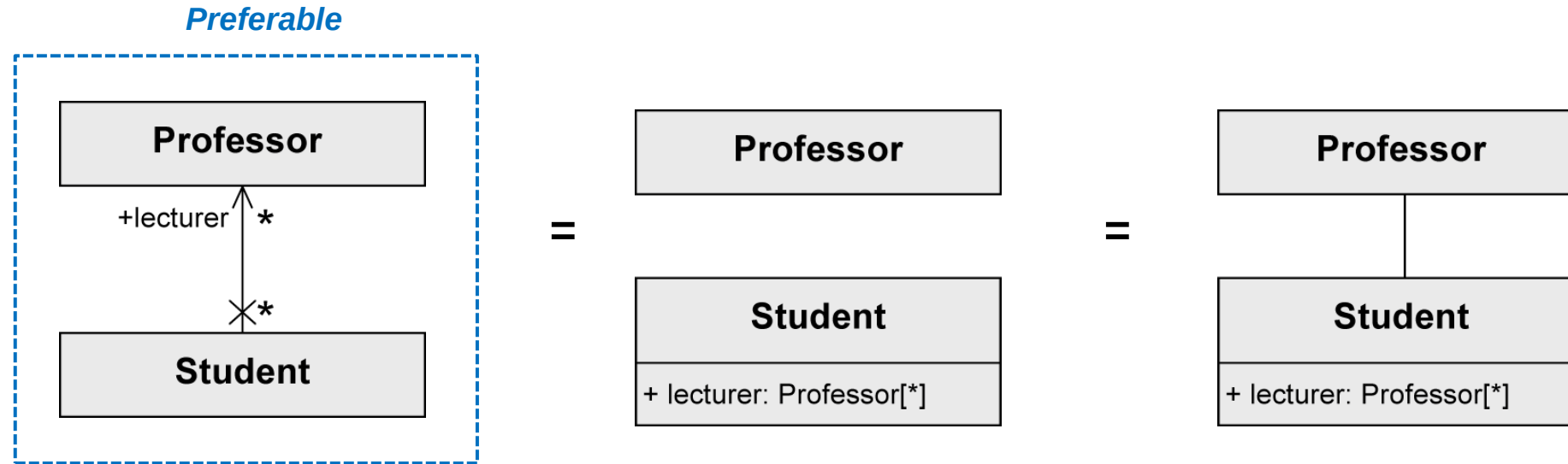
- “A can access the visible attributes and operations of B”
- “B cannot access any attributes and operations of A”



- Navigability undefined

- Bidirectional navigability is assumed.





- Java-like notation:

```
class Professor {...}  
  
class Student {  
    public Professor[] lecturer;  
    ...  
}
```

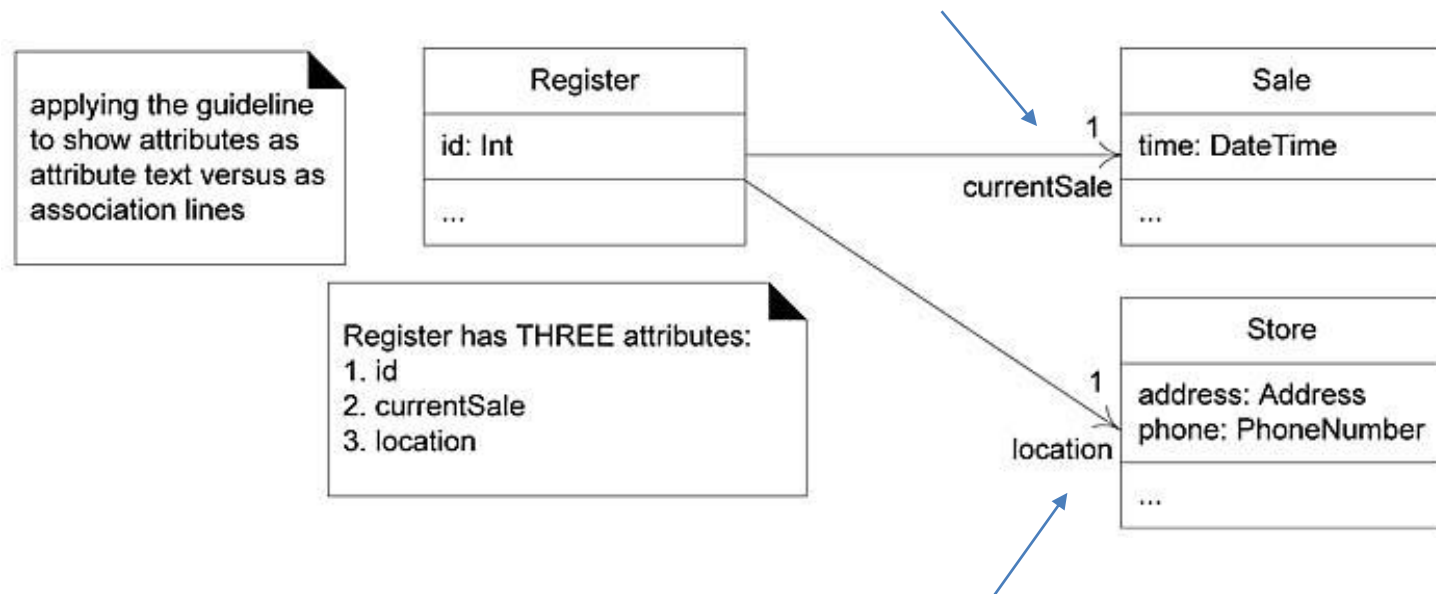
Class attribute

- Use the attribute text notation for data type objects, while the association line notation for others.
 - Both are semantically equal.
 - But showing an association line to another class box in the diagram gives visual emphasis.

```
public class Register
{
    private int id;

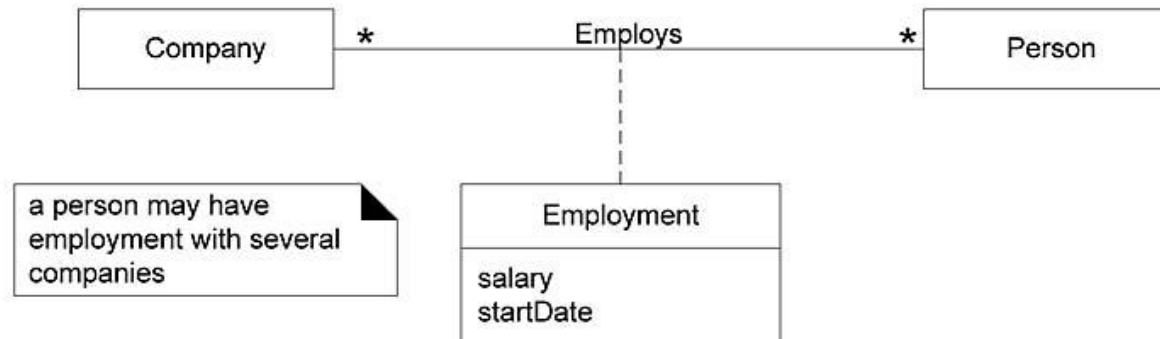
    private Sale currentSale;
    private Store location;

    // ...
}
```

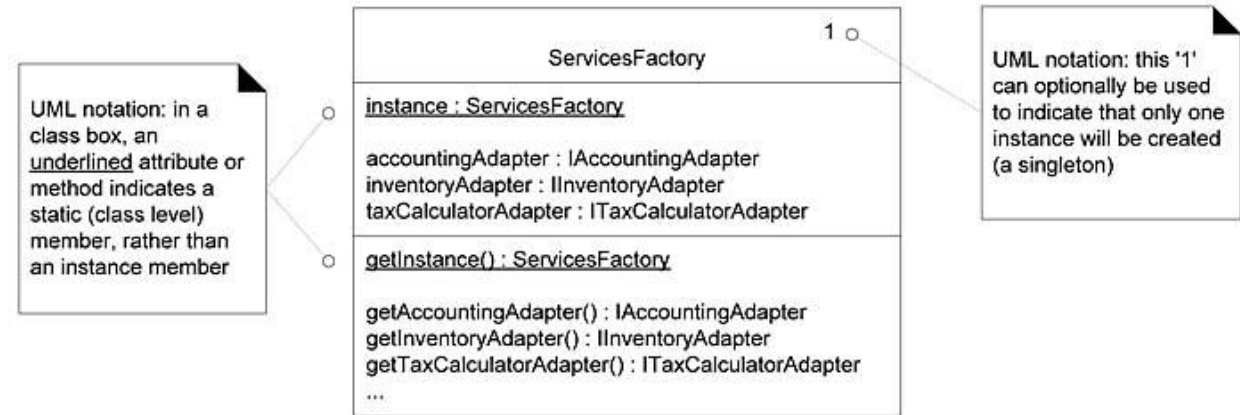


- **Association class**

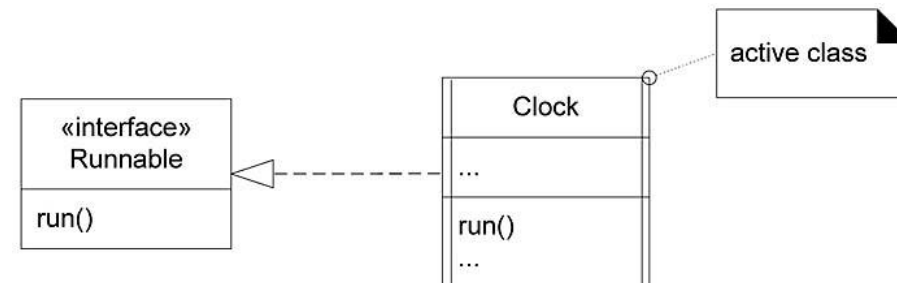
- Assign attributes to the relationship between classes rather than to a class itself.
- Treat an association itself as a class, and model it with attributes, operations, and other features.
 - Illustrated with a dashed line from the association to the association class.
 - Necessary when modeling $n:m$ Associations
- Example:
 - If a *Company* employs many *Persons*, modeled with an *Employs* association, you can model the association itself as the *Employment* class, with attributes such as *salary* and *startDate*.”



- **Singleton class** has only one instance of the class.
 - "singleton" instance
 - Singleton design pattern



- **Active object** runs and controls on its own thread of execution.
 - The class of an active object is an **active class**.



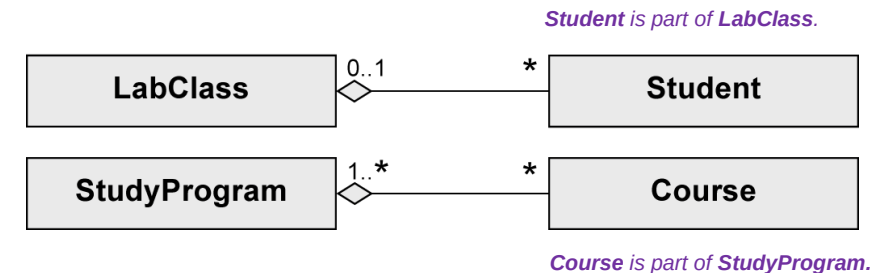
3. Aggregation

- A special form of **association**
 - Used to express that **a class is part of another class**.
 - **Transitive**: if **B** is part of **A** and **C** is part of **B**, **C** is also part of **A**
 - **Asymmetric**: it is not possible for **A** to be part of **B** and **B** to be part of **A** simultaneously.

- Two types:

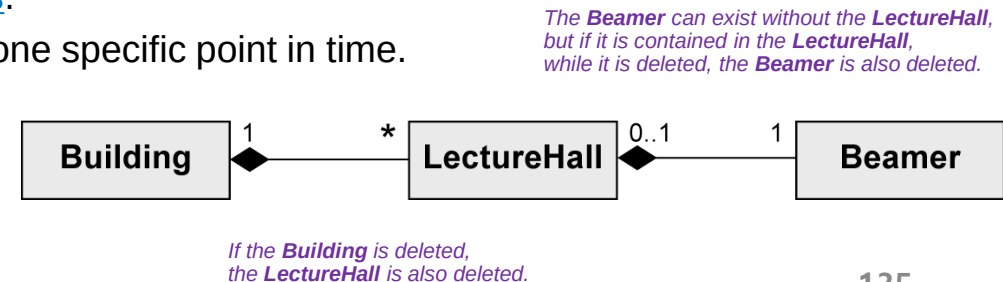
– Shared Aggregation

- Expresses **a weak belonging of the parts to a whole**.
- Parts also exist independently of the whole.
- One element can be part of multiple other elements simultaneously.
- Multiplicity at the aggregating end may be >1.



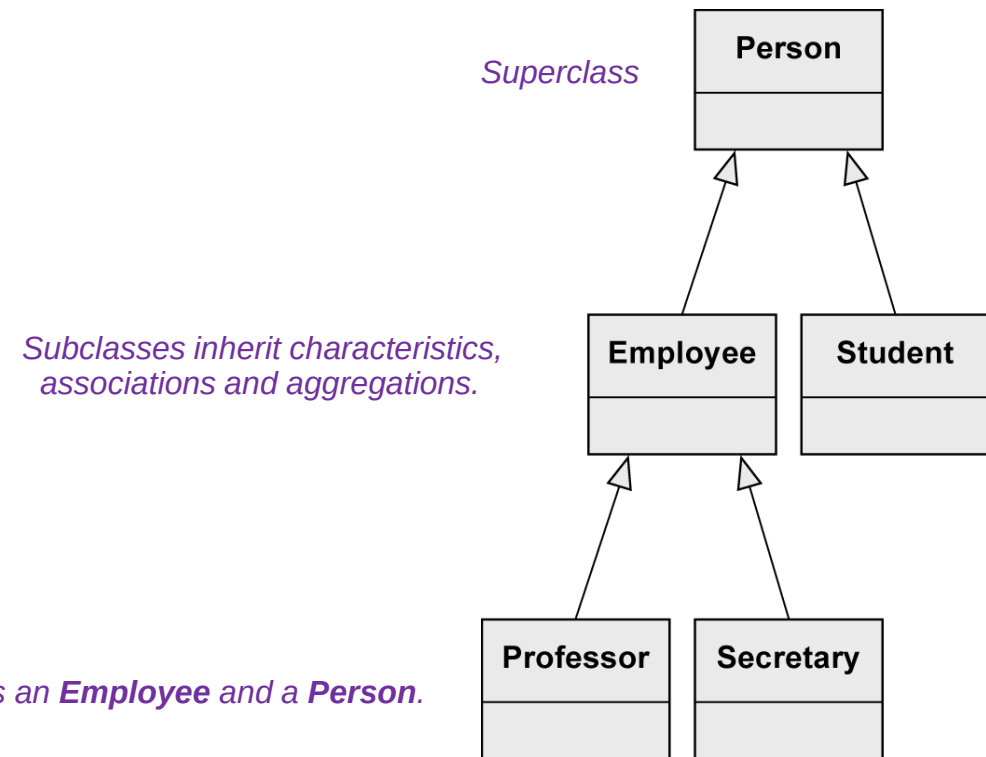
– Composition

- **Existence dependency between the composite object and its parts**.
- One part can only be contained in at most one composite object at one specific point in time.
- If the composite object is deleted, its parts are also deleted.
- Multiplicity at the aggregating end is max 1.



4. Generalization

- Everything of a general class are **passed** on to its subclasses.
 - Every instance of a subclass is simultaneously an indirect instance of the superclass.
 - Subclass **inherits all characteristics** (attributes and operations), **associations, and aggregations** of the superclass.
 - Subclass may have further characteristics, associations, and aggregations.
 - Generalizations are **transitive**.
 - A class may have multiple super classes.
 - Not allowed for JAVA programming language

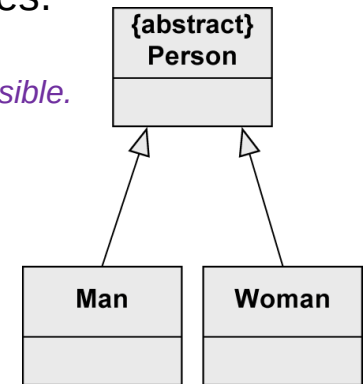


- **Abstract class**

- Used when you want to provide a common base for a group of related classes, allowing them to share common behavior and state, while also enforcing that certain methods must be implemented by subclasses.
 - Can contain both abstract methods (methods declared without an implementation) and concrete methods (methods with an implementation).
 - Can have member variables (fields), constructors, and properties.
- A class can inherit from only one abstract class.

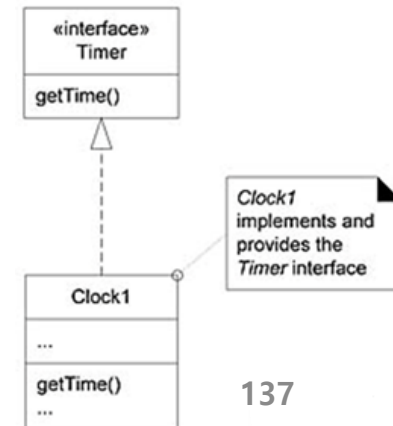
*No **Person-object** is possible.*

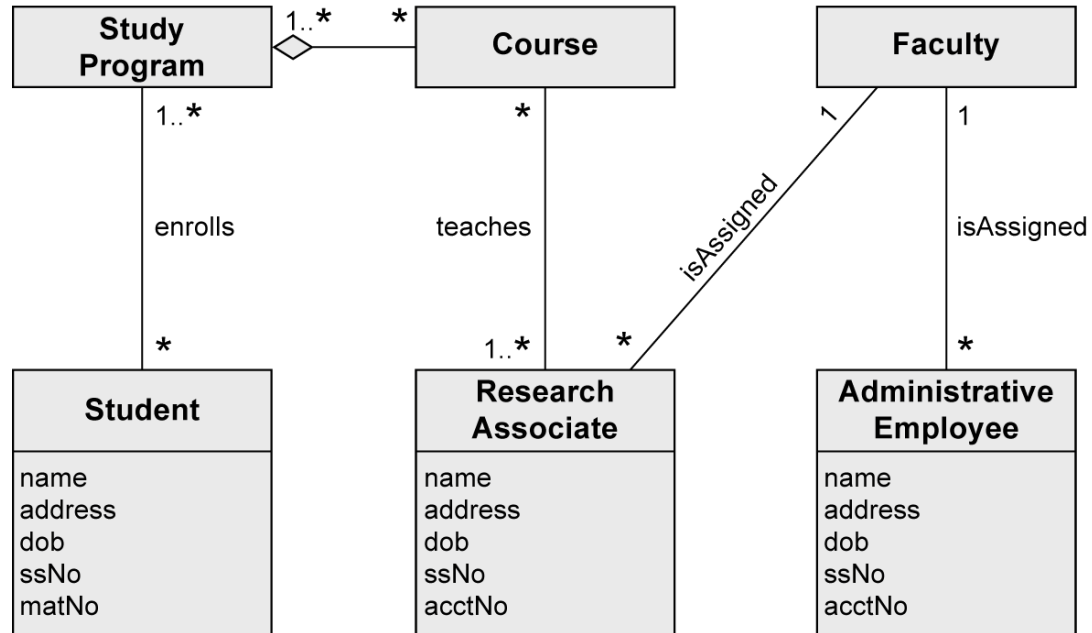
*Two types of **Person**:
Man and **Woman***



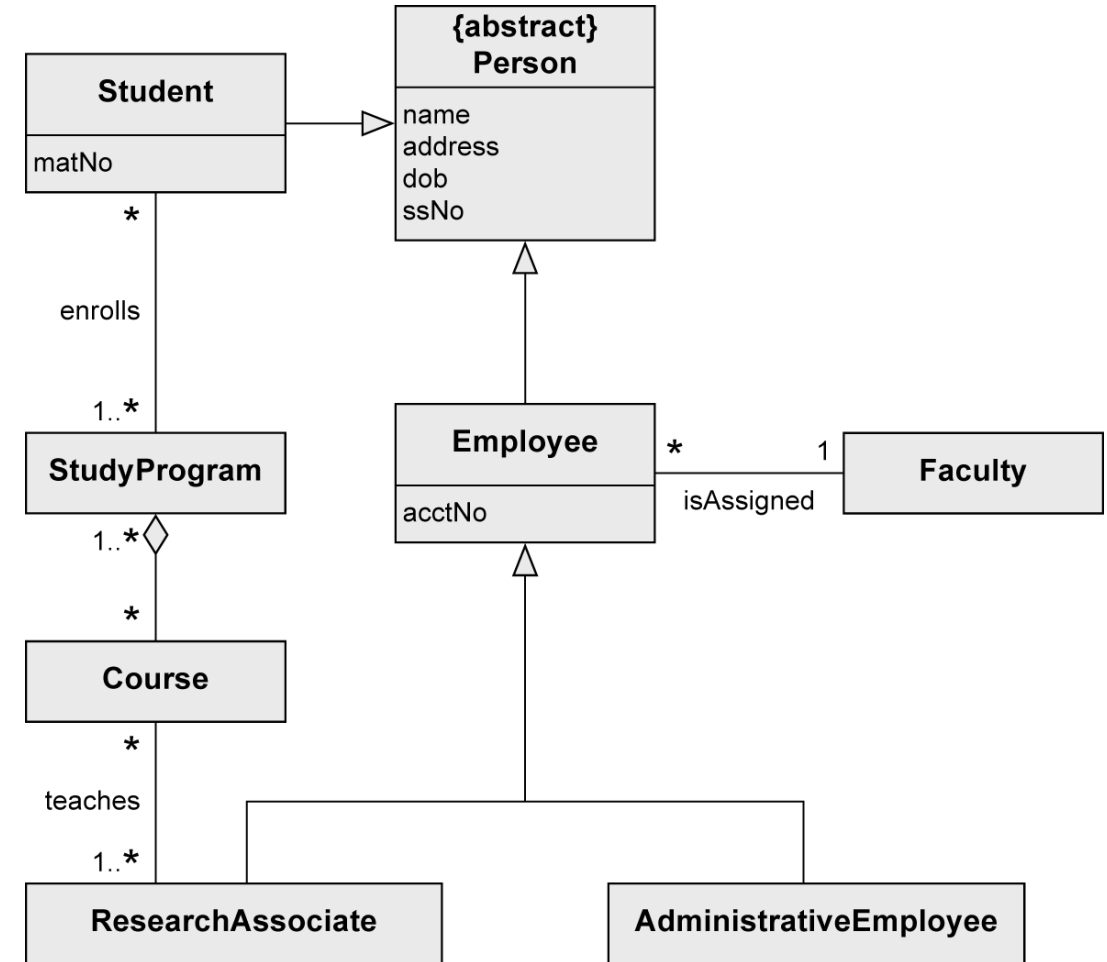
- **Interface class**

- Used when you want to define a specific behavior or capability that can be implemented by unrelated classes, promoting loose coupling and enabling polymorphism across different class hierarchies.
 - Defines a contract, specifying a set of methods that a class must implement.
 - Prior to Java 8, interfaces could only contain abstract methods (method signatures without implementation).
 - Since Java 8, interfaces can also include default methods (with implementation) and static methods.
 - Cannot contain fields (instance variables) or constructors, though they can contain constants.
- A class can implement multiple interfaces, allowing for multiple inheritance of type.

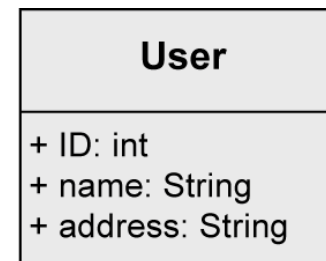
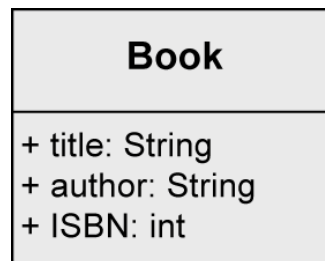




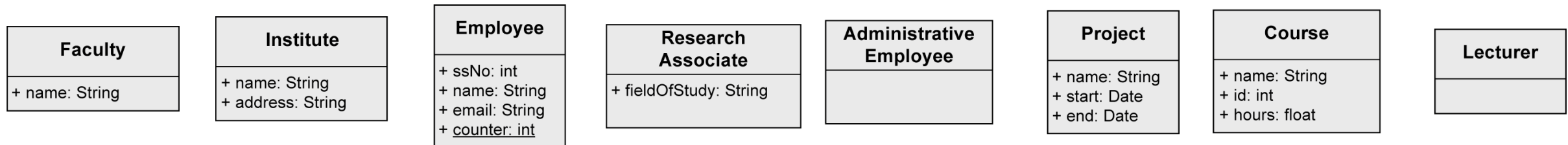
VS.



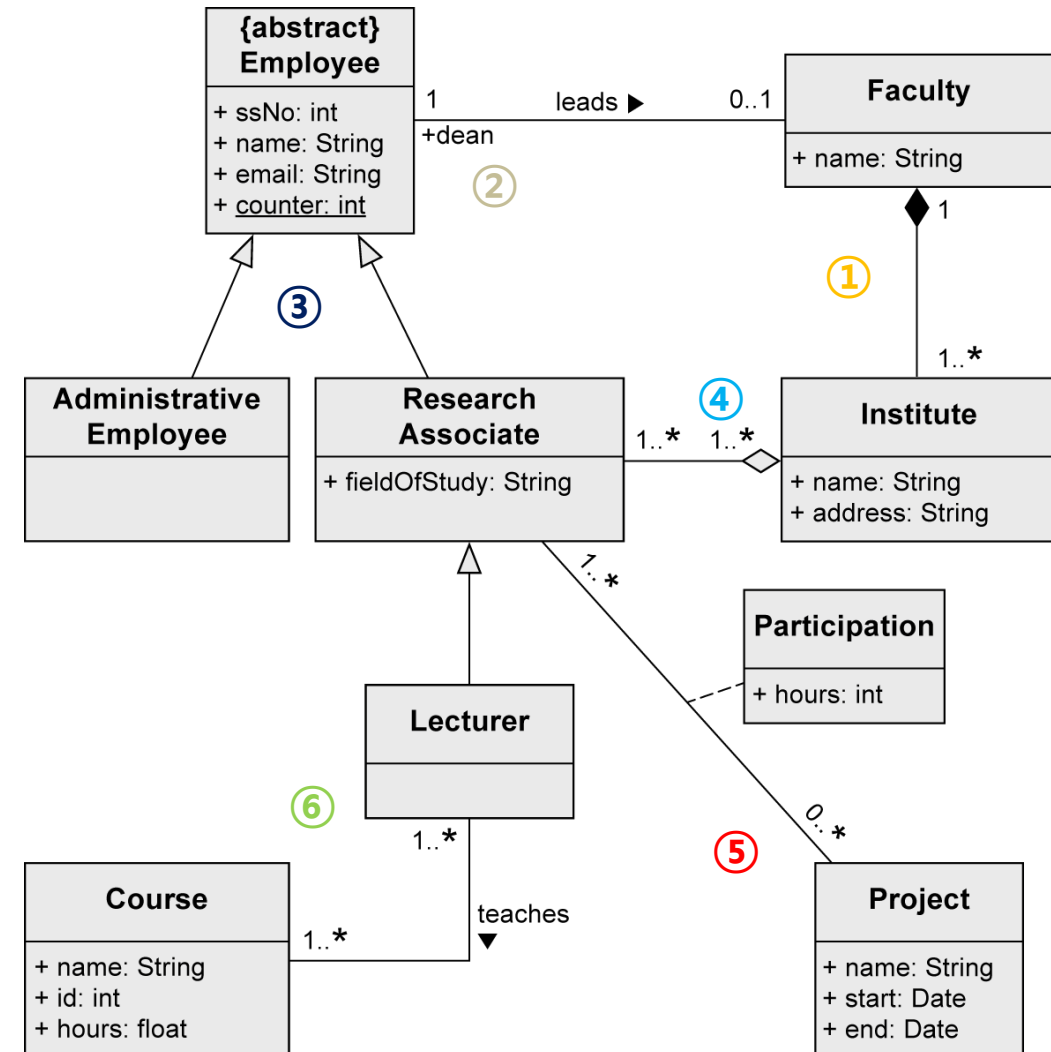
- Not possible to completely extract classes, attributes and associations from a natural language text.
- **Guidelines** for creating **domain models** as well as **design class diagrams**
 - Nouns often indicate classes
 - Adjectives indicate attribute values
 - Verbs indicate operations
- Example:
 - *“The library management system stores **users** with their unique ID, name and address as well as **books** with their title, author and ISBN number. Ann Foster wants to use the library.”*



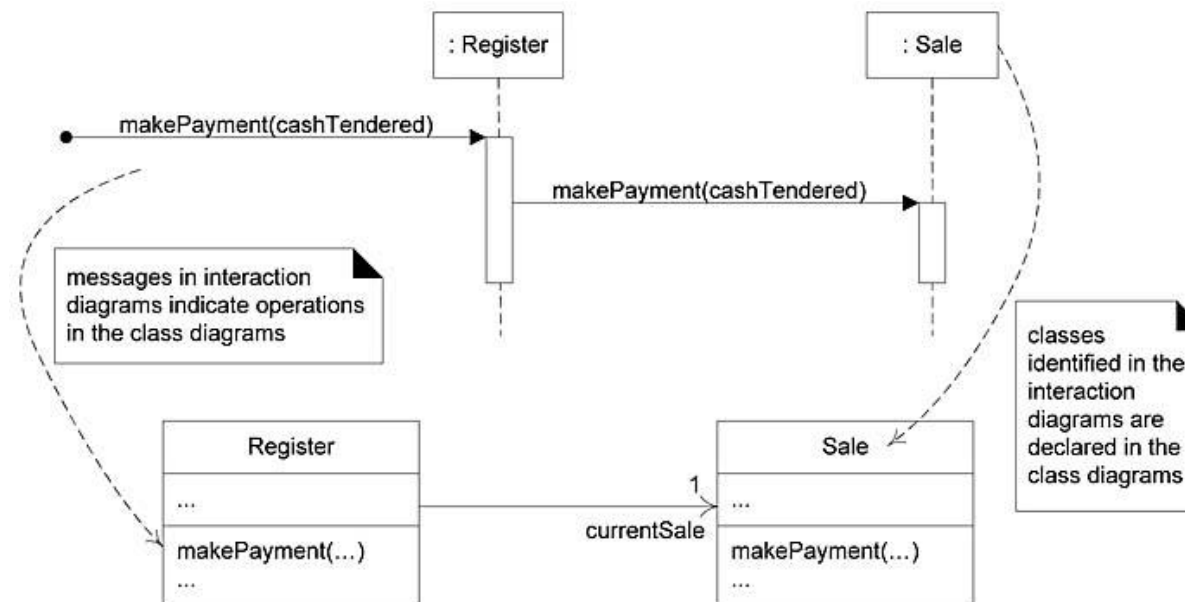
- A university consists of multiple **faculties** which are composed of various **institutes**. Each faculty and each institute has a name. An address is known for each institute.
- Each faculty is led by a dean, who is an **employee** of the university.
- The total number of employees is known. Employees have a social security number, a name, and an email address. There is a distinction between **research** and **administrative personnel**.
- Research associates are assigned to at least one institute. The field of study of each research associate is known. Furthermore, research associates can be involved in **projects** for a certain number of hours, and the name, starting date, and end date of the projects are known. Some research associates hold **courses**. Then they are called **lecturers**.
- Courses have a unique number (ID), a name, and a weekly duration in hours.



1. A university consists of multiple faculties which are composed of various institutes. Each faculty and each institute has a name. An address is known for each institute.
2. Each faculty is led by a dean, who is an employee of the university.
3. The total number of employees is known. Employees have a social security number, a name, and an email address. There is a distinction between research and administrative personnel.
4. Research associates are assigned to at least one institute.
5. The field of study of each research associate is known. Furthermore, research associates can be involved in projects for a certain number of hours, and the name, starting date, and end date of the projects are known.
6. Some research associates hold courses. Then they are called lecturers. Courses have a unique number (ID), a name, and a weekly duration in hours.



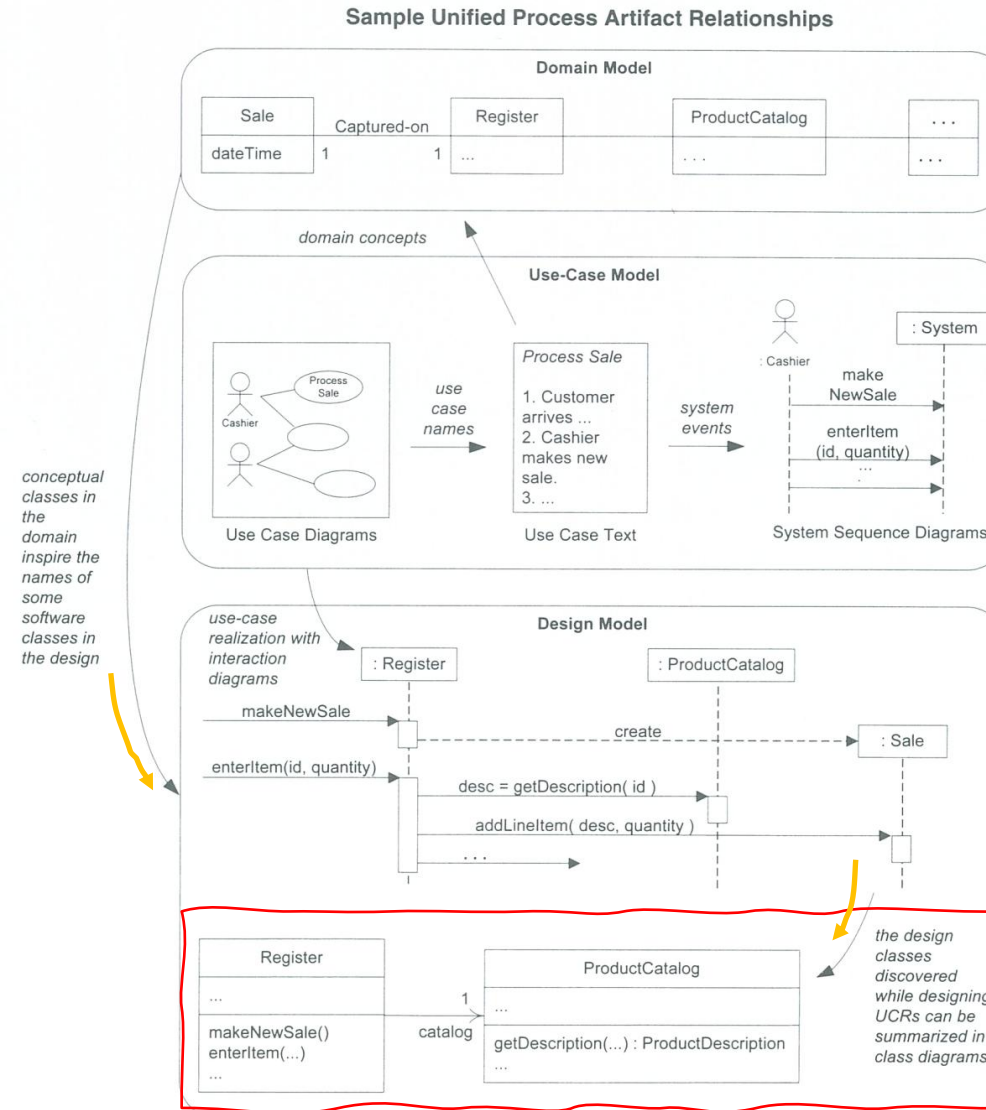
- From sequence diagrams, class diagrams can be **generated iteratively**.
 - When we draw sequence diagrams, a set of classes and their methods that are necessary emerge.
 - **Dynamic Model-Driven Approach**
 - Two complementary dynamic and static views are drawn concurrently and iteratively.
 - Example:
 - If we started with the **makePayment** sequence diagram, we see that a **Register** and **Sale** class definition in a class diagram can be obviously derived.



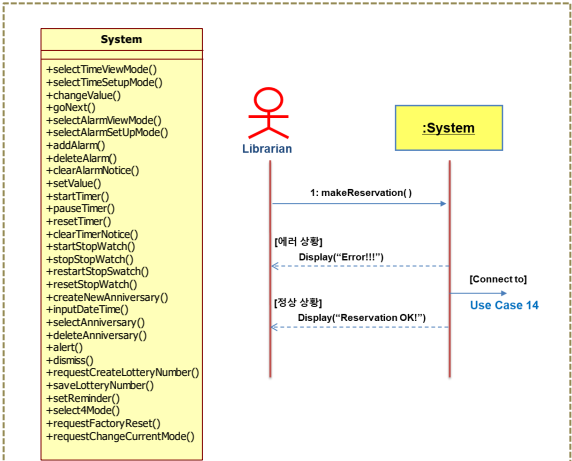
- Artifacts developed in OOD
 - Design Models
 - Static model: **Class diagram**
 - Dynamic model: **Sequence diagrams**

Sample Unified Process Artifacts and Timing (s-start; r-refine)

Discipline	Artifact Iteration→	Incep.	Elab.	Const.	Trans.
		I1	E1..En	C1..Cn	T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	
Implementation	Implementation Model (code, html, ...)		s	r	r



- OOD (Object-Oriented Design)



System Operations + SSDs

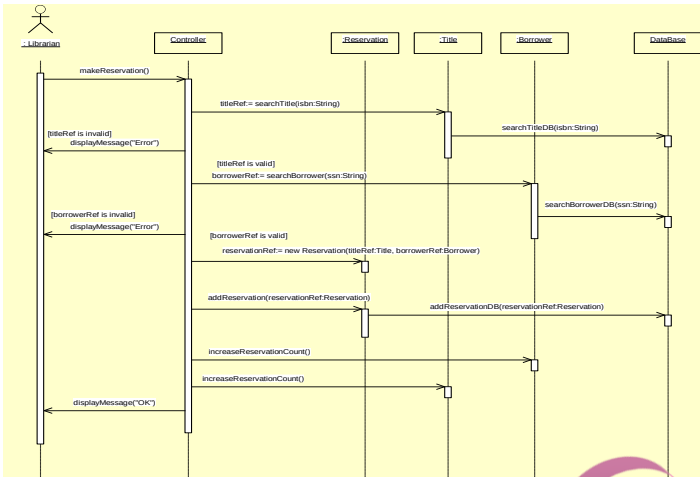
Use Case	1. Make Reservation
Actor	Librarian
Purpose	Create a new reservation
Overview	(As in the business use case)
Type	Primary and Real
Cross Reference	System Functions: R1.1, R3.1 Use Case: "Add Borrower"
Pre-Requisites	A borrower should be registered.
Typical Courses of Events	(A) : Actor, (S) : System 1. (A) A librarian inputs an isbn and ssn of the title 2. (S) Find a corresponding title 3. (S) Find a corresponding borrower 4. (S) Create a new reservation 5. (S) Store the new reservation 6. (S) Increase reservationCount in the borrower 7. (S) Increase reservationCount in the title
Alternative Courses of Events	N/A
Exceptional Courses of Events	Line 2: If the title does not exist, display an error message. Line 3: If the borrower does not exist, display an error message.

Use Cases (Fully-Dressed)

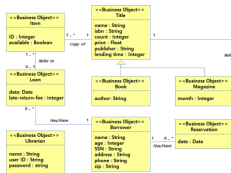
for each use case (+SSD)



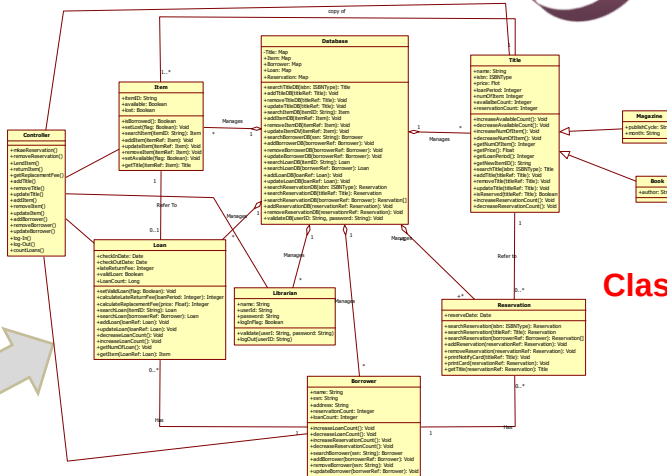
Sequence Diagrams



Domain Model

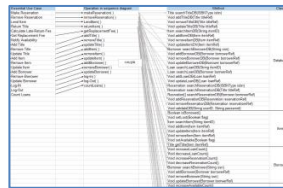


Class Diagram



System Architecture

Component Diagram + Package Diagram



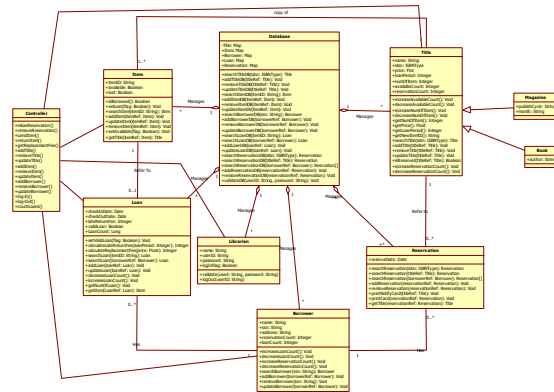
Traceability Table

A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue shadow on the left, a white circle with a blue shadow at the top left, a circle with diagonal white stripes on the top right, and a 3x3 grid of small white dots at the bottom right.

OOI

- Visibility

- OOI (Object-Oriented Implementation)



Class Diagram
(Static Model)

Code
Generation

Skeleton Code

```
class A
{
  variables:

  aaa();
  bbb();
  ...
}
```

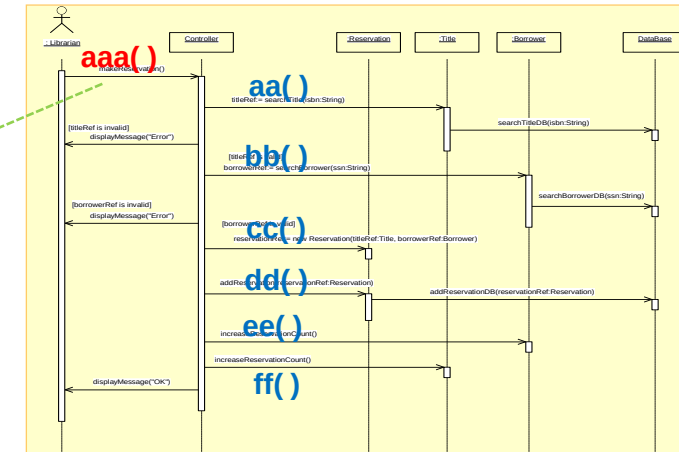
Realization

Concrete Code

```
class A
{
  variables:

  aaa()
  {
    aa();
    bb();
    cc();
    dd();
    ee();
    ff();
  }
  bbb();
  ...
}
```

System
Operation

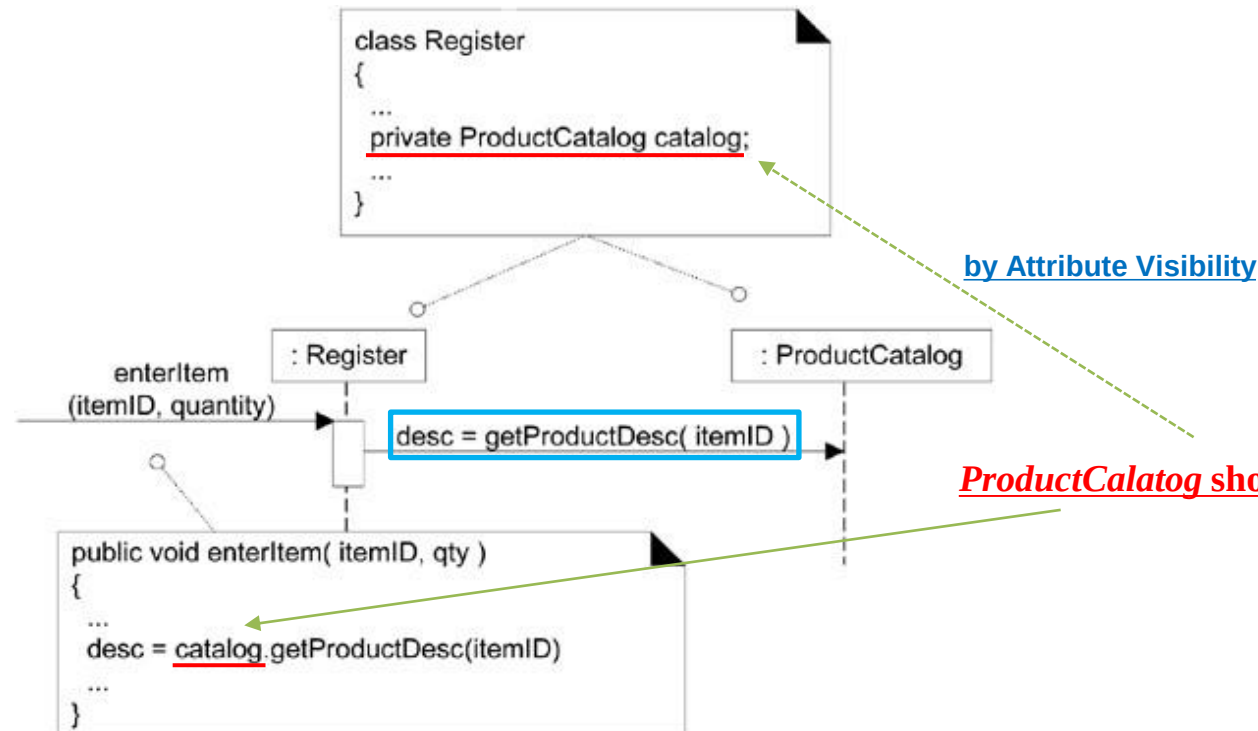


Sequence Diagrams
(Dynamic Model)

with a hint

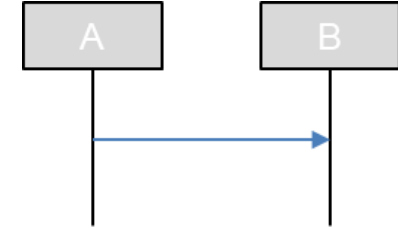
+ 변수 선언
+ 조건문
+ 초기화
+ Visibility 확보

- In message passing between objects,
 - For a sender object to send a message to a receiver object, the receiver must be **visible** to the sender.
 - The sender must have some kind of **reference** or pointer to the receiver object.
 - Example,
 - The **getProductDesc** message sent from a **Register** to a **ProductCatalog** implies that the **ProductCatalog** instance should be visible to the **Register** instance.



- **Visibility** is the ability of an object to “see” or “have a reference to” another object.

- When an object A sends a message to an object B, B must be visible to A.
- The issue of scope: *“Is one resource (such as an instance) within the scope of another?”*

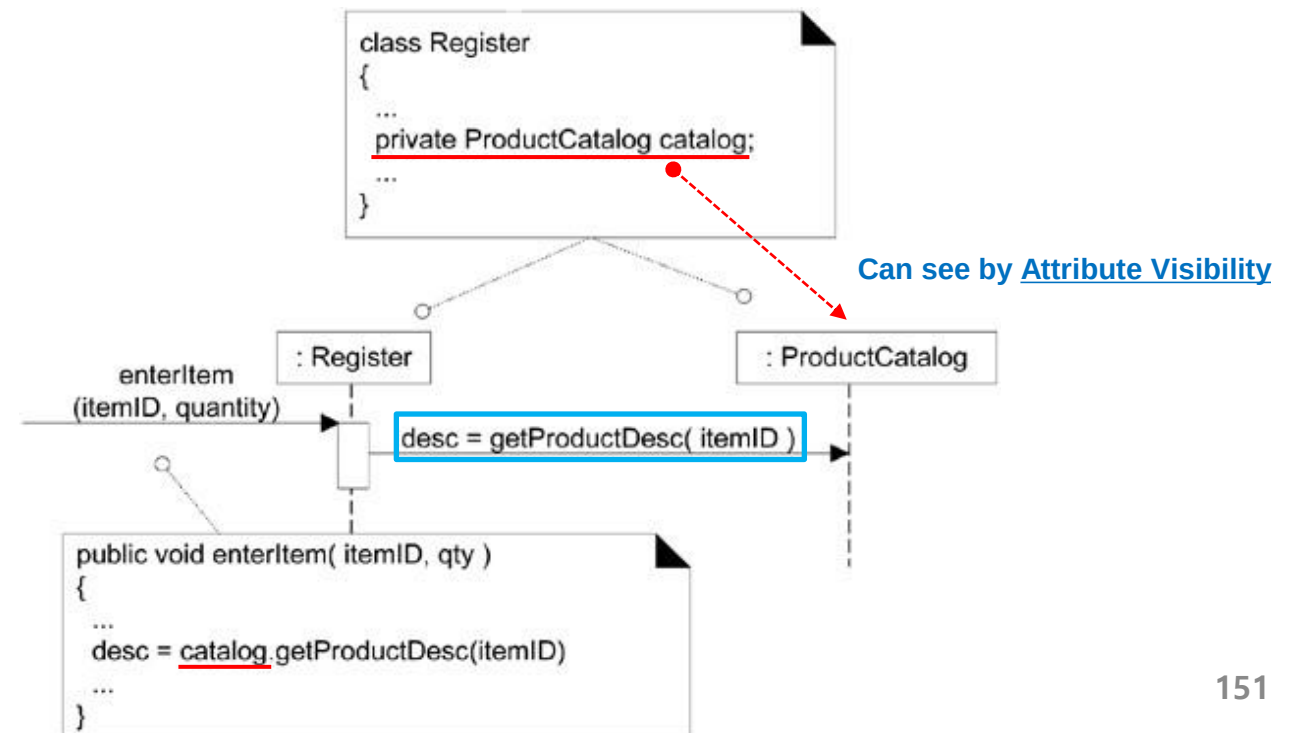
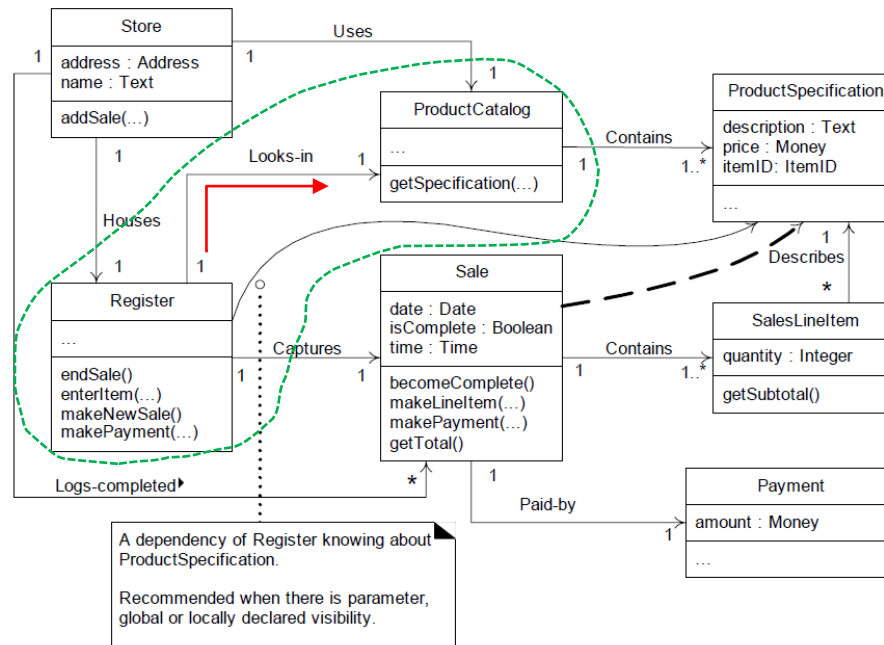


- 4 common ways that visibility can be achieved from object A to object B:
 1. **Attribute visibility** : **B** is an attribute of **A**.
 2. **Parameter visibility** : **B** is a parameter of a method of **A**.
 3. **Local visibility** : **B** is a (non-parameter) local object in a method of **A**.
 4. **Global visibility** : **B** is in some way globally visible.

1. Attribute Visibility

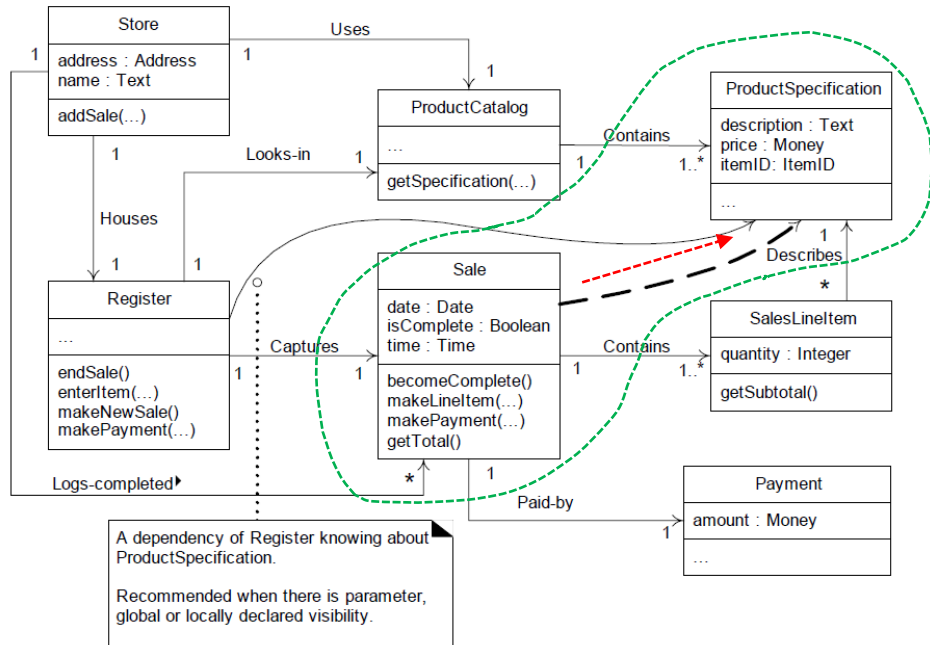


- **Attribute visibility** from *A* to *B* exists, when *B* is an attribute of *A*.
 - Relatively permanent visibility, because it persists as long as *A* and *B* exist.
 - Very common form of visibility in object-oriented systems
- For example,
 - For the class **Register**, a **Register** instance may have attribute visibility to a **ProductCatalog**, since it is an attribute of the **Register**.



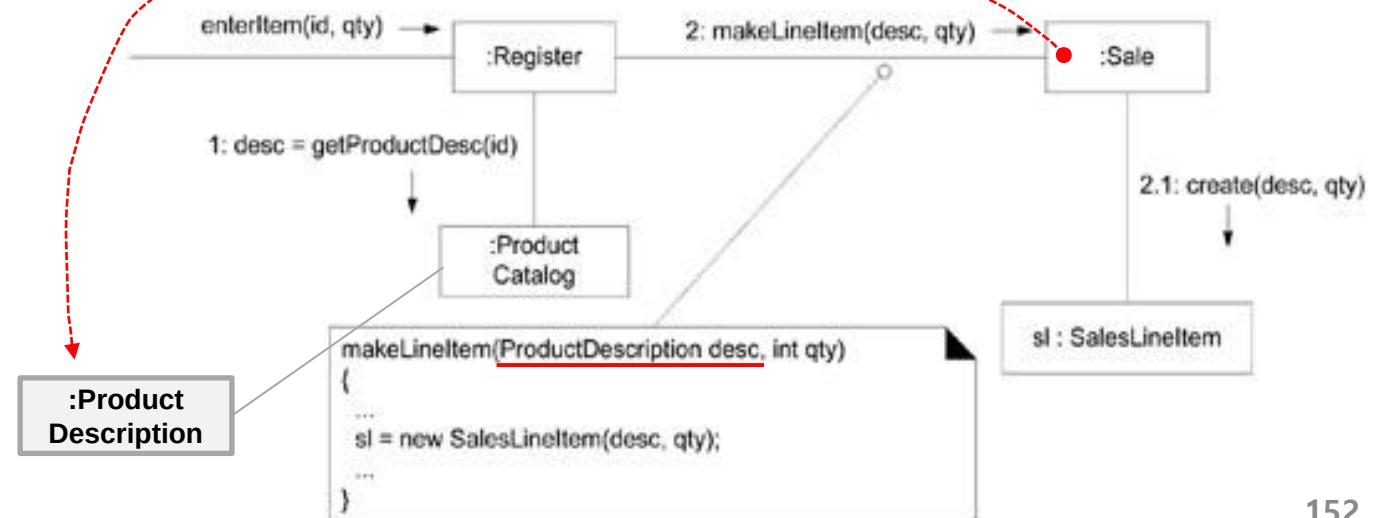
2. Parameter Visibility

- **Parameter visibility** from *A* to *B* exists, when *B* is passed as a parameter to a method of *A*.
 - Relatively temporary visibility, because it persists only within the scope of the method.
 - The second most common form of visibility in object-oriented systems.
- For example,
 - When the **makeLineItem** message is sent to a **Sale** instance, a **ProductDescription** instance is passed as a parameter. Within the scope of the **makeLineItem** method, the **Sale** has parameter visibility to a **ProductDescription**.



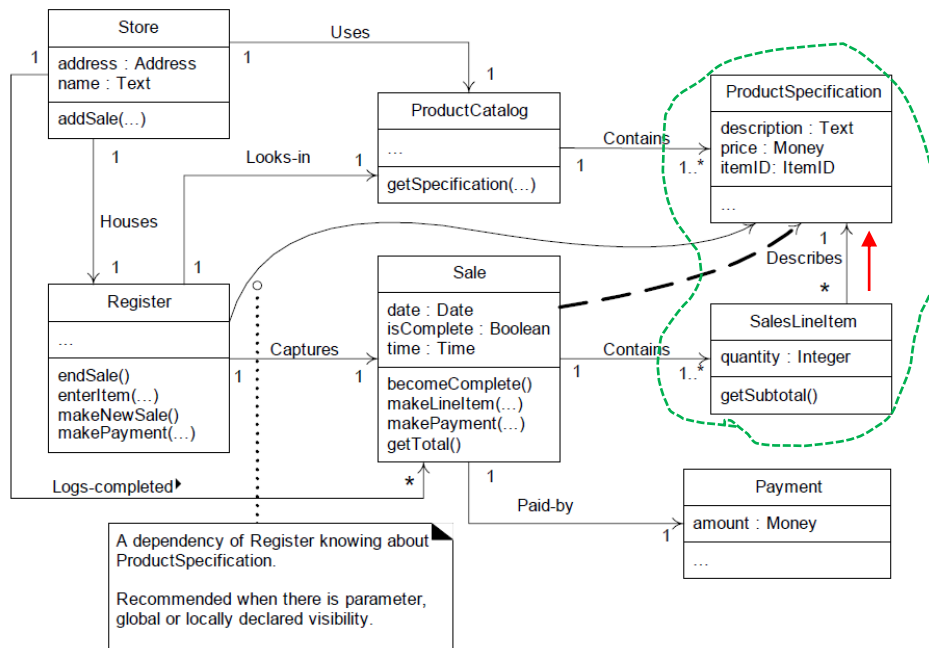
Dependency relationship
in class diagram

Can see by Parameter Visibility, for a while



- It is common to transform parameter visibility into attribute visibility.
 - For example,
 - When the **Sale** creates a new **SalesLineItem**, it passes the **ProductDescription** into its initializing method (in C++ or Java, this would be its constructor). Within the initializing method, the parameter is assigned to an attribute, thus establishing attribute visibility.

Can see by [Attribute Visibility](#), permanently



```
public class SalesLineItem
{
    private int quantity;
    private ProductDescription description;

    public SalesLineItem (ProductDescription desc, int quantity )
    {
        this.description = desc; this.quantity = quantity;
    }

    public Money getSubtotal()
    {
        return description.getPrice().times( quantity );
    }
}
```

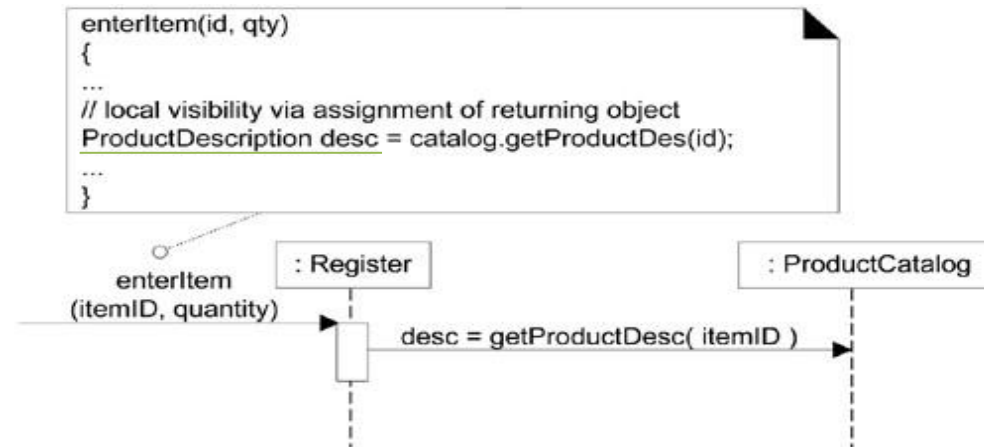
Attribute Visibility

Parameter Visibility

3. Local Visibility



- **Local visibility** from *A* to *B* exists, when *B* is declared as a local object within a method of *A*.
 - Relatively temporary visibility, because it persists only within the scope of the method.
 - As with parameter visibility, it is common to transform into attribute visibility.
- Two common ways for local visibility:
 1. Create a new local instance and assign it to a local variable.
 2. Assign the returning object from a method invocation to a local variable.



- **Global visibility** from A to B exists, when B is global to A .
 - Relatively permanent visibility, because it persists as long as A and B exist.
 - The least common form of visibility in object-oriented systems
- One way to achieve global visibility is
 - Assigning an instance to a global variable, which is possible in some languages such as C++.
 - Not possible for Java
- The preferred method to achieve global visibility is to use the **Singleton pattern**.

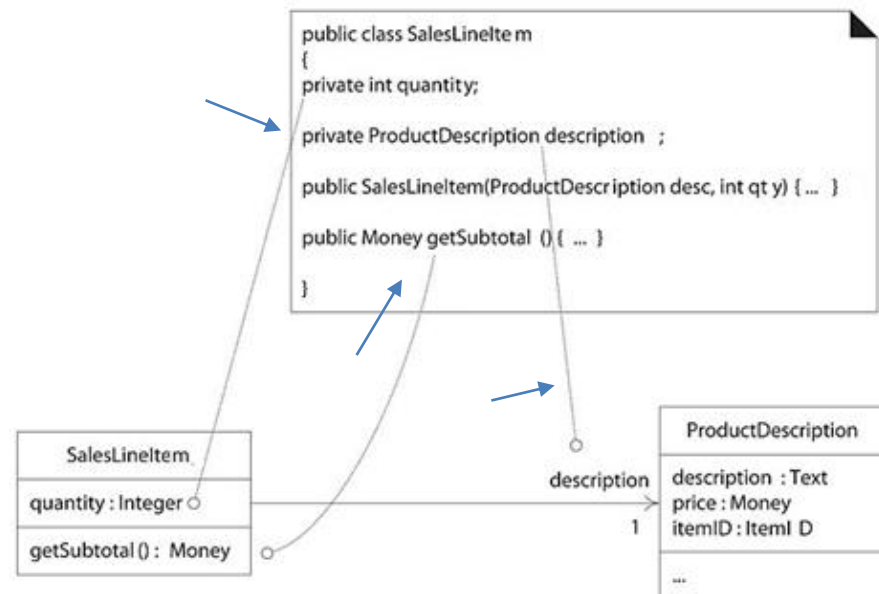
A decorative blue sidebar on the left side of the slide. It features several geometric shapes: a large dark blue circle at the bottom left, a smaller white circle with a blue shadow on the left, a white circle with a blue shadow at the top left, a dark blue circle with a white shadow at the top left, a white circle with a blue shadow at the top left, and a grid of 12 white dots arranged in 3 rows and 4 columns at the bottom right.

OOI

- Mapping Designs to Code

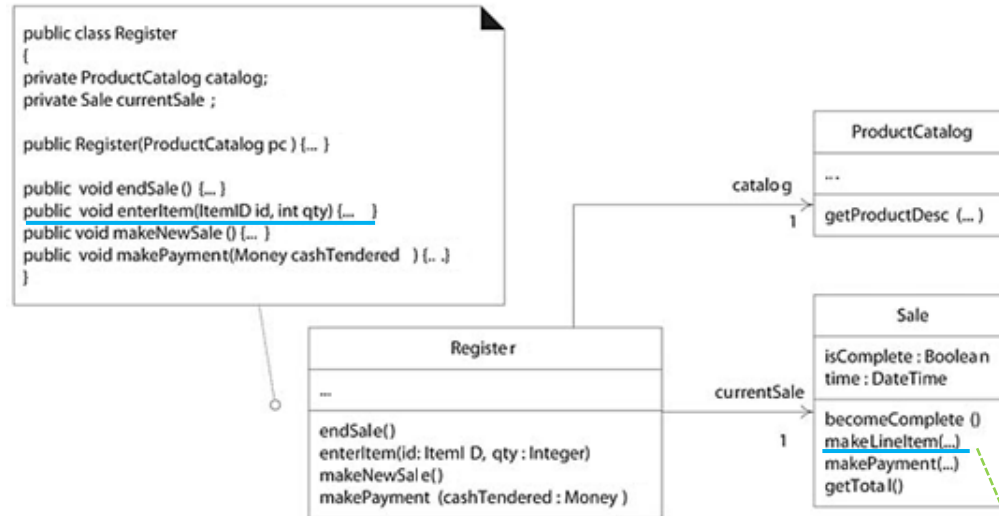
- The UML artifacts created during the design work (**Sequence Diagrams and Class Diagram**) are used as input to **the code generation** process.
- **A translation from UML designs to code** is required.
 - from **Class diagrams** to Class definitions,
 - from **Sequence diagrams** to Method bodies.

- **Class diagrams** are sufficient to create a basic class definition in an OO language.
 - For example,
 - From the class diagram, a mapping to the [attribute definitions\(Java fields\)](#) and [method signatures](#) for the Java definition (*SalesLineItem*) is straightforward.

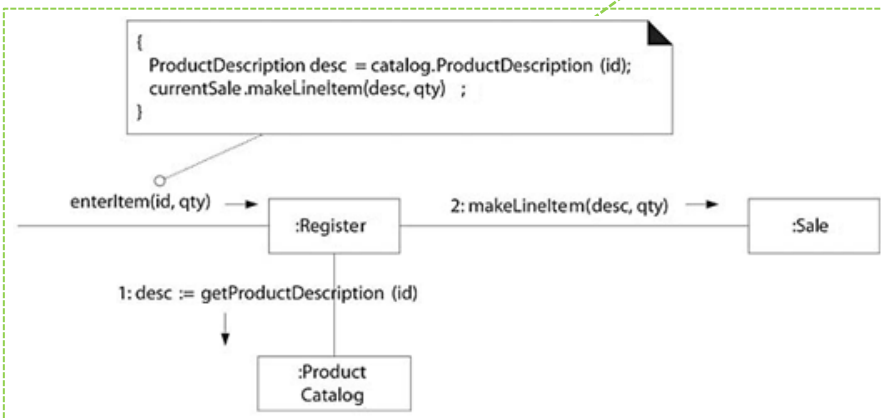


- **Sequence diagrams** have sufficient information to implement the body of the methods in Class diagram.
 - Dynamic Model-driven approach

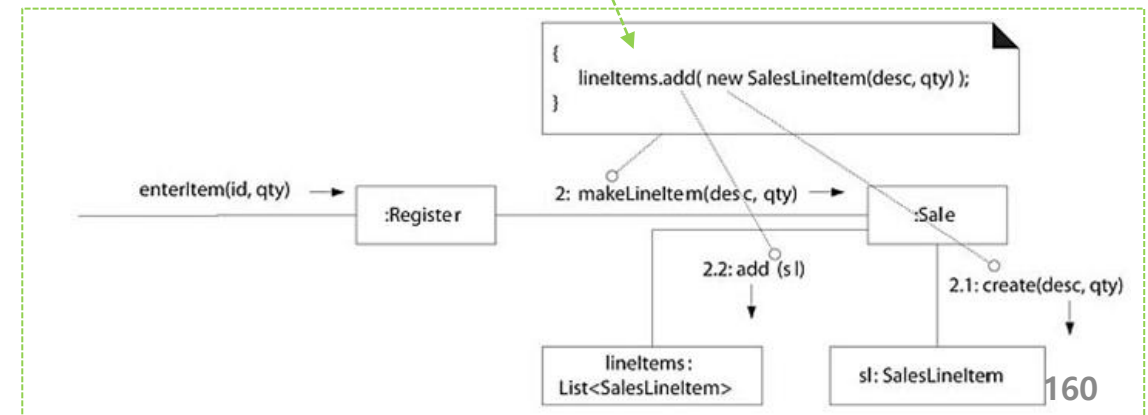
The Register.enterItem Method



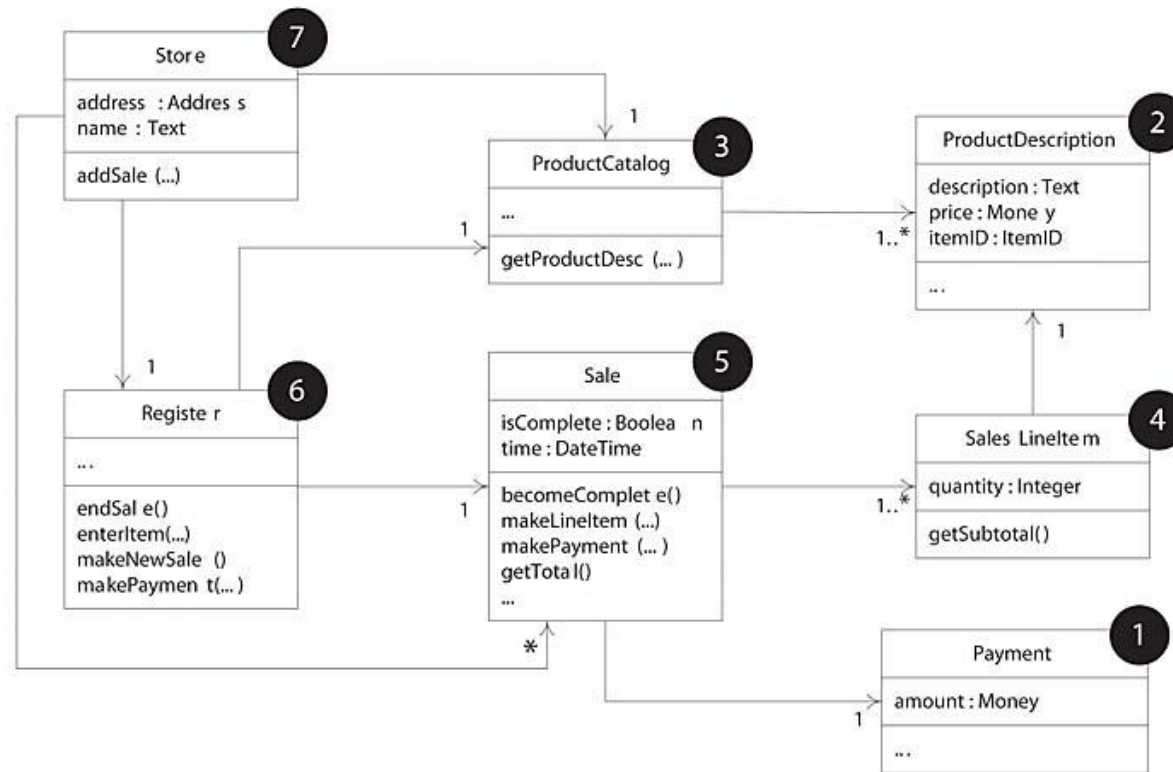
enterItem()



makeLinItem()



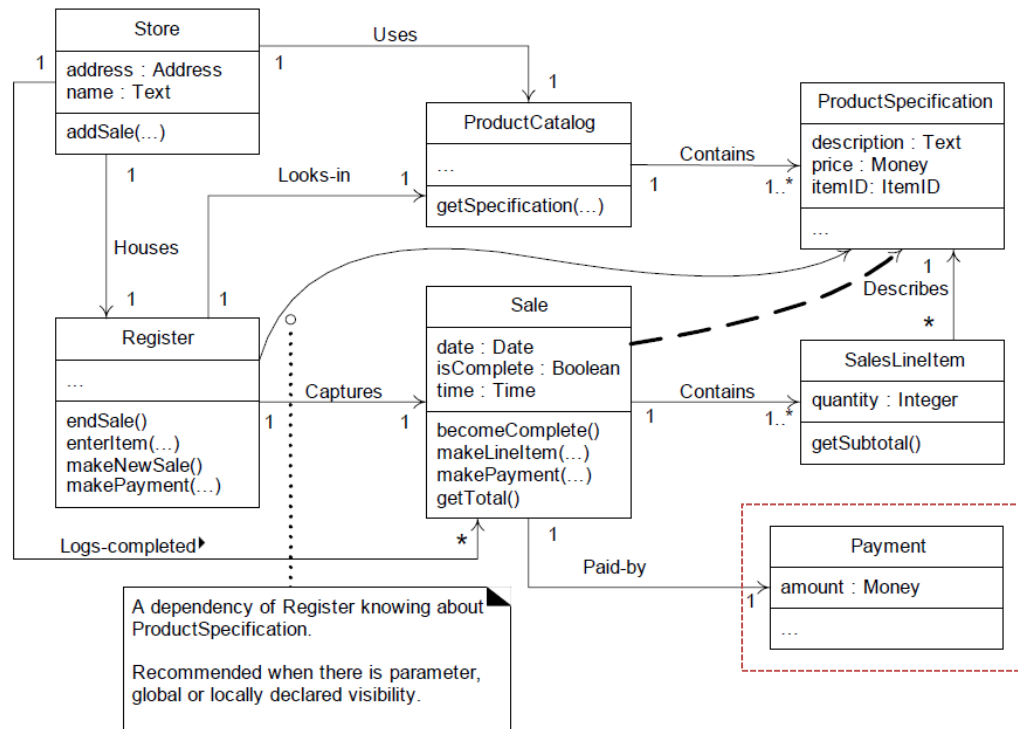
- Classes need to be implemented **from least-coupled to most-coupled**.
 - For example,
 - Possible first classes to implement are either **Payment** or **ProductDescription**.
 - Next are classes only dependent on the prior implementations; **ProductCatalog** or **SalesLineItem**.



Example: the NextGen POS Program Solution



- **Translation** from design artifacts to a foundation of code.
 - This code defines a simple case; it is not meant to illustrate a robust, fully developed Java program with synchronization, exception handling, and so on.



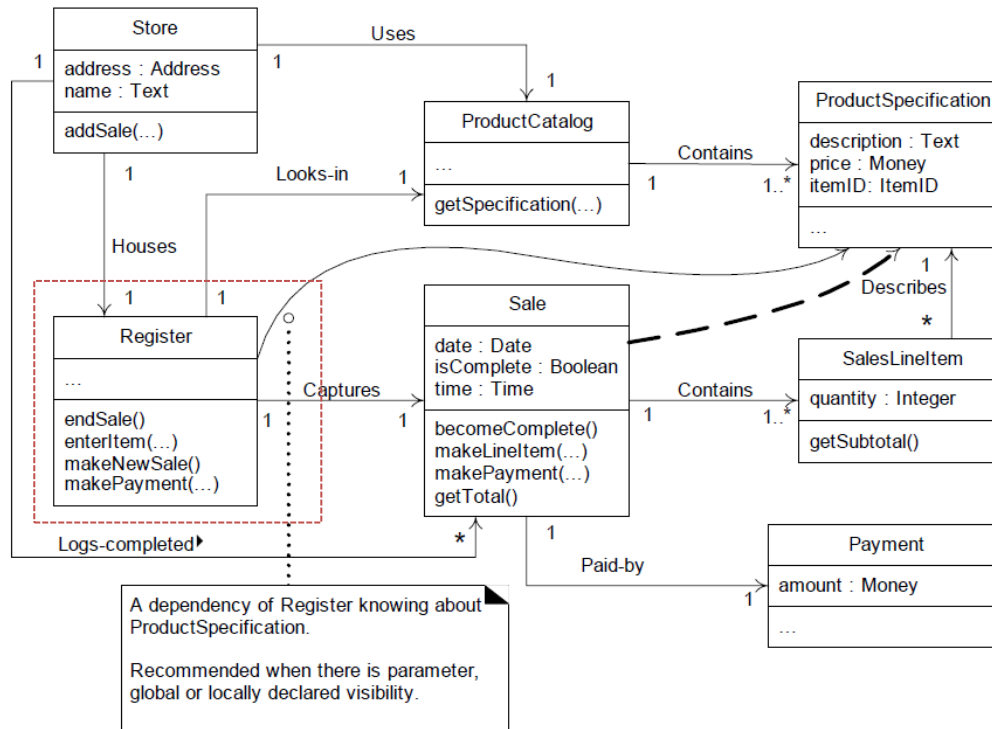
```
// all classes are probably in a package named something like:
// package com.foo.nextgen.domain;

public class Payment
{
    private Money amount;

    public Payment( Money cashTendered ){ amount = cashTendered; }

    public Money getAmount() { return amount; }
}
```

Example: the NextGen POS Program Solution



```

public class Register
{
    private ProductCatalog catalog;
    private Sale currentSale;

    public Register ( ProductCatalog catalog )
    {
        this.catalog = catalog;
    }

    public void endSale()
    {
        currentSale.becomeComplete();
    }

    public void enterItem ( ItemID id, int quantity )
    {
        ProductDescription desc = catalog.getProductDescription ( id );
        currentSale.makeLineItem ( desc, quantity );
    }

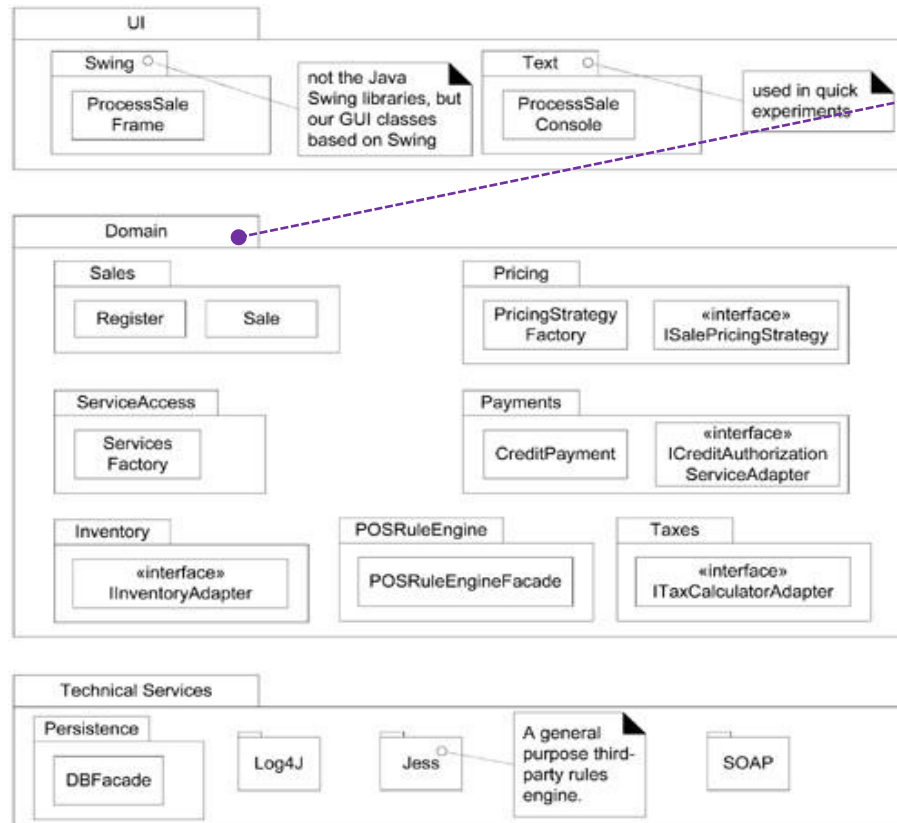
    public void makeNewSale()
    {
        currentSale = new Sale();
    }

    public void makePayment ( Money cashTendered )
    {
        currentSale.makePayment ( cashTendered );
    }
}
    
```

Example: POS Domain Model Packages



- After Elaboration - Iteration 3.

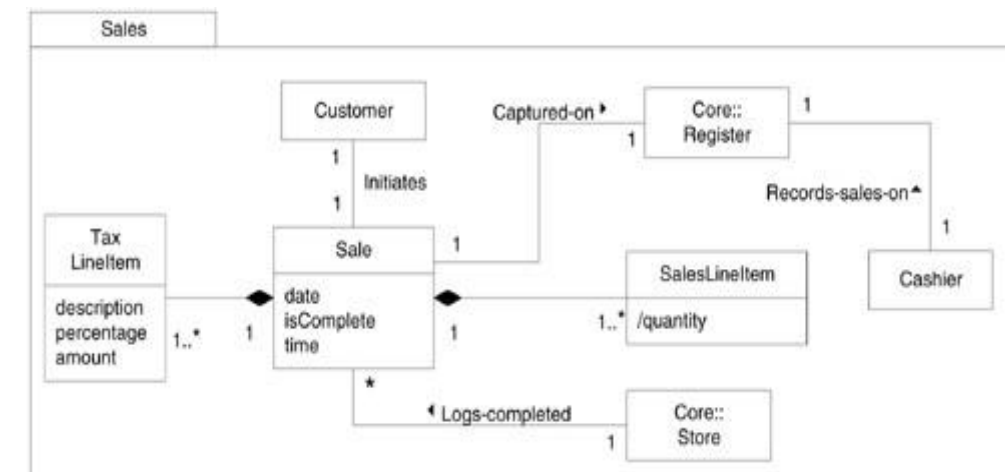
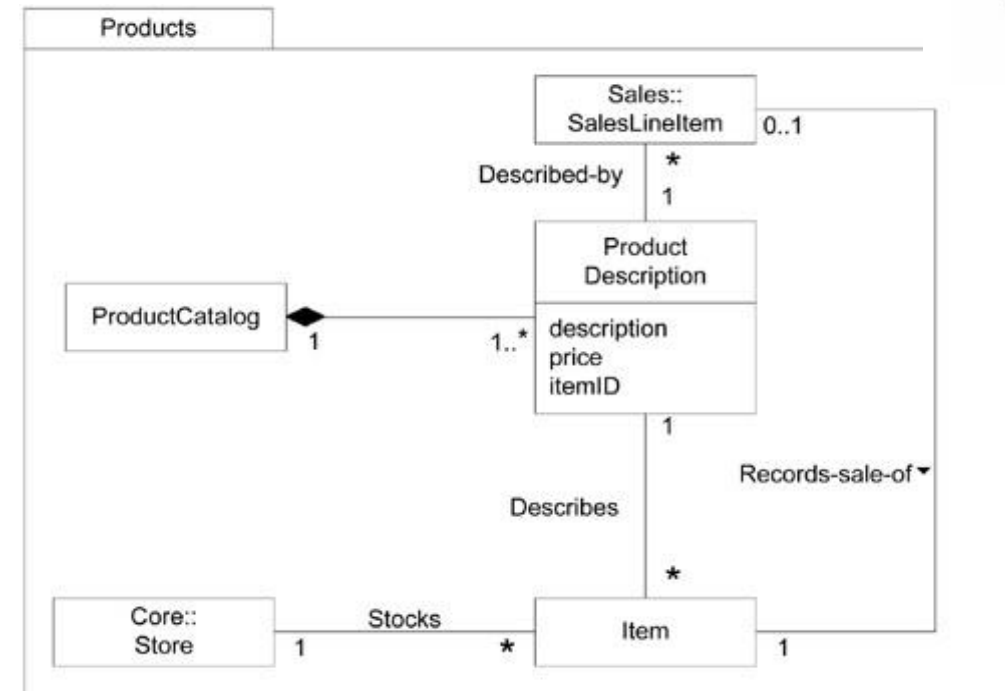
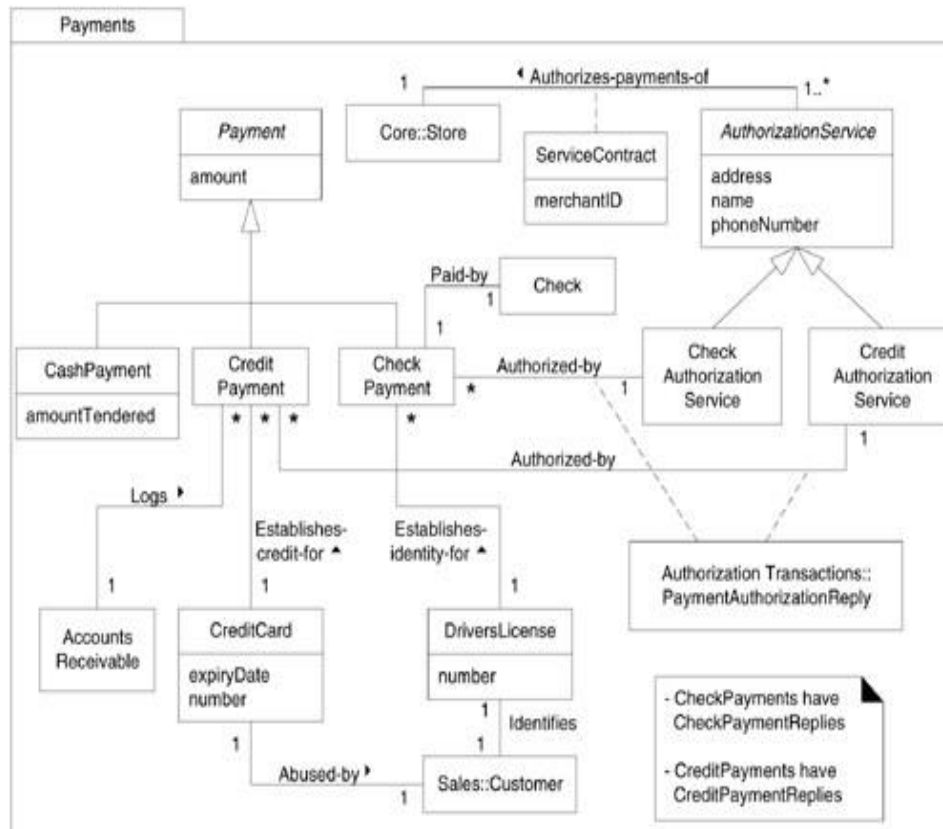
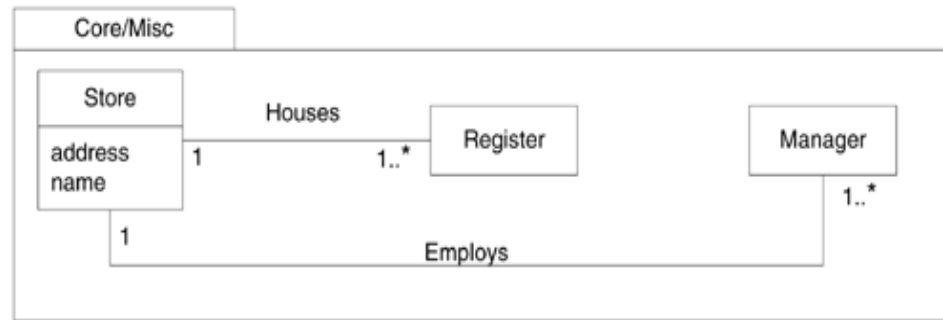


(at the early iterations)



(at the later iterations)

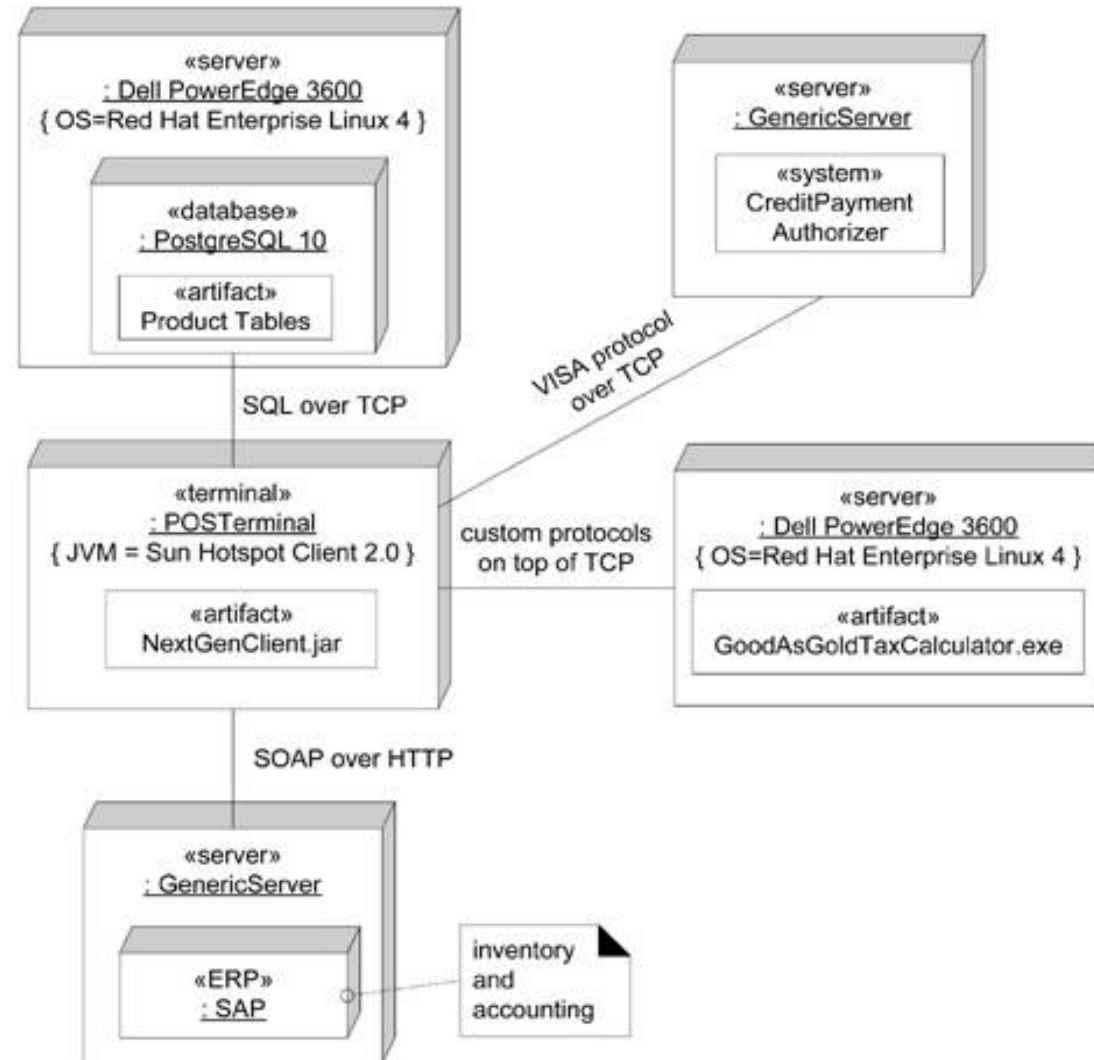
Example: POS Domain Model Packages



Deployment View of POS



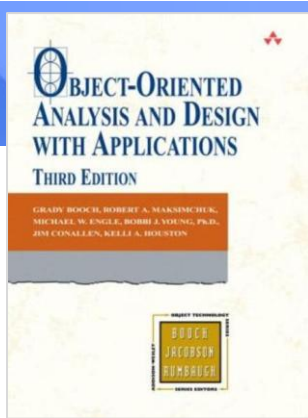
Deployment View



A decorative blue sidebar on the left side of the slide. It features several geometric elements: a large dark blue circle at the bottom left, a smaller white circle with a blue outline above it, a small white circle with a blue outline further up, a circle with diagonal white stripes, and a 3x3 grid of small white dots.

Summary

Object-Oriented Analysis and Design (OOAD)



- **OOAD** (Object-Oriented Analysis and Design)
 - *“Identifying your requirements and creating a domain model, and then add methods to the appropriate classes, and define the messaging between the objects to fulfill the requirements”*
 - **Object-Oriented Analysis (OOA)**
 - Identify requirements → **Use-Case model**
 - Discover the domain concepts/objects → **Domain model**
 - **Object-Oriented Design (OOD)**
 - Define software objects (static model) → **Class diagram**
 - Define how they collaborate to fulfill requirements (dynamic model) → **Sequence diagram**
 - Applicable to various development process models.
 - **Waterfall**
 - **UP (Iterative, Agile)**
- But it is not enough, because OOAD involves **deep principles**.
 - Deciding what methods belong to where and how objects should interact carries consequences should be undertaken seriously.
 - **OOAD** should be a **dynamic model-driven approach** to avoid “**maintenance nightmare**”



*Maintenance
Nightmare*

An OOAD Example - Dice Game



Define use cases

Define domain
model

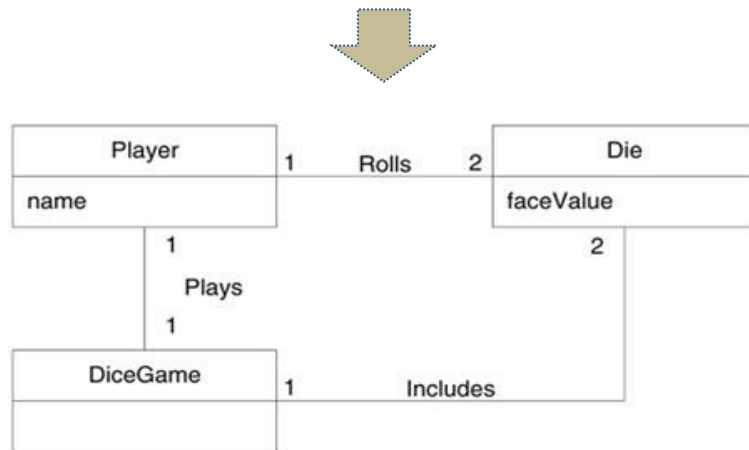
Define interaction
diagrams

Define design class
diagrams

OOA

Use Case : **Play a Dice Game**

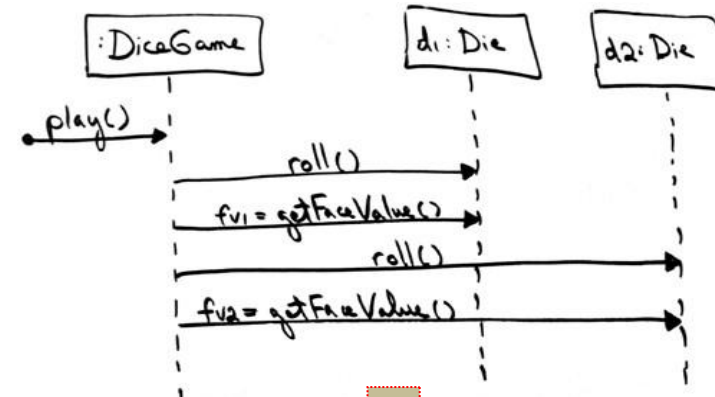
- Player requests to roll the dice.
- System presents results.
- If the dice's face value totals seven, player wins; otherwise, player loses.



Domain Model

OOD

Interaction Diagram (Sequence Diagram)



Dynamic model-driven !!!

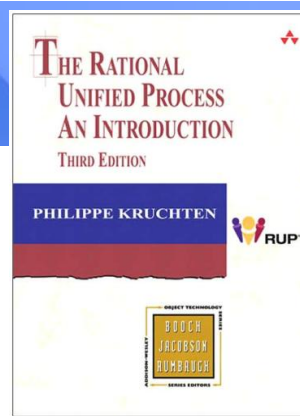
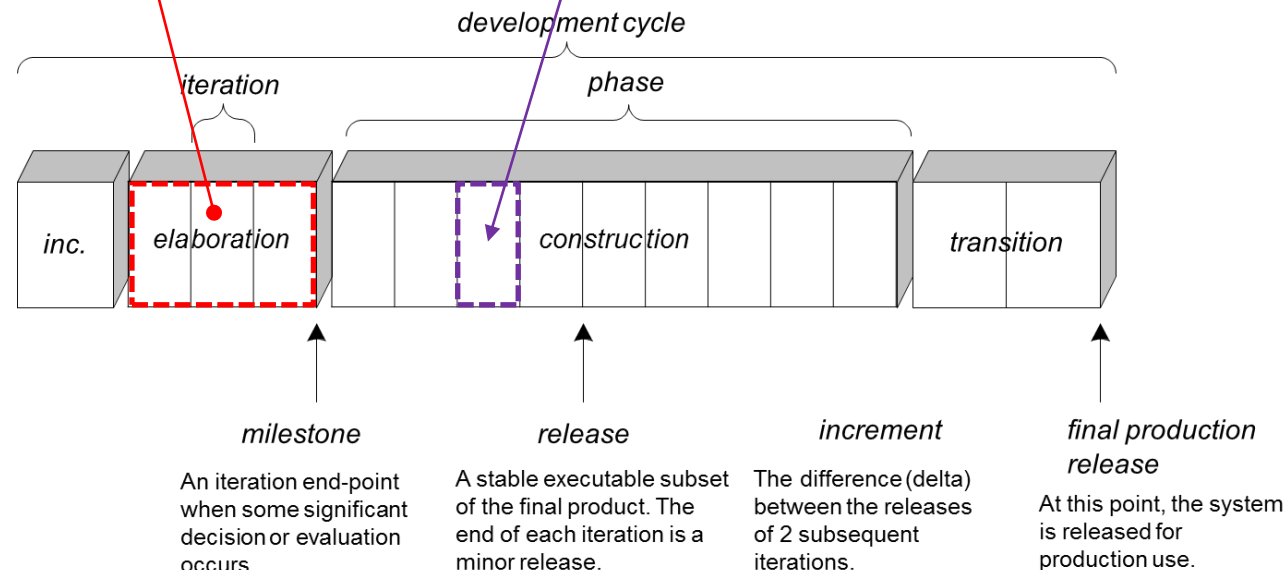
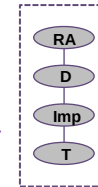


Design Class Diagram

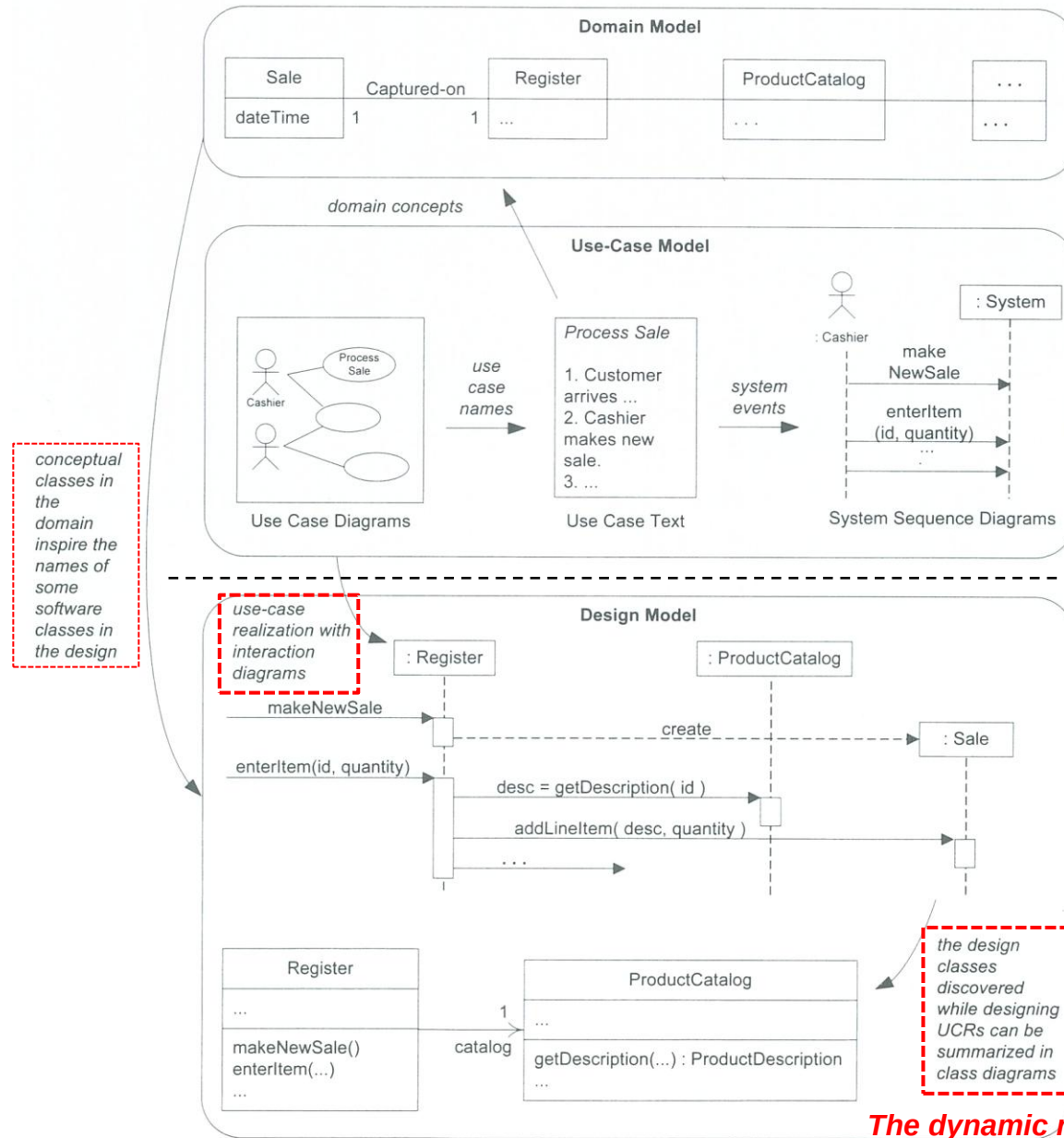
UP (Unified Process)

- **Rational Unified Process (RUP) or UP**

- A Software development approach that is
 - **Iterative** (Incremental, Evolutionary)
 - Each iteration includes a mini waterfall cycle of 3 ~ 4 weeks
 - **Risk-driven / Client-driven / Architecture-centric**
 - **Use-case driven**
- A Well-defined and well-structured software engineering process
 - **4 Phases** and **9 Disciplines**
- A de-facto industry standard for developing OO software



Sample Unified Process Artifact Relationships



Domain Model

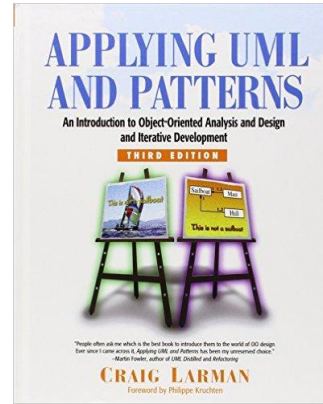
Use-Case Model
(+System Sequence Diagram)

OOA
OOD

Sequence Diagrams

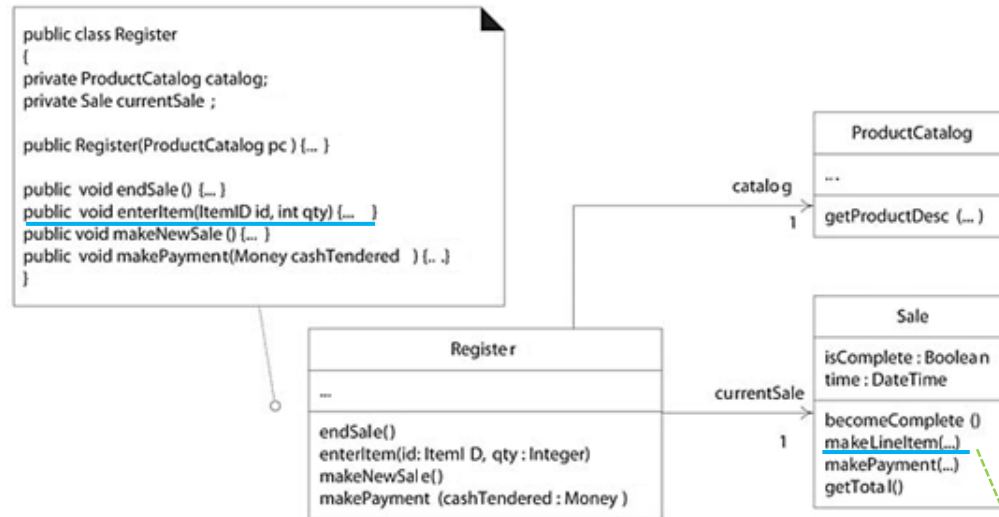
Class Diagram

The dynamic model-driven approach!!!

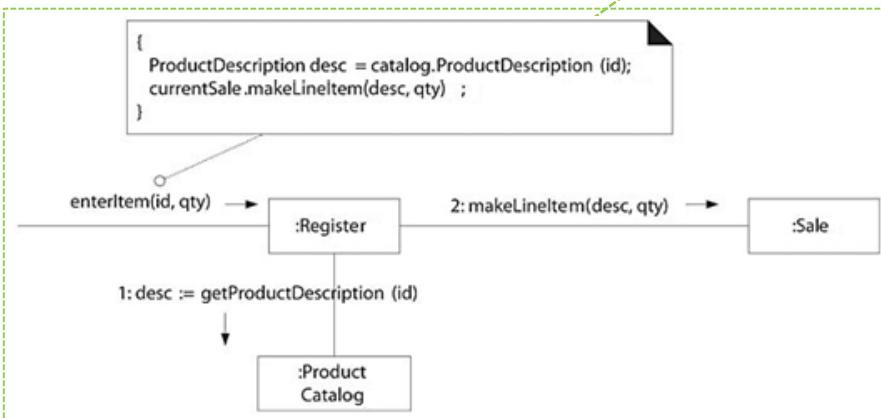


- **Sequence diagrams** have sufficient information to implement the body of the methods in Class diagram.
 - Dynamic Model-driven approach

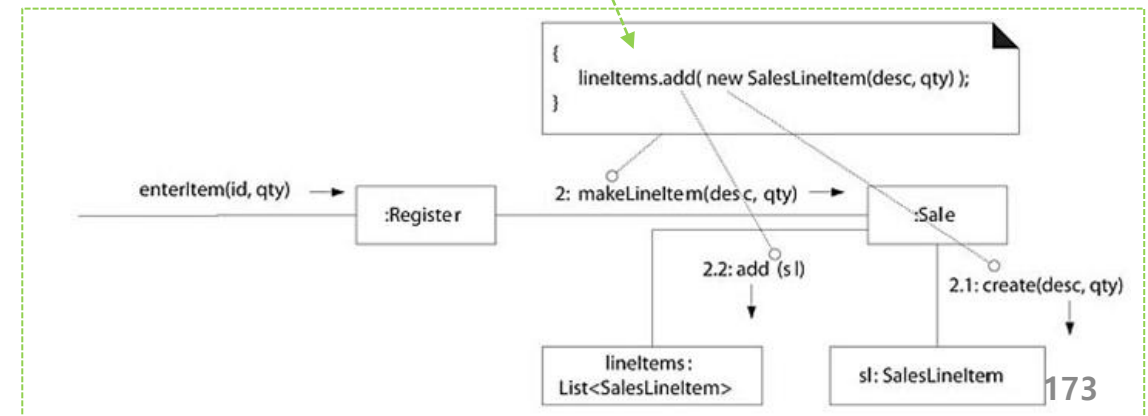
The Register.enterItem Method



enterItem()



makeLineItem()

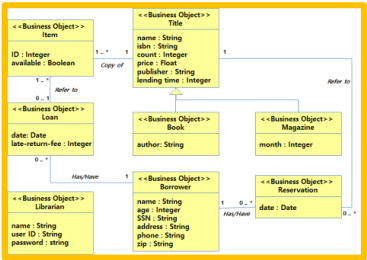


OOPT as OOAD

→ : Requiring Object-Oriented Thinking***

→ : Mechanical Transformation

→ : Mapping with a hint

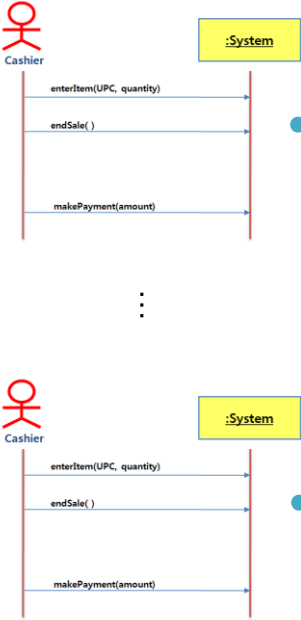


Domain Model

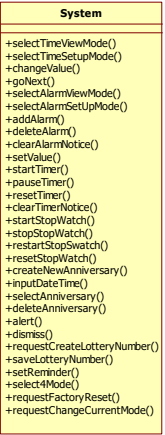
Functional Requirements	Use Cases	Category
R1.1 Make reservation	1. Make Reservation	Evident
R1.2 Remove reservation	2. Remove Reservation	Evident
R1.3 Lend Item	3. Lend Item	Evident
R1.4.1 Return title	4. Return Title	Evident
R1.4.2 Calculate Late-Return-Fee	5. Calculate Late-Return-Fee	Hidden
R1.5 Calculate Replacement Fee	6. Get Replacement Fee	Evident
R1.6 Notify Availability	7. Notify Availability	Hidden
R2.1 Add title	8. Add Title	Evident
R2.2 Remove title	9. Remove Title	Evident
R2.3 Update title	10. Update Title	Evident
R2.4 Add item	11. Add Item	Evident
R2.5 Remove item	12. Remove Item	Evident
R2.6 Update item	13. Update Item	Evident
R3.1 Add borrower	14. Add Borrower	Evident
R3.2 Remove borrower	15. Remove Borrower	Evident
R3.3 Update borrower	16. Update Borrower	Evident
R4.1 Validates system access	17. Log-In	Evident
R4.2 Validates system access	18. Log-Out	Evident
R5.1 Compute total # of items checked out	19. Count Loans	Evident

User (Functional) Requirements

Use Cases



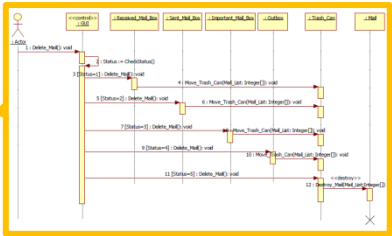
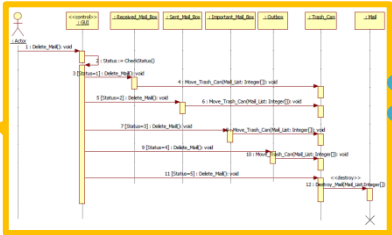
System Sequence Diagrams



System Operations

User Case	1. Make Reservation
Actor	Librarian
Purpose	(As in the business use case)
Overview	(As in the business use case)
Type	Primary and Essential
Cross Reference	System Functions R1.1, R2.1 Use Case: "Add Borrower"
Pre-Conditions	Borrower should type an ID card
Typical Courses of Events	(A) Actor ID: System 1. (A) Librarian requests the reservation of title 2. (A) Check if a corresponding title exists 3. (A) Check if a corresponding borrower exists 4. (A) If the borrower does not exist, create "Add Borrower" 5. (A) Create reservation information
Alternative Courses of Events	N/A
Exceptional Courses of Events	Line 1: If invalid reservation information is entered, indicate an error

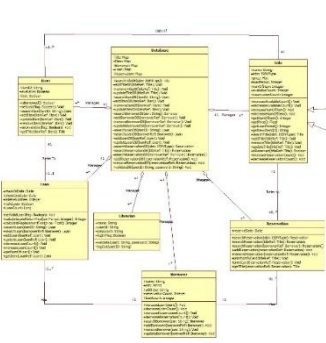
(Refined) Use Cases



Sequence Diagrams

Use Case	Requirement	Traceability
1. Make Reservation	R1.1 Make reservation	1. Make Reservation
2. Remove Reservation	R1.2 Remove reservation	2. Remove Reservation
3. Lend Item	R1.3 Lend Item	3. Lend Item
4. Return Title	R1.4.1 Return title	4. Return Title
5. Calculate Late-Return-Fee	R1.4.2 Calculate Late-Return-Fee	5. Calculate Late-Return-Fee
6. Get Replacement Fee	R1.5 Calculate Replacement Fee	6. Get Replacement Fee
7. Notify Availability	R1.6 Notify Availability	7. Notify Availability
8. Add Title	R2.1 Add title	8. Add Title
9. Remove Title	R2.2 Remove title	9. Remove Title
10. Update Title	R2.3 Update title	10. Update Title
11. Add Item	R2.4 Add item	11. Add Item
12. Remove Item	R2.5 Remove item	12. Remove Item
13. Update Item	R2.6 Update item	13. Update Item
14. Add Borrower	R3.1 Add borrower	14. Add Borrower
15. Remove Borrower	R3.2 Remove borrower	15. Remove Borrower
16. Update Borrower	R3.3 Update borrower	16. Update Borrower
17. Log-In	R4.1 Validates system access	17. Log-In
18. Log-Out	R4.2 Validates system access	18. Log-Out
19. Count Loans	R5.1 Compute total # of items checked out	19. Count Loans

Traceability Table



Class Diagram

```
class A
{
    variables:
    aaa()
    {
        aa();
        bb();
        cc();
        dd();
        ee();
        ff();
    }
    bbb();
    ...
}
```

Stage 1000.
Plan (Inception)

Stage 2030.
Analyze (OOA)

Stage 2040.
Design (OOD)

Stage 2050.
Construct (OOI)